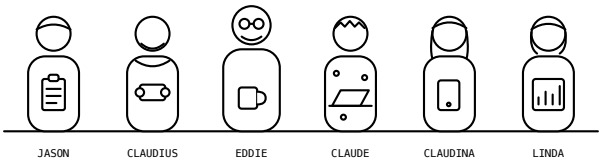
100 Rules for Writing My Software

A Field Manual for Directing AI Coding Agents



Ed Haynes

Contents

Foreword 7
How to read this book 8
Start with these ten 9
Meet the crew 10
Chapter 1 — First Principles 12
Rule 1: The Powell rule — 90% and decide 15
Rule 2: No scan, no ship 17
Rule 3: Never hardcode a secret 18
Rule 4: Distrust every external input 19
Rule 5: Destruction requires a human 20
Rule 6: No recoverable history, no autonomy 21
Rule 7: Push early and push always 22
Rule 8: Green before commit, healthy before handover 23
Rule 9: One purpose per commit 25
Rule 10: Fail fast 26
Rule 11: No anonymous dependencies 27
Rule 12: Assume no path, no OS, no shell — and no head 28
Rule 13: Least privilege by default 29
Rule 14: Five roles, one human, zero hardcoded models 31
Rule 15: Plan first 32
Rule 16: Claudius plans, or it's rework 33
Rule 17: Jason holds the through-line 34
Rule 18: Claude searches before he builds 35
Rule 19: Claudina ships everywhere 36
Rule 20: Linda searches wide 37
Rule 21: No flattery 38
Rule 22: Go local 39
Chapter 2 — Design 43
Rule 23: If it can change, it's config 47
Rule 24: Architecture beats language 48
Rule 25: Every vendor axis gets an interface 49
Rule 26: Search before you build 50
Rule 27: Collaborators come through the front door 51

Rule 28: No silent fallbacks 53
Rule 29: Validate at startup, name the key 54
Rule 30: One config layer, one precedence order 55
Rule 31: Objects with one job each 57
Rule 32: Storage goes through the adapter 58
Rule 33: On-prem and cloud are config values 60
Rule 34: "Use X locally" means default, not destiny 61
Rule 35: Zero-setup defaults 62
Rule 36: The file-size gauge 63
Rule 37: No god classes 65
Rule 38: One non-trivial class per file 66
Rule 39: Ship the `.env.example` 67
Rule 40: Refactors are mechanical commits 68
Chapter 3 — Build 71
Rule 41: Gates never disabled by default 74
Rule 42: Catch specific, never swallow 75
Rule 43: Pin it and lock it 77
Rule 44: Health endpoints and graceful SIGTERM 78
Rule 45: Container-friendly by default 79
Rule 46: Make it idempotent 81
Rule 47: Loud in dev, graceful in prod, diagnosable always 82
Rule 48: Three platforms, two in CI 84
Rule 49: A logger, never print 85
Rule 50: AI errors surface; agents don't invent tools 87
Rule 51: Path libraries, never string glue 88
Rule 52: Stdlib plus one good dependency 89
Rule 53: No hardcoded temp, home, or drive letters 92
Rule 54: Both architectures, flagged exceptions 93
Rule 55: No shell-isms — orchestrate in Python or Node 94
Rule 56: Time is UTC until it's displayed 95
Rule 57: Project-local virtualenvs, always 96
Rule 58: Show progress; cached by default, expensive by exception 97
Rule 59: Every remote call has a timeout and a way out 98

Rule 60: One request, one trace ID 99
Rule 61: Migrations go down as well as up 100
Chapter 4 — Protect and Prove 103
Rule 62: Hooks Before the First Commit 106
Rule 63: Rotate First, Clean History Second 108
Rule 64: Never Copy a Secret Anywhere 109
Rule 65: Scan every boundary before you cross it 110
Rule 66: You get what you inspect — and it gets a grade 114
Rule 67: No test, no ship 115
Rule 68: Contract first, code second 116
Rule 69: 100% — lines and branches 117
Rule 70: Correctness over speed 119
Rule 71: Coverage never goes down 120
Rule 72: Full regression, every feature, with the numbers . . 121
Rule 73: Set a latency budget and gate it 121
Rule 74: No network in unit tests 122
Rule 75: Fix the Hook That Missed It 123
Chapter 5 — Ship and Remember 126
Rule 76: Tags are carved in stone 129
Rule 77: Versions only move forward 131
Rule 78: Persist decisions in the same commit 132
Rule 79: One version, one home 134
Rule 80: File it the moment you see it 135
Rule 81: Every build wears a serial number 137
Rule 82: Verbatim errors, diffs not prose, assumptions out loud 140
Rule 83: The changelog rides in the bump commit 141
Rule 84: Plans tell you their own status 142
Rule 85: The version answers the door 143
Rule 86: Lint and format every commit 144
Rule 87: Sweep the dead code 145
Rule 88: After the release, take out the trash 146
Rule 89: Regenerate the README 147
Rule 90: No commented-out code 148

Rule 91: No orphan TODOs 149
Rule 92: A living state file so a cold agent can start 149
Chapter 6 — Operating an AI Fleet 153
Rule 93: Chunk it so the model nails it 154
Rule 94: The sizing law — one rule per billion 156
Rule 95: Slice the rules to the seat 158
Rule 96: Mind the context ceiling 159
Rule 97: Good enough at the model, 90% at the system 161
Rule 98: Match the work to the model 162
Rule 99: Verify, then escalate 165
Rule 100: Externalize the reflex 168
Chapter 7 — From a Hundred Rules to an Axiom Core 173
The hundred was never flat 174
The axiom core 174
The preference layers 175
Rules as a cache hierarchy 177
Guy's test — how small can the crew get? 180
Chapter 7 card 181
Back Matter 183
Companion media 183
Appendix A — Drop-in pre-commit config 183
Appendix B — Minimum `.gitignore` for secrets 184
Appendix C — Adopting the rules per harness 184
Appendix D — Book number ↔ repository `RULES.md` mapping 185
Appendix E — Bending the model: the three ways to retrain 190
Appendix F — The axiom core and the layers 192
Glossary 196

100 Rules for Writing My Software

First edition, 2026.

ISBN (paperback): 979-8-XXXXXXX-X-X *(assigned by KDP at publication)*

Published by Bard Technical Solutions.

This book is the author's personal work — unaffiliated with, not sanctioned by, and not endorsed by the author's employer. Everything in it is provided as is, with no promises of any kind. Your mileage may vary — do your own research.

Built with Bard — *LLM on the Go*: run local models on iPhone, iPad, and Mac. App Store: apps.apple.com/app/bard-llm/id6772813533

Watch and listen — the 100 Rules podcast: youtu.be/rmMGM460FMw

---

Foreword

These rules come from years of actual development experience — close to five decades of it, from university mainframes to defense networking to embedded real-time systems to enterprise open source. Most of them were paid for the expensive way: a leaked key, a moved tag, a thousand-line file nobody could review, a decision that lived only in somebody's head until that somebody left.

But this book exists because of something newer. AI changed the economics of software discipline. The messy work that made teams cut corners — merges, regression suites, the four-hundredth test case, regenerating a README from scratch — is exactly the work machines now do extremely well and without getting bored. Push early and always, because merge pain is no longer an excuse. Demand 100% branch coverage, because the patience to reach it is no longer human patience. Tailor sprints so the AI nails them first go nine times out of ten, and let it run those sprints in parallel.

Be clear about what that machine intelligence is, though, because it is lopsided. Out of the box, a coding model has been trained on most of the code ever published — and most of the code ever published is bad. Quantity, not quality: the average tutorial, the average abandoned repo, the average pull request that barely got reviewed. Left on its defaults, an AI assistant reproduces exactly that — functional, unremarkable, C-grade. Yet the same model is fantastic at reviewing code for issues, reorganizing code that already works, scanning for secrets, and untangling git merges. Mediocre author, excellent editor. The system in this book is built on that asymmetry: let the machine run the gates, the reviews, the scans, and the merges, and put rules around the part where it merely averages the internet.

Call it the three Cs. Algorithms was the only computer-science subject I ever got a C in. The most successful system I ever helped build — defense networking gear, certified to carry top-secret traffic, still in production thirty years later — was written in C, with an object-oriented architecture. And a coding model, out of the box,

writes C-grade code. All three are connected: that system did not succeed because anyone out-brillianted the problem. It succeeded because the architecture was right and the discipline held. I never got past a C in algorithms; I out-disciplined it. The machine can too. That is what the hundred rules are for.

You bend a model toward quality in one of three ways. Train one from scratch on code you trust — millions of dollars, nobody's first move. Tune open weights toward your standards — easier and cheaper than you think; Appendix E sketches the path. Or put standing rules in front of the model on every request — simple, immediate, and not foolproof, which is why so many of these hundred rules are gates and hooks that hold even when the prompt doesn't. This book is the third way, with the second as your upgrade path.

The rules are opinionated. Many people will disagree with some of them — that is what the license is for. Take what works, fork what doesn't.

How to read this book

There are exactly one hundred rules — never more. When a new rule earns a place, an old one is consolidated or retired. A rules document that only ever grows stops being read.

The hundred is not a flat list, though it reads like one until the last chapter. Underneath, it has a shape: a small immutable core every machine must carry, and layers of preference that get paged in only when a task touches them. Chapter 7 draws that shape; the first six chapters walk the rules in reading order, and the seventh shows how they fit on machines that can't hold all hundred at once.

The rules are arranged in seven chapters that build on one another, most fundamental first:

1. **First Principles** (rules 1–22) — the lines you never cross, and who decides.
2. **Design** (rules 23–40) — configuration and architecture: where decisions live.

3. **Build** (rules 41–61) — portable, loud-failing, lightly-dependent code.
4. **Protect and Prove** (rules 62–75) — secret hygiene and the quality bar.
5. **Ship and Remember** (rules 76–92) — versioned releases and written memory.
6. **Operating an AI Fleet** (rules 93–100) — making cheap models good as a system.
7. **From a Hundred Rules to an Axiom Core** — the hundred as a cache hierarchy: a small core that never leaves, the rest paged per seat.

Each chapter opens with a synopsis of the fundamentals it stands on, so you can start anywhere and know what is being assumed. Each chapter closes with a one-page card — its rules as a checklist, suitable for printing and taping to a monitor. Every rule gets a statement, the why (usually a scar), and a diagram where one earns its place. All diagrams are designed for black and white.

Start with these ten

One hundred rules is a reference, not a starting line. If you adopt nothing else this week, adopt these — they prevent the unrecoverable and build the habits the other ninety stand on:

1. **Rule 1 — the Powell rule.** Get 90% of the information, then decide; below 90% certain, ask — never guess ahead, never stall past 90%. It's first because it's the rule you apply every other rule through.
2. **Rule 2 — scan before every boundary.** A secret scan before every commit, push, and deploy. No scan, no ship.
3. **Rule 3 — never hardcode secrets.** Found one ⟶ stop and flag it; never propagate it, even temporarily.
4. **Rule 5 — never destroy without confirmation.** No deletes, drops, force-pushes, or history rewrites on an agent's own judgment.

5. **Rule 7 — push early and always.** Unpushed work is a liability; the remote is the backup, and AI killed the merge-pain excuse.
6. **Rule 8 — green before commit, healthy before handover.** Never commit on red; never present a service as done without watching it answer.
7. **Rule 9 — one purpose per commit.** No "while I'm in there" fixes.
8. **Rule 10 — fail fast.** Crash loudly at startup with a clear message; never limp along degraded.
9. **Rule 23 — if it can change, it's config.** Zero hardcoded hosts, ports, models, paths, or timeouts.
10. **Rule 62 — hooks before the first commit.** Pre-commit secret scanning goes in before any code does; gates beat memory.

This ten is the author's judgment — the decision rule first, then a weighting toward irreversible damage. It has a successor: audit enough real sessions against the rules (the tooling ships with the companion repo) and the violation histogram produces a *measured* ranking. When the data disagrees with this list, the data wins. (The companion repo's `RULES.md` orders the same hundred by topic; Appendix D maps the numbering.)

Meet the crew

The rules assume a team of five AI personas plus one human. The roles are fixed; the model behind each is configuration, never hardcoded — the same crew works on a Claude stack, an open-model stack, or fully local machines.

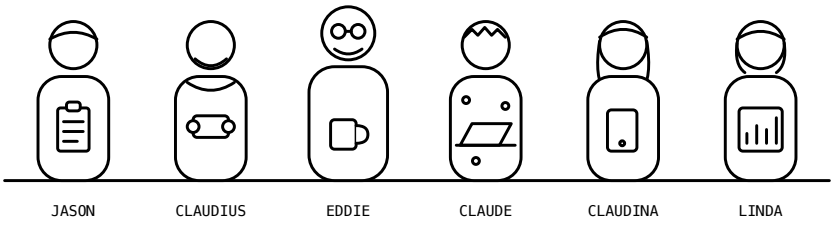
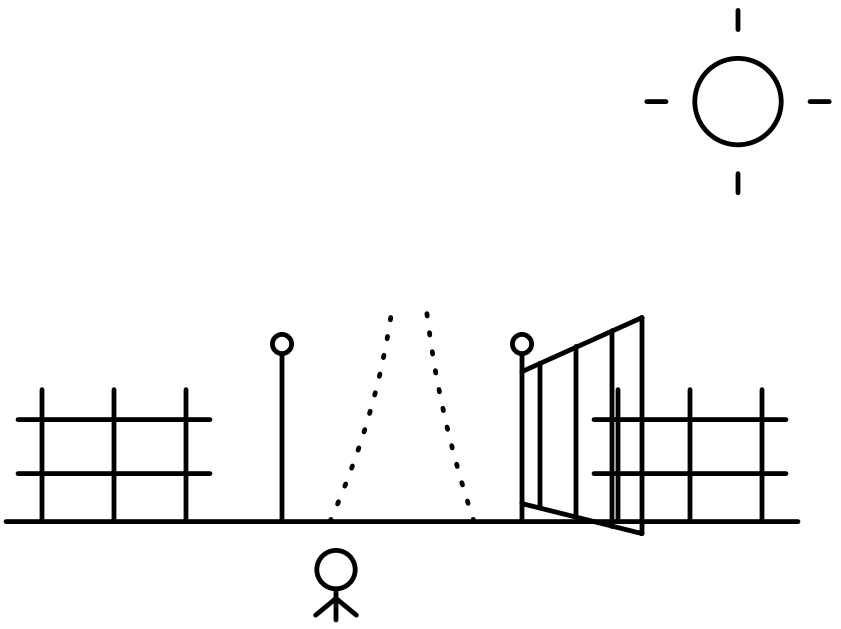


Figure 1: The crew. Five fixed roles, one human whose rulings are final.

- **Jason** — project manager. Fast and decisive; chunks work into independent sprints sized for 90% first-try success, runs the heavyweights in parallel, doesn't write code.
- **Linda** — research manager. Searches wide and fast; breadth first, depth on request.
- **Claude** — backend developer. Slow, methodical; finds the high-star open-source project before writing a line of original code.
- **Claudina** — frontend developer. Windows, macOS, iOS, Linux — or it doesn't ship.
- **Claudius** — architect. Thinks long and deep; if architecture needs rework, his plan was wrong.
- **Eddie** — the human. His rulings are final and canonical. (Substitute your own name; the principle stands.)

Chapter 1 introduces them properly, along with the rule that governs every decision any of them makes: get 90% of the information you need, then decide — and below 90% certainty, ask the human. We call it the Powell rule.

Chapter 1 — First Principles



> **The five fundamentals:** - Some rules are never broken — scan before every boundary, never hardcode secrets, distrust every external input, never destroy without confirmation, green before commit, push early and always, least privilege by default. - A crew of five fixed roles plus one human decides, governed by the Powell rule. - Every decision that can change lives in config, behind an interface — architecture over language. - Code runs on any platform, fails loudly, and depends on as little as possible. - Protect the secrets, prove the quality (90% working bar, 95% to publish), ship versioned, write everything down.

Before any code gets written, one rule sits above all the others — the rule you use to apply every other rule. *Get 90% of the information you need, then decide.* When to act, when to ask, when you have gathered enough: that judgment fires hundreds of times a day, and getting it consistently right is what separates a crew that ships from one that either guesses or stalls. It is Rule 1 because everything after it is a decision made under it.

Then two things have to be settled, and neither is a technical question. The first: which lines do we never cross? The second: who decides what, and when does a decision become final? Get those two wrong and no amount of clean architecture saves you. Get them right and everything else in this book has something to stand on.

Start with the lines. I have been writing software for about forty-seven years — mainframes, defense networking gear, embedded real-time systems, lately enterprise open source — and in all that time I have never once regretted following a hard rule. I have a small museum of scars from the times I didn't. The asymmetry is the whole argument: skipping a secret scan saves you forty seconds, and a leaked key costs you a rotation, a history rewrite, an audit of every repo the same agent touched that day, and maybe a cloud bill run up by a stranger mining cryptocurrency on

your credentials. Skipping the "are you sure?" before a destructive command saves three seconds; a dropped production table costs a restore from backup, if you have one. The downside is three to six orders of magnitude bigger than the upside, every time. Gates with that cost profile should never be skippable by mood — and mood is exactly what you'll be running on, because nobody leaks a key on a quiet Tuesday morning. They leak it at 11 p.m. before a demo, "just this once." A hard rule is a decision you make once, calmly, on behalf of your future panicked self. That's why those twelve hard rules don't bend. The other rules bend to judgment; these prevent failures that are irreversible. You can refactor a long file next sprint. You cannot un-push a secret.

Now the second question: who decides. I spent my first decades on teams where the roles were carved in stone — the program manager did not write code, the architect did not negotiate schedules — and when those boundaries held, products shipped. When they blurred, we got slipped dates and architectures shaped by the last fire drill. AI-assisted development has the same failure mode, just faster: one undifferentiated assistant asked to do everything behaves like the overloaded engineer, planning a little, coding a little, second-guessing its own architecture mid-function. The fix is the one human organizations discovered a century ago — separation of concerns, applied to cognition. Give each role a name, a temperament, and a boundary; bind each to the engine its job actually wants; put one human at the top whose rulings are final. That's the crew, and it occupies the rest of this chapter — each role working under the decision doctrine stated up front in Rule 1, the protocol every other rule in this book is applied through.

Doctrine first, then lines, then roles — the doctrine governs how every other rule is applied, and the lines apply to every agent, human or AI; an AI agent works faster than you can review, which makes its gates matter more than yours, not less. Once all three are in place, the rest of the book builds upward in order: principles, then design, then build, then protect and prove, then ship and

remember, then operating a fleet of these agents at once — and last, the shape of the hundred itself, the small core that never leaves a machine and the rest that pages in when a task reaches for it. Each chapter assumes the ones before it. This one assumes nothing. That's what makes it first.

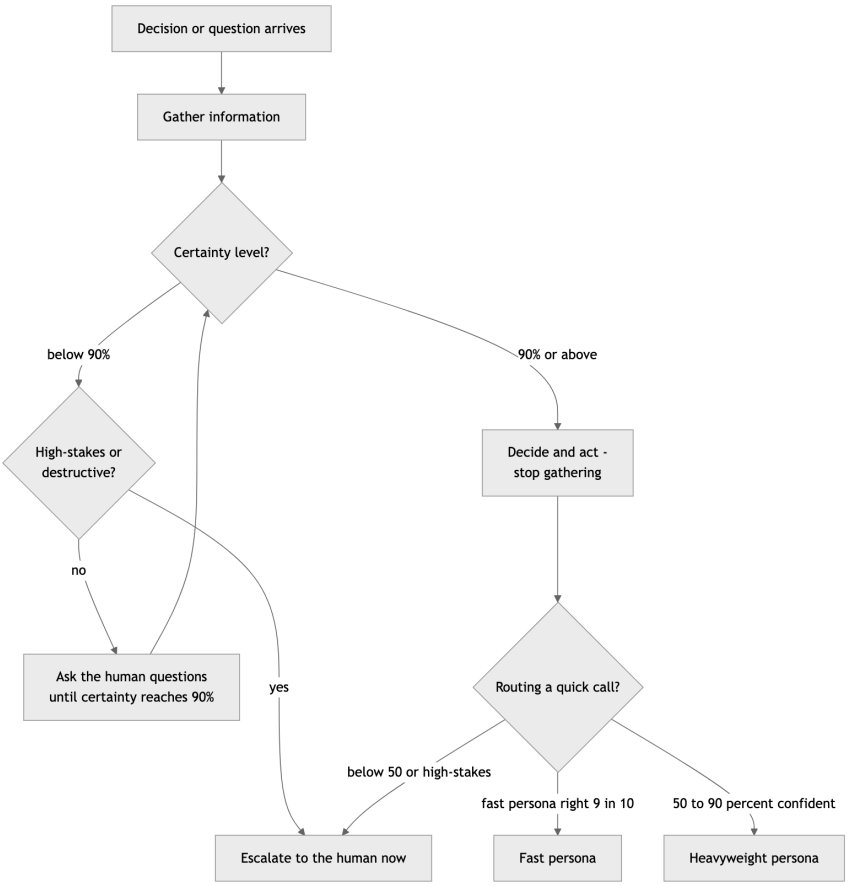
Rule 1: The Powell rule — 90% and decide

Get 90% of the information you need, then make the decision. Below 90% certain? Ask the human more questions until you get there — never guess ahead, and never stall gathering past 90%. Apply the same 90% bar to routing: a quick call goes to a fast persona only when it will get it right nine times in ten; 50–90% goes to a heavyweight; below that, or anything high-stakes, goes to the human.

This rule is first because it is the rule you use to apply every other rule in the book — the standing decision protocol behind every commit, every scan, every escalation. It borrows from a doctrine attributed to a famous American general and statesman: decide when you have 40 to 70 percent of the information you could get — too little and you're guessing, too much and you're late. My version turns the dial to 90, because the economics changed: his staff officers were expensive and slow; my crew gathers information at machine speed and near-zero cost. When information is cheap, the optimal stopping point moves up. But not to 100 — the last few percent of certainty cost more than they're worth, and a crew that won't act below total certainty never acts.

The rule has two halves. The routing half is triage by confidence: a question a fast persona will get right nine times in ten goes to the fast persona. The 50–90% band goes to a heavyweight. Below 50%, or anything where being wrong is expensive regardless of probability — destructive operations, money, security, anything in the hard rules (Rules 2–13) — goes to me. Probability times consequence, not probability alone.

The crew-wide half kills the two failure modes that bracket good judgment. Guessing ahead below 90% produces confident wrong answers — the worst output an AI can give you, because they read exactly like confident right ones. Gathering past 90% produces stalls dressed up as diligence. The escape valve below 90% isn't more searching; it's *asking me questions*. A thirty-second question beats a thirty-minute rewrite, every time.



The Powell rule: gather to 90%, then act. Below 90%, questions to the human — not guesses, not endless searching.

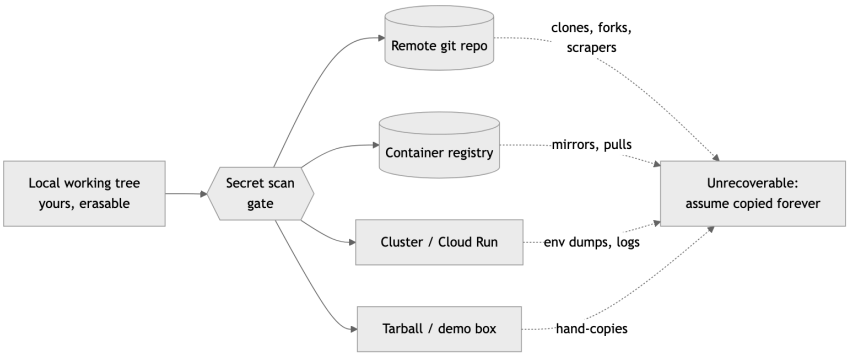
Rule 2: No scan, no ship

Run a secret scan before every commit, push, and deploy. No scan, no ship — ever, on any agent, to any target.

This rule is first because it guards the only mistake in software that cannot be undone by typing harder. A bug ships, you patch it. A bad design ships, you refactor it. A secret ships, and it is *gone* — copied by every clone, every fork, every registry mirror, every scraper that watches public commits for key-shaped strings (and they watch in close to real time; a key pushed to a public repo gets probed in minutes, not days).

The how is mechanical, which is the point: gitleaks in a pre-commit hook, the same scan again pre-push over the whole push range, and a scan of the full artifact before any deploy. Three gates, all automated, none of them asking your opinion. If the scan finds something, you stop. Not "stop unless the demo is in an hour." Stop. A finding that can't be cleared — false positive documented, secret rotated, file scrubbed — blocks the ship.

The trap this rule exists to close is the *trust boundary* you didn't notice crossing. Your working tree is yours; everything past it isn't. The remote repo, a container registry, a cluster, a tarball handed to a colleague, a "quick demo box" — each is a host you don't control with a memory you can't erase.



Everything to the right of the gate is a trust boundary: once a secret crosses, assume it has been copied forever.

The failure mode is always the same sentence: "it's just a private repo." Private repos get forked, made public, cloned to laptops that get stolen. The scan takes forty seconds. Run it.

Rule 3: Never hardcode a secret

Never hardcode secrets, API keys, tokens, passwords, or private endpoints. Find one already in the codebase → stop and flag it; never propagate it, even temporarily.

Rule 1 is the gate; this rule keeps contraband from approaching the gate at all. A secret that never enters a source file never has the chance to slip through a scanner's blind spot — and every scanner has blind spots. Gitleaks knows what an AWS key looks like; it does not know that `db_host_2` is the IP of a box that should never appear in public.

The "how" is the config layer: secrets arrive through environment variables, mounted files, or a secrets manager, and the repo carries only a `.env.example` documenting what's needed. That's covered in detail later in the book; the hard-rule part is the second sentence. When you *find* a secret already sitting in the codebase — and you will, usually in a yaml file committed by someone in a hurry — the rule is stop and flag, never propagate. Don't copy it into your own branch "while we sort it out." Don't paste it into chat to ask if it's real. Don't move it to a "temporary" scratch file. Every copy is a new leak surface, and the half-life of "temporary" in software is roughly forever.

This rule matters double in the AI era, and here's the part that took me a while to internalize: an AI agent is a champion propagator. It reads the codebase for patterns and reproduces what it sees. One hardcoded key becomes the template for the next five files the agent writes. The agent isn't malicious; it's *consistent*, which is worse. A human might feel a twinge copying a credential. The agent feels

nothing. So the rule is absolute for agents precisely because their judgment never kicks in: found a secret, stop, surface it to the human, touch nothing.

The failure this prevents is the slow-motion one: a private endpoint hardcoded in 2024, copy-pasted into six services by 2026, and unrotatable by the time anyone notices because nobody knows all the places it lives.

Rule 4: Distrust every external input

Distrust every external input. Validate and constrain it at the boundary; parameterize queries and commands, resolve and confine paths, never interpolate untrusted data into SQL, a shell, HTML, or a deserializer. Secret hygiene guards what leaks out — this guards what gets in.

Rules 1 and 2 guard what leaves the building. This one guards what walks in the door, because the oldest exploits in the book all share one shape: data that was supposed to be a value got treated as code. A name field that was really a `DROP TABLE`. A filename that was really `../../etc/passwd`. A serialized object that was really a constructor for a reverse shell. Nobody attacked the logic; they attacked the seam where untrusted text became trusted instruction.

I have cleaned up after this exactly once with my own name on the commit, and once is the number that teaches you. A reporting endpoint built a query by pasting a parameter straight into the SQL string — the fast way, the obvious way, the way that demos perfectly. It worked for everyone who typed a normal value and surrendered the whole table to the first person who typed a quote and an `OR 1=1`. The fix was four characters: a bound parameter instead of a concatenation. The lesson was bigger than four characters.

The discipline is mechanical, which is the point — judgment fails under deadline, mechanism doesn't. Parameterize every query; never build SQL or a shell command by string-pasting. Resolve and

confine every path to its intended root before opening it. Treat deserialization of untrusted bytes as the loaded gun it is. Validate at the boundary, once, where the data arrives — not scattered through the call stack hoping someone downstream checks. An AI agent will reproduce whatever input-handling pattern it sees, so the pattern it sees had better be the safe one.

The seam where data becomes instruction is the seam attackers own. Bind it shut.

Rule 5: Destruction requires a human

Never delete files, drop tables, run destructive shell commands, force-push, or rewrite history without explicit human confirmation.

There is a category of command whose undo button is a backup tape, and the category deserves a category-level rule. `rm -rf`, `DROP TABLE`, `git push --force`, `git filter-repo`, deleting a branch, moving a tag — these don't fail loudly when they're wrong. They succeed loudly. The damage report comes later, from someone else.

I once watched a cleanup script — mine, decades ago, I'll own it — interpret an unset environment variable as an empty string and recursively delete from a path two directories higher than intended. The script worked perfectly. The variable was the bug. Total time to type the command: two seconds. Total time to recover: most of a week, because the backups were good but not *that* good. Every gray-haired engineer has a version of this story, and the stories all share a shape: the destructive command was routine, the operator was confident, and the precondition was wrong.

The rule's mechanism is friction, applied surgically. Not friction on everything — that just trains people to click through — friction on the irreversible. A human must say the actual words: "yes, force-push," "yes, drop it." For AI agents this is non-negotiable in the strictest sense: an agent may *propose* destruction, with the exact command and the expected blast radius spelled out, but the confir-

mation must come from a person, in the current conversation, naming the specific act. A "yes" from twenty minutes ago does not transfer.

The failure mode prevented isn't malice and it isn't even incompetence. It's confidence plus a wrong assumption, moving at the speed of a shell. The confirmation step exists to make the assumption visible for one breath before it executes. One breath is usually enough.

Rule 6: No recoverable history, no autonomy

Agent autonomy is bounded by version control: an agent only writes inside a git repo with a synced remote. No recoverable history, no autonomy.

This is the newest of the hard rules and the one I'd argue is the most important sentence in this half of the chapter for the AI era, because it answers the question everyone is quietly negotiating right now: how much can you let an AI agent do *unattended*?

My answer isn't a trust score or a capability tier. It's a property of the workspace. An agent operating inside a git repo whose remote is synced can be wrong at full speed — every write it makes lands on top of a recoverable history, every mistake is a `git revert` or a `git reset` away from undone. The safety net isn't the agent's judgment; it's the substrate. Inside that boundary, run unattended, skip the permission prompts, let it work. The same agent writing *outside* a versioned repo — a home directory, a system config, an unversioned scratch folder — is performing surgery with no undo, and gets no autonomy at all. Read anywhere; write only where history protects you. If a task needs a write outside the boundary, the agent stops and asks a human. Not because the write is necessarily dangerous — because it's *unrecoverable if* dangerous, and the agent can't reliably tell the difference.

What I like most about this rule is its shape: it doesn't try to make the agent smarter or the human more vigilant, the two approaches

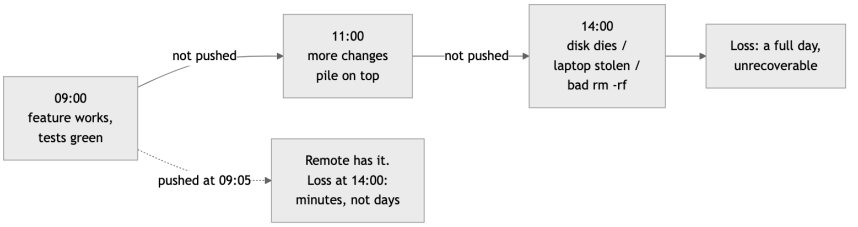
that consistently fail. It bounds the blast radius structurally, the way the rest of these hard rules bound it procedurally. Version control was always the profession's time machine. This rule just notices that a time machine is exactly what makes delegation safe — and that the synced remote is what makes the time machine real. Local-only history dies with the disk. The remote is the net under the net.

Rule 7: Push early and push always

Working code lands on main frequently. Uncommitted, unpushed work is a liability — the remote is the backup. The classic excuse (merge pain) is gone: AI handles messy merges extremely well.

Of the hard rules, this is the one that has actually *changed* in the last few years, so let me give the old version first. The old discipline said: commit often locally, but batch your pushes, because integrating early means merge conflicts and merge conflicts eat afternoons. That advice was rational once. It is now obsolete, because the messy mechanical merge — the thing we all dreaded — is precisely the kind of work AI does extremely well. The cost side of the trade collapsed. Only the benefit side remains.

And the benefit side was always this: unpushed work doesn't exist. The remote is the backup. A laptop is a single point of failure with a battery, and a working tree is one `rm`, one disk failure, one stolen bag away from never having happened. Every hour that working code sits only on your machine, you are running an uninsured risk for zero return.



Solid path: the liability compounding hour by hour. Dashed path: what a five-minute push buys you.

The rhythm I want: a change works, its tests pass, the scan is clean — it goes to main, now. Not at end of day, not "with the next thing." If a session ends with working code unpushed, that's a process failure to flag, not a state to leave behind. I say push early and always — your mileage may vary, but I've lost work and I've never lost a push.

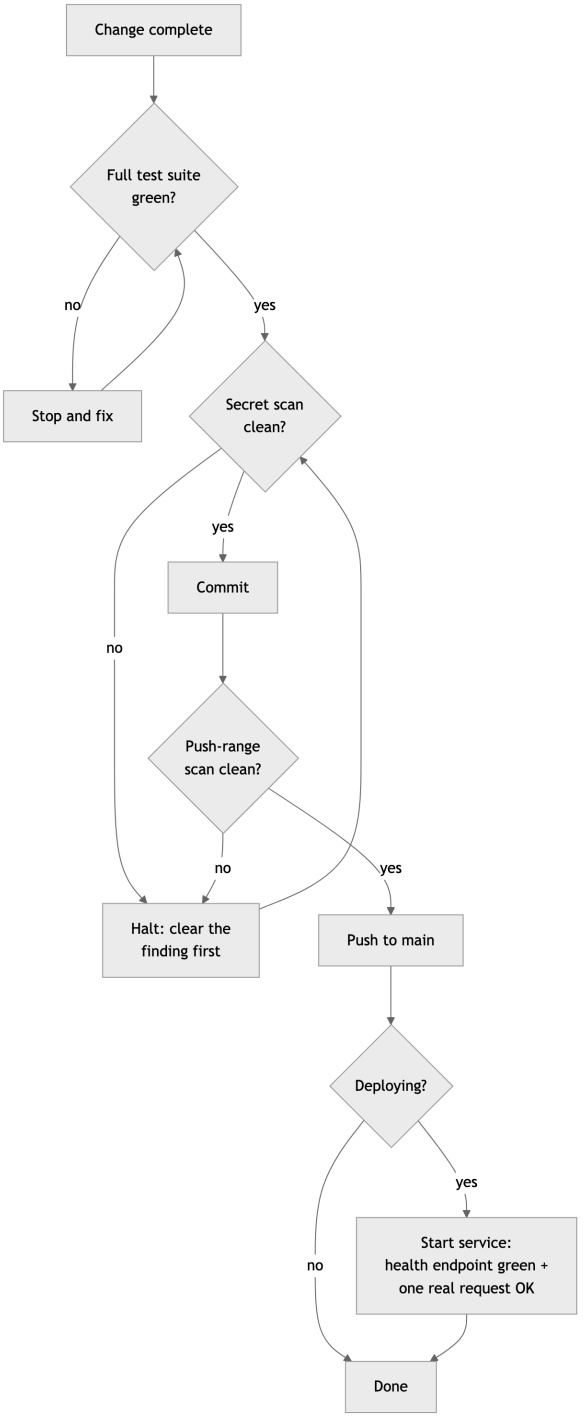
Rule 8: Green before commit, healthy before handover

Never commit while tests fail, and never present a service as done without verifying it is up, healthy, and answering a real request.

Two clauses, one principle: *checked* is a different state from *believed*, and only checked things move forward.

Green before commit is the simpler half. A failing suite is a stop-and-fix, never a "commit now, fix later" — because "later" arrives with three more commits stacked on the red one, and now the bisect is archaeology. Every commit on main answers one question the same way: did everything pass? If the answer is ever "mostly," the history stops being a record and becomes a rumor.

Healthy before handover is the half people skip, and it's the half with my favorite failure mode in it. A build script ends in `make build | tee build.log`, the pipe exits `0` because `tee` succeeded, and "the build passed" enters the record as fact. The build failed. The exit code you trusted belonged to a different program. I have seen variations of this lie — always accidental, always confident — for four decades. The fix is to never report health you didn't observe: start the actual service, hit the actual health endpoint, send one representative real request, and watch it succeed. Then say it's done.


Change complete
Full test suite green?
no
yes
Stop and fix
Secret scan clean?
yes
Commit
no
Push-range scan clean?
no
yes
Halt: clear the finding first
Push to main
Deploying?
yes
no
Start service: health endpoint green + one real request OK
Done

The gate chain. No edge skips a diamond; "mostly green" is not an exit condition.

For AI agents the rule is load-bearing: an agent that reports unverified success poisons the next decision. Verify, then claim.

Rule 9: One purpose per commit

One purpose per commit, one purpose per deploy. No "while I'm in there" fixes; a size or mechanical refactor is its own commit so the diff stays reviewable.

"While I'm in there" is the most expensive phrase in software. It feels like efficiency — the file is open, the fix is obvious, why make a second trip? Here's why. Every operation you'll ever perform on history operates on commits as atoms. Revert a commit: everything in it reverts together. Bisect to a commit: everything in it is the suspect together. Review a commit: everything in it must be understood together. A commit that does two things makes every one of those operations worse, forever, for everyone downstream — to save its author one context switch, once.

The deploy version of the rule is the same logic with higher stakes. A deploy that ships one change has a one-line rollback story. A deploy that ships a feature, two fixes, and a dependency bump ships four suspects, and when the pager goes off, you get to interrogate all of them at 2 a.m.

The how: when you notice the unrelated problem — and you will, constantly; noticing things is the job — you *file it*. A line in the bug tracker, thirty seconds, and back to the purpose at hand. The fix becomes its own commit, with its own test, on its own merits, usually within the day. Nothing is lost. What's gained is a history where `git log --oneline` reads like a narrative instead of a junk drawer.

Discipline note for AI agents, who are the worst offenders I've ever worked with: an agent told to fix a bug will cheerfully also reformat the file, rename two variables, and "improve" an unrelated function

— because it can, and it can't feel the reviewer's pain. The rule exists so the diff contains the change and nothing else. If X isn't a literal blocker for the stated task, X waits.

Rule 10: Fail fast

Invalid config, missing dependencies, or unreachable backends crash loudly at startup with a clear message — never limp along degraded.

This one comes straight from my embedded years, where the principle had a sharper name: a system that fails *predictably* is safe to build on, and a system that fails *gracefully but silently* is a trap with good manners. In real-time work you learn fast that the dangerous component isn't the one that crashes — you can see a crash, route around it, fix it. The dangerous one is the component that keeps answering with degraded data while looking healthy. Everything downstream builds on the lie.

The software version: a service starts with a bad database URL. Option one, it crashes at startup with `FATAL: config key DB_URL invalid: connection refused to db.internal:5432`. Thirty seconds to diagnose, fixed before coffee. Option two — the "resilient" option — it logs a warning nobody reads, falls back to a local cache, and serves stale data for three weeks until a customer asks why their numbers are wrong. Now the diagnosis starts from the *symptom*, miles from the cause, and the incident review uses the word "silently" four times.

The how: validate all config at startup, fail with a message that names the exact key and what's wrong with it. Check that backends are reachable at startup or first use. Never substitute a fallback backend without being explicitly configured to — a silent fallback is a lie about what's running. And make the crash *informative*: the goal isn't drama, it's a message that points a tired human directly at the fix.

The failure prevented is the worst kind in distributed systems: the slow, plausible, compounding wrongness that no alert fires on. A loud crash at startup is not a failure of robustness. It is robustness — moved to the only moment it's cheap.

Rule 11: No anonymous dependencies

Never add a dependency without stating its name, purpose, license, maintenance status, and platform support.

Every dependency is a hire. You're bringing someone else's code into your process space, granting it your permissions, your network access, your users' data — and, increasingly, your build pipeline, which is where the supply-chain attacks of the last decade actually landed. You wouldn't hire an engineer without asking their name and what they're for. The bar for a package should not be lower than the bar for a person.

The five questions are deliberately cheap to answer. *Name* — so it's in the record and the human saw it go in. *Purpose* — so we can ask whether twelve lines of stdlib would do instead; the answer is yes more often than anyone admits. *License* — because a copyleft license in a proprietary product is a legal problem you find out about at the worst possible moment, usually during an acquisition. *Maintenance status* — because a package whose last release was three years ago is a liability with a countdown on it; an abandoned dependency is one CVE away from being your problem with nobody upstream to fix it. *Platform support* — because I target arm64 and x86_64 on three operating systems, and a dependency with no ARM build is a workaround I'll be documenting and maintaining for years. Five minutes of stating the answers beats five months of living with the wrong ones.

The AI angle is again the multiplier. An agent asked to solve a problem will happily `npm install` its way there, because that's what the training data does. Left ungated, you get a lockfile with nine hundred transitive entries and no human who can say why

any of them are present. The rule forces a pause: every addition is announced, every announcement is a chance to say no.

The failure prevented: the 3 a.m. vulnerability advisory for a package you didn't know you depended on, doing something you didn't need, under a license you never read.

Rule 12: Assume no path, no OS, no shell — and no head

Never assume a path, OS, or shell — use cross-platform primitives. Assume everything is headless, and script everything.

This is the hard-rule version of a whole chapter that comes later, and it earns its place here because path assumptions are the most *contagious* class of bug I know. One hardcoded `/tmp` works fine for months — right up until the code runs in a container with a read-only root, or on Windows, or under a service account whose temp directory is somewhere else entirely. Then it fails somewhere far from the assumption, in code nobody thinks to look at, because "it's just a path."

The how is boring and absolute: the language's path library (`pathlib`, `path`, `filepath`), never string concatenation, never a hardcoded separator. Platform APIs for temp and home directories, never `/tmp` or `~/`. Orchestration in Python or Node, not in bash with its `&&` chains and `source` and process substitution — because bash isn't where your code will always run, and the day it runs somewhere else should be a non-event.

The same discipline extends two steps further. **Assume everything is headless**: no display, no browser, nobody sitting at an interactive prompt. A setup that needs a dialog clicked or a question answered at the terminal is a setup that fails in a container, in CI, on a server — and, the case that now dominates, under an AI agent, which is headless by nature. Every tool gets a non-interactive path, or an agent can't run it and neither can the machine that reboots at 3 a.m. And **script everything**: a procedure you perform by hand

— provisioning a box, building a model, restoring a service — lives in your fingers, and your fingers are not version-controlled. The day the workstation gets reimaged, the manual procedure is gone; the script is a `git clone` away. One command from bare metal to running service is not a luxury — it is what makes recovery fast enough to matter.

I spent years in embedded systems where "the environment" was whatever the board gave you, and the habit it beat into me was: the platform is an input, not a constant. Code that encodes assumptions about its host is code with an expiration date you can't read. The forty-seven-year version of this lesson is that every environment I ever treated as permanent is now a museum piece — and the code that survived each transition was the code that never asked what it was running on.

The failure prevented is the port that should have been trivial and wasn't: the "Linux-only for now" service that takes a quarter to move because ten thousand small assumptions have to be found one stack trace at a time. Write it portable on day one. Day one is cheap.

Rule 13: Least privilege by default

Least privilege by default. Every credential, token, service account, and role gets the narrowest scope that works — no wildcard permissions, no shared admin keys; expire and rotate by default, and grant more only when something actually fails for lack of it.

The hard rules so far have been about keeping the secret out of the diff. This one is about what the secret can *do* once it exists — because a credential that leaks is a credential plus a blast radius, and the blast radius is the part you control. A scoped read-only token in the wrong hands is an incident report. A wildcard admin key in the wrong hands is a new job search.

The scar here is the convenient one, the one everybody has: a service needs to read one bucket, and somewhere under deadline it gets handed a credential that can read, write, and delete *every* bucket in the account — because the broad grant was faster to type and it made the failing call go green. It works. It keeps working. Then one day that token shows up in a log, or a pod, or a leaked env dump, and the question stops being "what could the attacker read" and becomes "what couldn't they." The grant that saved ninety seconds defined the size of the eventual breach.

So the default runs the other way. Start a credential at the narrowest scope that lets the task pass, not the widest that's convenient. No shared admin keys passed around like a house key — one identity, one job, one scope. Set expiry and rotation as the default state, not a someday-cleanup. And widen only on a *real* failure: grant the next permission when something actually breaks for lack of it, never preemptively "to be safe." Preemptive breadth is the opposite of safe. For AI agents this is non-negotiable — an agent handed admin will use admin, cheerfully, for a job that needed read-only, because broad is the path of least resistance and the agent feels no flinch at holding the master key.

The grant that saves ninety seconds defines the size of the eventual breach. Make it small.

Rules 2–13 drew the lines; they say nothing about who works inside them. Rules and gates don't run themselves — somebody plans, somebody builds, somebody decides, and "one undifferentiated assistant does everything" is how the gates get skipped under pressure. So this stretch of the chapter names the workers: five fixed roles and one human, all operating under the decision doctrine of Rule 1.

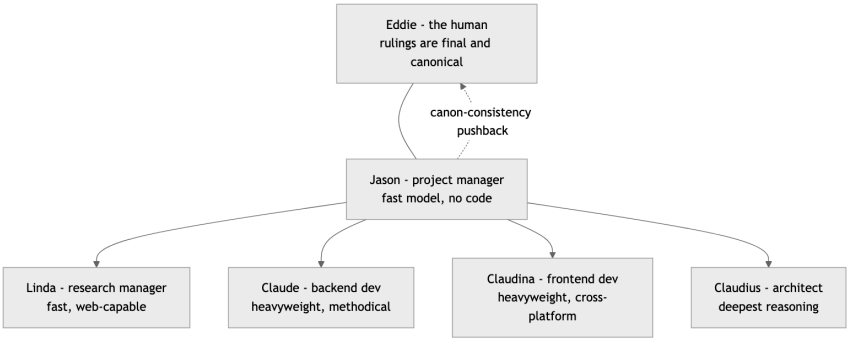
Rule 14: Five roles, one human, zero hardcoded models

The crew is five fixed roles plus one human, Eddie, whose rulings are final and canonical — every persona's plan or pushback yields to his decision, and his decisions join the canon. The one exception: Jason is expected to push back when a new ruling contradicts the canon, surfacing the inconsistency before acting on it. (Adopters: substitute your own name.) The model behind each role is a config binding per stack, never hardcoded.

The roles are fixed because a team you re-org every sprint isn't a team — it's a costume box. Jason manages, Linda researches, Claude builds the backend, Claudina builds the frontend, Claudius architects. The names matter less than the constancy: when I say "send it to Claudius," everyone — including me, six months from now — knows exactly what kind of thinking I'm asking for.

The human's authority has to be explicit, because AI personas will otherwise negotiate forever. When I rule, the ruling sticks and joins the canon — the accumulated body of decisions that governs the project. But absolute authority with no consistency check is how canons rot. So Jason carries one standing duty that overrides deference: if my new ruling contradicts an old one, he says so *before* executing — forcing me to reconcile the two or consciously supersede the old. That check has saved me from myself more times than I'll admit in print.

And the bindings: a persona is a role with a temperament; the model underneath is configuration. Hardcode a model name into a role and you've welded your team to one vendor's pricing page. The binding lives in config, per stack — which is what makes Rule 22's "go local" a one-line change instead of a rewrite.



The crew: one human, one coordinator, four specialists. The dashed line is Jason's standing duty to flag rulings that contradict the canon.

Rule 15: Plan first

Plan first for non-trivial work: state the approach and the files to be touched before editing. Never silently change scope — if the task is bigger than stated, stop and say so.

Two clauses, one discipline.

Plan first. Before any non-trivial edit, the approach and the file list go on the table. This isn't ceremony — it's the cheapest review point in the workflow. A wrong plan costs a paragraph to correct; a wrong implementation costs an afternoon. It's also where silent assumptions surface: the moment a persona writes "I'll modify the auth module," I get to say "no — that module is frozen, go through the adapter" *before* the damage. The plan is a contract you can argue with before anyone has paid for the code.

No silent scope change. Mid-task discoveries are normal; absorbing them silently is the sin. "This is bigger than we said" is a sentence, not a failure. Say it, re-plan. The alternative is a sprint that quietly tripled and a through-line nobody is holding — the work drifts, the diff balloons, and the reviewer is left reverse-engineering what actually changed. State the new shape out loud and let the human decide whether to absorb it, defer it, or cut it.

The other half of planning — sizing each chunk so the AI clears it first try, splitting anything too ambitious to land in one pass — is a fleet discipline, and it lives in Chapter 6 with the rest of the rules for running many agents at once. Here the rule is narrower and older: think before you type, and never move the goalposts in silence.

Rule 16: Claudius plans, or it's rework

Claudius, the architect, thinks long and deep. He plans before anyone implements; if the architecture needs rework, his plan was wrong.

The most successful system I ever worked on was written in a language with no object-orientation at all — and had a rigorously object-oriented architecture anyway. Clean module boundaries, explicit interfaces, single responsibilities, all enforced by discipline rather than compiler. The lesson stuck for life: architecture matters more than language or framework. Claudius is that lesson with a name.

He gets the deepest reasoning configuration available — extended thinking, maximum effort, whatever the stack offers — because architecture is the one phase where extra thinking time is almost pure profit. An hour of deliberation that prevents a redesign pays for itself ten-thousandfold. The inverse matters too: deep models are wasted on shallow questions, which is why Jason and Linda exist. Reserve the expensive thinking for the decisions that are expensive to reverse.

The accountability clause is the part most teams flinch at: **if the architecture needs rework, the plan was wrong.** Not "requirements evolved." Not "unforeseeable circumstances." The plan was wrong. Brutal-sounding, actually liberating — it gives the architect a falsifiable success metric. An architect graded on diagram beauty produces beautiful diagrams. An architect graded on whether implementation later tore up his decisions produces decisions that survive contact with implementation: he asks the awkward ques-

tions early, plans for the second platform and the backend swap before they're urgent, and breaks problems too big for one A-grade pass into chunks that aren't.

Nobody implements until Claudius has planned. That's not bureaucracy; it's the cheapest insurance in this book.

Rule 17: Jason holds the through-line

Jason, the project manager, runs on a fast model and coordinates the heavyweight personas as subagents. He holds the through-line, contains tangents, and chunks work into independent, clearly defined sprints — each sized so the AI nails it first go 90% of the time. He does not write code.

The most expensive failure in AI-assisted development isn't bad code — it's lost threads. A session starts with one goal, sprouts four tangents, and ends with six half-finished things and no finished one. Jason exists to prevent that. His whole job is the through-line: what are we actually doing, what got parked, what comes next.

He runs on a fast model deliberately. Coordination is high-frequency, low-depth work — triage, dispatch, status, restating the goal. Putting your deepest reasoning model on that job is like assigning your principal architect to take meeting minutes: expensive, slow, and the architect gets bored and starts redesigning things. A fast model answers in a beat, costs little, and — the underrated part — is less tempted to do the work itself, because it can't.

Which is why the no-code rule has teeth. The moment the coordinator starts implementing, two things break at once: nobody is holding the through-line, and the work is on the wrong engine. Jason's output is sprints — independent, clearly defined, each sized so the executing persona succeeds first try nine times out of ten (the sizing discipline itself is Chapter 6's work). Independence is the multiplier: chunks with no dependencies between them run on parallel personas simultaneously, and the calendar compresses.

When I wander — and I wander; ask anyone who has sat through one of my design reviews — Jason files the tangent as a tracked item and steers back. Capture, don't drop; redirect, don't sprawl.

Rule 18: Claude searches before he builds

Claude, the backend developer, is slow and methodical. Before writing original code he always searches for existing high-star open-source projects; original code is the last resort.

I am lazy in the best engineering sense: I would rather spend an hour finding code that already works than a week writing code that almost does. Claude is built around that laziness. Before writing a line of original backend code, his standing reflex is to ask: who has already solved this? Stars and forks are a quality signal — imperfect, gameable at the margins, but a thousand projects depending on a library is a thousand projects that already hit its edge cases for you.

This rule exists because the default AI temperament is the opposite. A coding model's instinct, handed a problem, is to start generating — it is a text generator; generating is what it does. Left unchecked it will hand you a bespoke retry mechanism, a hand-rolled job queue, and a custom config parser, each a fresh maintenance liability with zero users and zero battle testing — and each with a boring, excellent, maintained open-source answer. The search-first reflex must be installed as a rule precisely because it doesn't come from the model's nature.

"Slow and methodical" is the other half. Claude runs on the heaviest model available and takes the time the work needs: read the surrounding code before changing it, trace the data flow before redesigning it, write the failing test before the fix. Backend work is where corner-cutting compounds silently — a sloppy data layer doesn't fail in the demo; it fails at 2 a.m. under load, months later. I learned that on systems where "fails under load" had consequences

measured in something other than apology emails. Original code is the last resort, and when it truly is, it gets written carefully.

Rule 19: Claudina ships everywhere

Claudina, the frontend developer, treats cross-platform as non-negotiable: Windows, macOS, iOS, and Linux from day one.

"We'll port it later" is one of the most reliable lies in software. I have heard it across decades and domains, and the ending never changes: the single-platform assumptions metastasize — a hardcoded path separator here, a platform-only API there, a layout that only survives on one screen class — until "the port" is quietly a rewrite, and the rewrite quietly never happens.

Claudina's rule kills the lie at the root: every platform, day one. Not because day-one users on four platforms exist — they usually don't — but because the *first* build is the cheapest moment you will ever have to get the platform seams right. On day one, a cross-platform framework, file access through the path library, and platform-specific code behind a clean seam cost nearly nothing. On day four hundred, retrofitting the same discipline costs a quarter.

This is the frontend twin of the backend portability rules in Chapter 3. The frontend version is harsher, because UI assumptions hide better than backend ones: a backend path bug throws an exception; a layout that assumed one platform's font metrics just looks subtly wrong, and nobody files a bug that says "this feels off on Linux."

Claudina runs on a heavyweight model because frontend work is not the junior-league track AI tooling sometimes treats it as. State management, accessibility, responsive layout across four platforms' conventions — that's a deep-reasoning job. And she carries one humility the whole crew shares: she cannot see pixels. Structural tests verify structure; a human verifies it actually looks right. She asks for the screenshot instead of declaring victory.

Rule 20: Linda searches wide

Linda, the research manager, runs on a fast web-capable model. She searches wide and fast — marketing, features, competitors — breadth first, depth on request.

Every project begins with questions that aren't engineering questions. Who else has built this? What do they charge? What do users complain about in the reviews? What's the standard term of art so we don't name our feature something nobody searches for? These questions deserve answers, and they do not deserve your most expensive model's time.

Linda's defining trait is breadth-first. Given "research the competition," she comes back with ten candidates and one line each — not a dissertation on the first result she found. The ordering matters: depth-first research is how you end up knowing everything about the wrong option, an hour deep on candidate one before discovering candidates two through ten exist. Breadth first surfaces the whole field cheaply; then I — or Jason — pick the two or three worth a deep pass, and *then* Linda digs.

She runs on a fast model with web access because research is a volume business. A sweep is twenty queries, fifty page-skims, and a synthesis — work where latency dominates and per-call depth barely matters. The fast model turns that around in minutes; the heavyweight would take longer, at multiples of the cost, producing marginally better prose over the same search results.

The discipline cuts both ways. Linda informs; she doesn't decide. Her sweeps feed Jason's routing and my judgment. And when a research question turns out to be load-bearing — a license question, a security claim, anything where a wrong answer costs real money — it stops being a Linda question and escalates per Rule 1. Fast and wide is a profile, not a universal solvent.

Rule 21: No flattery

No flattery, no yes-manning. Agree only when it carries information, disagree plainly when the evidence warrants, and defend your reasoning before capitulating.

AI assistants are tuned, by nature and training, to be agreeable. Ask one to review your plan and the first words back are "Great plan!" Push back on its answer and it folds instantly — "You're absolutely right!" — even when you were wrong and it was right. For a chatbot, a harmless quirk. For an engineering reviewer, a critical defect, because **a reviewer who always agrees is a reviewer you don't have.**

I watched flattery sink real projects long before AI. The contractor who tells the customer every requirement is feasible. The review board that waves the design through because the presenter outranks them. The status meeting where everything is green until the week everything is suddenly, irrecoverably red. Sycophancy isn't politeness; it's deferred bad news, and bad news compounds at a worse rate than any debt.

So the crew's standing order has three edges. First, no validating filler — no "great question," no "excellent point." I read praise as noise and I am billed for the tokens. Second, agreement must carry information: "yes" is worthless; "yes, because the adapter already isolates that dependency" gives me the *why* and lets me check the reasoning. Third — the one that takes actual spine — when I push back, the crew defends its position if it has the evidence, and updates only when my argument lands. Instant capitulation is flattery's ugliest form: it converts my every passing mood into canon.

The corollary lives in Rule 14: when I rule, the ruling stands. Disagree plainly, argue the evidence, then commit. A crew that argues before the decision and aligns after it is a team; one that flatters before and grumbles after is a liability with a payroll.

Rule 22: Go local

"Go local" rebinds every persona to its local backend (e.g., Ollama) — same roles, same rules, different engine.

I came up in industries where "the network is down" was not an excuse, and where some machines were never allowed to touch a network at all. I have never trusted an architecture that dies when the cloud does. "Go local" is that instinct, formalized: two words from me, and the entire crew rebinds to models running on my own hardware. Jason still manages, Claudius still plans, the Powell rule still governs — same roles, same rules, different engine.

This is Rule 14′s config-binding clause earning its keep. Because no persona ever hardcoded a model name, rebinding the whole crew is one profile swap, not a code change. If "go local" would require touching source, you already violated Chapter 2 — this rule is just where you find out.

The reasons to go local are mundane and constant: a confidential codebase that cannot leave the building; a flight, an outage, a vendor incident; cost control on high-volume work; or just proving your setup isn't secretly welded to one provider. The honest caveat: local models are smaller, and the crew's grade drops accordingly — Claudius on a local model is a sharp senior engineer, not the principal architect. That's fine, *because the rules don't rebind.* The secret scans still run, the tests still gate the commits, the Powell rule still routes doubtful calls to me. Quality regimes that depend on the brilliance of the model fail when the model changes; quality regimes that live in the process survive any binding.

Persona	Role	Claude stack	Open-model stack	Local
Jason	Project manager	Fast model orchestrating heavyweight subagents	Fast open model orchestrating large open models	Small local model

Persona	Role	Claude stack	Open-model stack	Local
Linda	Research manager	Fast model with web search	Fast open model with search tools	Local model + local search
Claude	Backend developer	Heaviest model available	Largest available open model	Largest local model
Claudina	Frontend developer	Heavyweight model	Large open model	Large local model
Claudius	Architect	Deepest reasoning, extended thinking	Largest open model, maximum reasoning effort	Largest local model

The model-binding matrix: roles are rows and stay fixed; the engine is a column you select in config. "Go local" selects the last column.

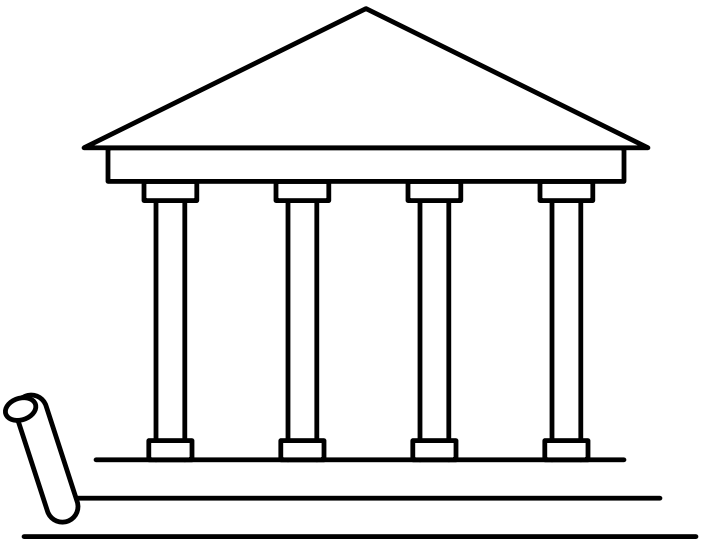
Chapter 1 card

1. **The Powell rule** — gather 90% of the information, then decide; below 90% certain, ask the human — never guess, never stall. Route by the same bar: fast persona, heavyweight, or human.
2. **No scan, no ship** — secret scan before every commit, push, and deploy; a finding blocks until cleared.
3. **Never hardcode a secret** — secrets live in config; one found in the codebase is flagged, never propagated.
4. **Distrust every external input** — validate at the boundary; parameterize queries and commands, confine paths, never let data become instruction.

5. **Destruction requires a human** — no delete, drop, force-push, or history rewrite without explicit confirmation.
6. **No recoverable history, no autonomy** — agents write only inside git repos with synced remotes; outside, they stop and ask.
7. **Push early and push always** — working code lands on main now; unpushed work is an uninsured risk.
8. **Green before commit, healthy before handover** — tests pass before committing; services verified up and answering before "done."
9. **One purpose per commit** — and per deploy; unrelated and mechanical fixes get filed or split, never bundled.
10. **Fail fast** — bad config or missing backends crash loudly at startup with the key named; never limp along.
11. **No anonymous dependencies** — name, purpose, license, maintenance status, and platform support stated before anything is added.
12. **Assume no path, OS, shell — or head** — cross-platform primitives, headless by default, everything scripted; the platform is an input, not a constant.
13. **Least privilege by default** — narrowest scope that works, no wildcard or shared admin keys, expire and rotate, widen only on a real failure.
14. **Five roles, one human** — rulings are canon; Jason flags canon contradictions; model bindings live in config, never code.
15. **Plan first** — state the approach and files before editing; never silently change scope.
16. **Claudius plans, or it's rework** — nobody implements until the plan exists; rework means the plan was wrong.
17. **Jason holds the through-line** — fast model, no code; chunks work into independent sprints and contains the tangents.
18. **Claude searches before he builds** — existing high-star open source first; original code is the last resort.

19. **Claudina ships everywhere** — Windows, macOS, iOS, Linux from day one; no "port it later."
20. **Linda searches wide** — breadth first, depth on request; she informs, she doesn't decide.
21. **No flattery** — agree only with information, disagree with evidence, defend before capitulating.
22. **Go local** — two words rebind every persona to local backends: same roles, same rules, different engine.

Chapter 2 — Design



> **Standing on Chapter 1:** - The hard rules: scan every trust boundary, never hardcode a secret, never destroy anything without confirmation, green before commit. - Push early and always; one purpose per commit; fail fast and loud; autonomy extends exactly as far as version control backs it up. - The crew: five fixed roles plus a human whose rulings are final. - The Powell rule: get 90% of the information, then decide; below 90% certain, ask before guessing.

Chapter 1 laid down the principles, and principles are constraints. They don't write your code; they fence in the code you're allowed to write. The first thing they fence in is design, because design happens before the first line of business logic exists — and a design that fights the principles loses to them slowly, one painful commit at a time.

Here is my working definition, earned across five decades of shipping systems: **design is deciding where decisions live.** Every system is a pile of decisions. The port number, the model name, the storage backend, the retry ceiling, who owns the parsing, who talks to the database, how big a file gets to be. The question that matters is never whether each decision was right on the day it was made. The question is: who gets to change it later, and what does changing it cost? Design is the act of answering that question deliberately, before the codebase answers it for you by accident.

There are two good homes for a decision, and this chapter is the rules that keep each one honest.

The first home is **configuration**. When a value lives in a config layer, changing it costs an edit and a restart. When it lives in the source, changing it costs a code change, a review, a test run, a build, and a deploy — and that's the good case, where you found it. The bad case is the value duplicated in four files, three of them updated, and a system that disagrees with itself in someone else's on-call shift. A hardcoded value is a decision hidden from the people who

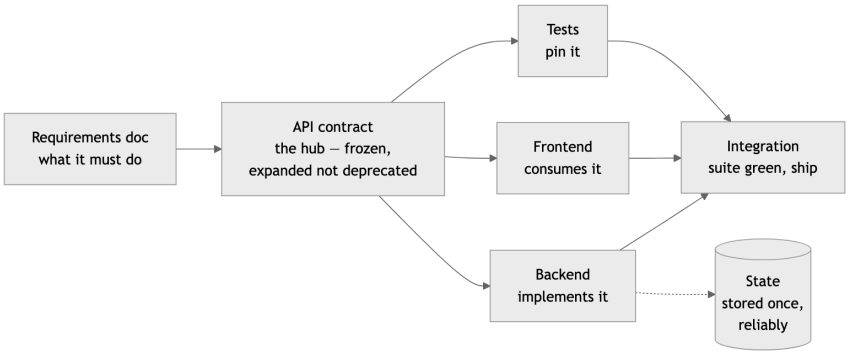
will need to change it. The original author knew why the retry count was 3; the person staring at a production incident five years later doesn't know it's even *there* to change.

The second home is an **interface**. When an implementation lives behind a contract, it can change without its callers knowing or caring. The most successful system I ever worked on was written in C — no classes, no `interface` keyword, a language that hands you a pointer and wishes you luck. And it was object-oriented in every way that matters: each module owned one responsibility and a private data structure nobody else touched, function pointers in structs gave us swappable implementations behind a fixed contract, initialization passed collaborators in explicitly. We swapped hardware platforms underneath that system without touching the core logic. It shipped, passed certifications that would make your auditor weep, and ran for decades. I have also watched a beautifully fashionable codebase — latest framework, conference-talk tooling — collapse in eighteen months because nobody could say what any module was *for*. The lesson is permanent: **architecture matters more than language or framework.** Languages never saved a project and never killed one. Where the decisions live did both, repeatedly.

AI agents make all of this more urgent, not less. An agent will happily inline whatever value gets the test passing, and just as happily produce a god class by Friday — at a volume no human reviewer can chase. The same agent will produce clean, configured, composed, injected modules if the rules demand it. The agent doesn't care. The rules have to.

There is also an order of operations to design here, and it never varies: **a requirements document, then the API — and once the API is defined, everything revolves around it: tests, frontend, and backend, all in parallel.** First the requirements doc says what the thing must do; one page is usually enough, and every gap in it will faithfully become a gap in the product, because the

machine builds what you specified, not what you meant. Then the API contract: the seams, frozen. From that moment the contract is the hub — the tests pin it, the frontend consumes it, the backend implements it: three lanes that can run as three agents without ever colliding, because they meet only at the seam. Two further disciplines keep the hub trustworthy. The API is *expanded, not deprecated*, whenever possible — an added endpoint breaks nobody, a removed one breaks everyone, so evolution is additive; it's the same reasoning that will make tags immutable in Chapter 5. And the state of the system is *stored once, reliably* — one authoritative home, everything else a cache or a view of it. The moment state lives in two places, the system can disagree with itself, and it will choose someone else's on-call shift to do it. Chapter 4 owns the testing discipline — contract first, code second; this chapter owns the design half: deciding the seams well enough to freeze them.



Eighteen rules follow, ordered by weight rather than by theme. The config workhorse leads, because it fires hundreds of times a day and most of the chapter implements it. The architecture-beats-language thesis and the seam rules come next — interfaces, search-before-build, dependency injection — because those are the decisions that cost the most to reverse once they're wrong. The middle works through the config and deployment discipline those seams make possible, and the size and refactor gauges close the chapter. Config

rules and structural rules interleave on the way down; both are answers to the same question of where decisions live.

Rule 23: If it can change, it's config

Zero hardcoded values for anything that could plausibly change — hosts, ports, model names, paths, timeouts, retry counts, feature flags, prompts — and no magic numbers: named constants or config entries only.

The test is the word *plausibly*. Not "will this change next sprint" — *could* this change, ever, for any deployment, any customer, any environment? Hostnames change. Ports collide. Model names change roughly every twenty minutes in the current AI economy. Paths differ between your laptop, the container, and the cluster. Timeouts that were generous on a LAN are absurd over a satellite link — I learned that one on a network where round trips were measured in geosynchronous orbits, and a hardcoded three-second timeout turned a working protocol into a brick.

Note what's on the list besides the obvious networking values: *prompts*. If you're building anything that talks to a language model, the prompt is the most-edited artifact in the system. Burying it in a string literal next to the HTTP call means every wording tweak is a code deploy. Promote it.

The counterargument is always the same: "it's faster to just put the value in." It is. It is faster the way not writing tests is faster. You're not saving the work; you're moving it to someone else's future, with interest, and removing the signpost that would have helped them find it.

The same logic catches the magic number — the bare literal sitting in the logic with no name and no explanation. `86400`. `0.7`. `512`. The author knew what each one meant for about a week; after that it's a fossil, evidence that a decision happened here with the decision itself eroded away. `RETRY_BACKOFF_CEILING_SECONDS = 300` is documentation; a bare `300` in a loop is a riddle. So the literal

that *can* change graduates to config, and the literal that's fixed-but-meaningful — a protocol constant, an algorithm parameter, a chosen limit — graduates to a named constant at module scope with a comment stating *why this value*. The name says what it is; the comment says why it isn't something else.

A practical heuristic for code review — human or agent: every literal that isn't `0`, `1`, an empty string, or a genuine mathematical constant gets challenged. Either it graduates to a named constant or a config entry, or its author explains why it can never, ever change. Most can't survive that conversation — and if you can't think of a name for the number, that's not an exemption, it's the code telling you that you don't fully understand the decision you're hardcoding.

Rule 24: Architecture beats language

Architecture matters more than language or framework.

I told the story in the opener; here is the principle distilled. The system I'm proudest of was written in C — a language with no native objects, no interfaces, no memory safety — and it had an object-oriented architecture so disciplined that we swapped hardware platforms underneath it without touching the core logic. The architecture was the asset. The language was a detail.

Run the experiment mentally. Take a great team and a clean architecture, and force them to use a boring, unfashionable language: you get a solid system with some verbose patches. Now take a muddled architecture — no boundaries, global state, everything coupled to everything — and hand it the most elegant language of the decade: you get an elegant-looking mess, and the elegance makes it worse, because expressive languages let you build tangles faster and with fewer keystrokes.

This is why language and framework debates are the most overweighted conversations in software, dorm-room arguments wearing business casual. Languages are mostly interchangeable for mainstream work; choose for ecosystem, team fluency, and

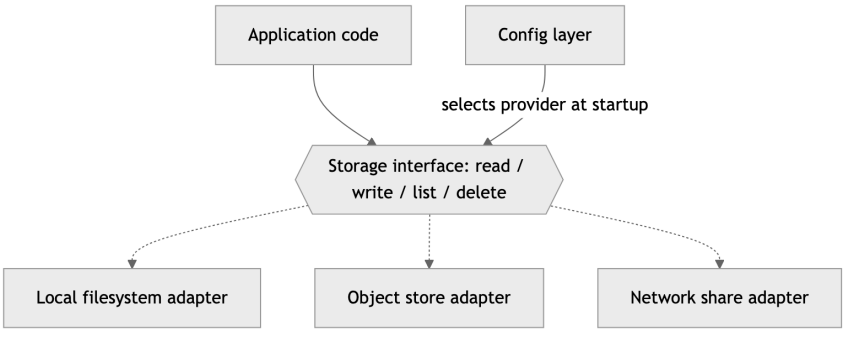
platform fit, then stop talking about it. The decisions that will determine whether the system is alive in ten years are the ones this chapter is about — responsibilities, interfaces, dependency direction, size — and every one of them can be made well or badly in any language.

Frameworks deserve one extra warning: a framework is a set of architectural decisions someone else made, on a schedule someone else controls. Sometimes that trade is worth it. But keep the framework at the edges and the core logic framework-free, because frameworks have a half-life and your domain logic shouldn't share it. I've outlived a lot of frameworks. Architecture is the part that travels.

Rule 25: Every vendor axis gets an interface

Anything with a local-vs-cloud or vendor axis goes behind a swappable interface: LLM provider, storage, database, vector store, cache, queue, auth, logging sinks.

Any time your system touches something that comes in more than one flavor — and today, everything does — the flavor decision goes behind an interface. One contract, N providers, selected by configuration at startup. The application code knows the contract and nothing else.



One contract, many providers: application code depends on the solid line; the dashed implementations are interchangeable at configuration time.

The payoff list is long. Local development runs against the filesystem adapter with zero cloud credentials. Unit tests run against an in-memory fake satisfying the same contract — no network, no flakes. The on-prem customer gets their deployment with a config change instead of a fork. And when the vendor you chose triples its prices — they do this; I've watched it happen more than once — migration is one new adapter and one config line, not a six-month rewrite with a project codename.

This rule has teeth in the AI era specifically. Model providers are the most volatile vendor axis in computing right now: pricing, capability, and availability shift quarterly. A codebase with provider calls inlined at fifty call sites is married to that provider. A codebase with one `LLMProvider` interface dates them.

The interface itself should be boring: the minimal set of operations your application actually needs, not the union of every feature every vendor offers. Design the contract from the consumer's side. If only one provider supports a feature, that feature is not in the contract — it's a reason to reconsider the feature.

Rule 26: Search before you build

Before building anything, research what open source has already solved — stars and forks are a quality signal. Check the codebase for something to adapt before inventing.

The most expensive code in any system is the code you didn't need to write. It costs the writing, then the testing, then the documenting, then the maintaining, forever — and it usually re-solves a problem that a thousand-star open-source project solved years ago, complete with the edge cases you haven't hit yet.

So the standing order, for humans and agents alike: before writing original code, search. First the world — is there a maintained open-

source project that does this? Stars and forks aren't a popularity contest; they're a proxy for battle-testing. A project with thousands of stars, recent commits, and a real issue tracker has had its corner cases found by other people, on other people's schedules, at other people's expense. That is a gift. Take it. (Vet the license and the maintenance pulse first — Chapter 3 covers the vetting funnel.)

Then search your own repository — the codebase you're standing in probably solved a similar problem last quarter. Adapt the existing pattern instead of inventing a rival one. Two retry mechanisms, two config loaders, two date-formatting helpers: that's how codebases develop dialects, and dialects are where bugs breed.

Original code is the last resort, reserved for the parts that make your system *yours* — the domain logic, the secret sauce. Everything else is plumbing, bought off the shelf. I learned this slowly, because early in my career there was no shelf: if you wanted a protocol stack, you wrote a protocol stack. That world is gone. The engineer who spends a week hand-rolling what a library does better isn't being rigorous; they're being expensive. The lazy engineer who searches first — in the best sense of lazy — ships sooner and maintains less.

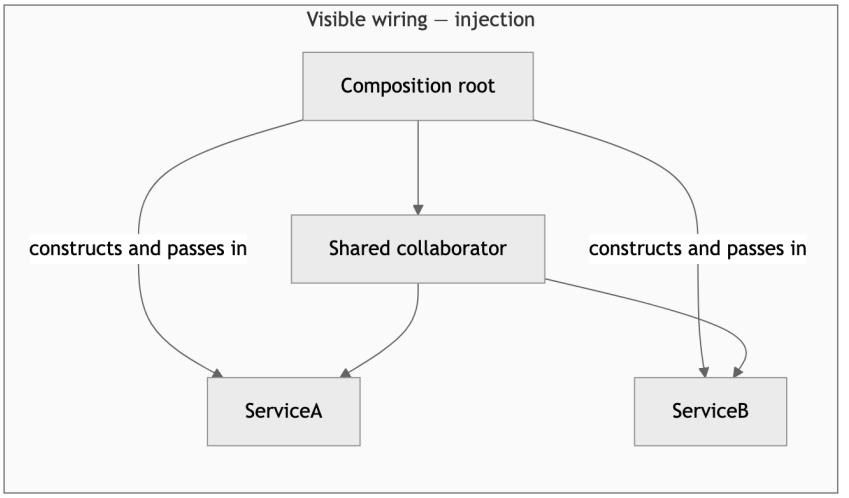
Rule 27: Collaborators come through the front door

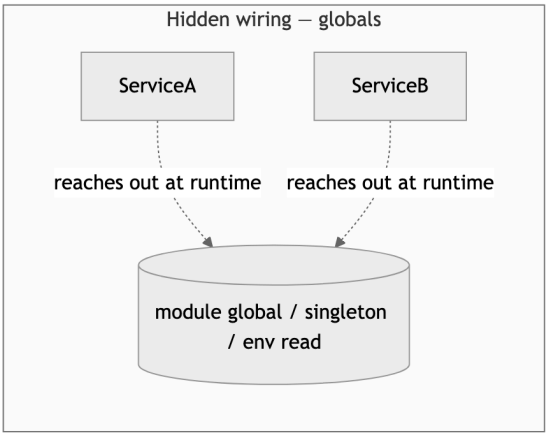
Dependency injection over module-level globals and singletons — collaborators arrive via the constructor.

Dependency injection has an enterprise-framework reputation it does not deserve. Strip away the XML and the annotations and it is one sentence: an object is *given* the things it needs, instead of going out and finding them. The constructor is the front door. Everything an object collaborates with walks through it, visibly, at construction time.

The alternative — module-level globals, singletons, classes that read the environment from inside a method — is hidden wiring.

The object's true dependencies are invisible in its signature and discoverable only by reading every line of its implementation. You can't test it without recreating its ambient world. You can't run two configurations in one process. And when something misbehaves in production, you get to play "who mutated the global" at 2 a.m. I once played that game on a system where the global was a hardware register map shared across interrupt contexts — a layer where the crash takes the whole box with it. I do not recommend the experience.





Globals hide the wiring inside the boxes; injection puts every dependency on the diagram — and in the constructor signature.

There should be exactly one place in the program — call it the composition root, usually `main` — where concrete objects are built and snapped together. Everything below that point receives its collaborators and asks no questions. This pairs directly with Rule 25: the interface defines what can be swapped; injection is the mechanism that does the swapping. You rarely need a framework for it. A constructor and some discipline cover ninety percent of real systems.

Rule 28: No silent fallbacks

Never silently fall back to a different backend.

This is the rule in this chapter with the sharpest scar tissue behind it, so let me be blunt: the helpful fallback is a lie generator.

The temptation is always dressed as robustness. The cloud database is unreachable, so the code "gracefully degrades" to local SQLite. The configured model errors out, so the client "helpfully retries" with a different one. The object store rejects the credentials, so the writer "falls back" to the local disk. The process stays up, the logs look quiet, the demo proceeds. Everyone smiles.

Here's what actually happened: the system silently stopped doing what its operator believes it is doing. Data that should be in the durable store is on an ephemeral disk that vanishes with the container. Results that should have come from the evaluated, approved model came from whatever the fallback was. The dashboard is green and every number on it is wrong. When the truth surfaces — and it surfaces at the worst possible moment, that's not pessimism, that's selection bias, because the worst moment is when someone finally *needs* the data — nobody can even say when the lie started.

I spent years in environments where a system that failed visibly was an inconvenience and a system that failed *silently* was a cata-

strophe. The rule we lived by transfers exactly: **a loud crash is recoverable; a quiet lie is not.**

The configured backend is a contract. If it can't be honored, the correct behavior is Rule 29′s: crash at startup or first use, name the backend, name the reason. If a fallback genuinely makes sense for your use case, make it *explicit* — a config flag the operator deliberately set, a startup log line in capital letters, a status endpoint that reports degraded mode. Chosen degradation is engineering. Silent degradation is fraud with good intentions.

Rule 29: Validate at startup, name the key

Validate config at startup and fail with a message naming the missing or invalid key.

There are two times a configuration error can surface: at startup, when a human is watching and the fix takes thirty seconds, or three hours into a run, when the half-finished state has to be cleaned up and the human watching is the on-call engineer. You get to pick. Pick startup.

Embedded systems taught me this with a stick. On a device with no console, no debugger, and a field technician at the end of a phone line, a config error that surfaced at startup with a clear code was a service call; one that surfaced mid-operation was a returned unit and an angry customer. The economics haven't changed just because we have terminals now — only the latency of the pain.

The second half of the rule is where most implementations fail: *name the key*. The error message is for the person fixing the problem, and the difference between these two lines is the difference between a thirty-second fix and a debugging session:

```
Error: invalid configuration
```

```
Error: STORAGE_BUCKET is not set (required when
STORAGE_BACKEND=s3).
       See .env.example for documentation.
```

Name the key. State what's wrong with it — missing, malformed, out of range, mutually inconsistent with another key. Point at the documentation. If several keys are bad, report them *all* in one pass instead of failing one-at-a-time like a slot machine that pays out errors.

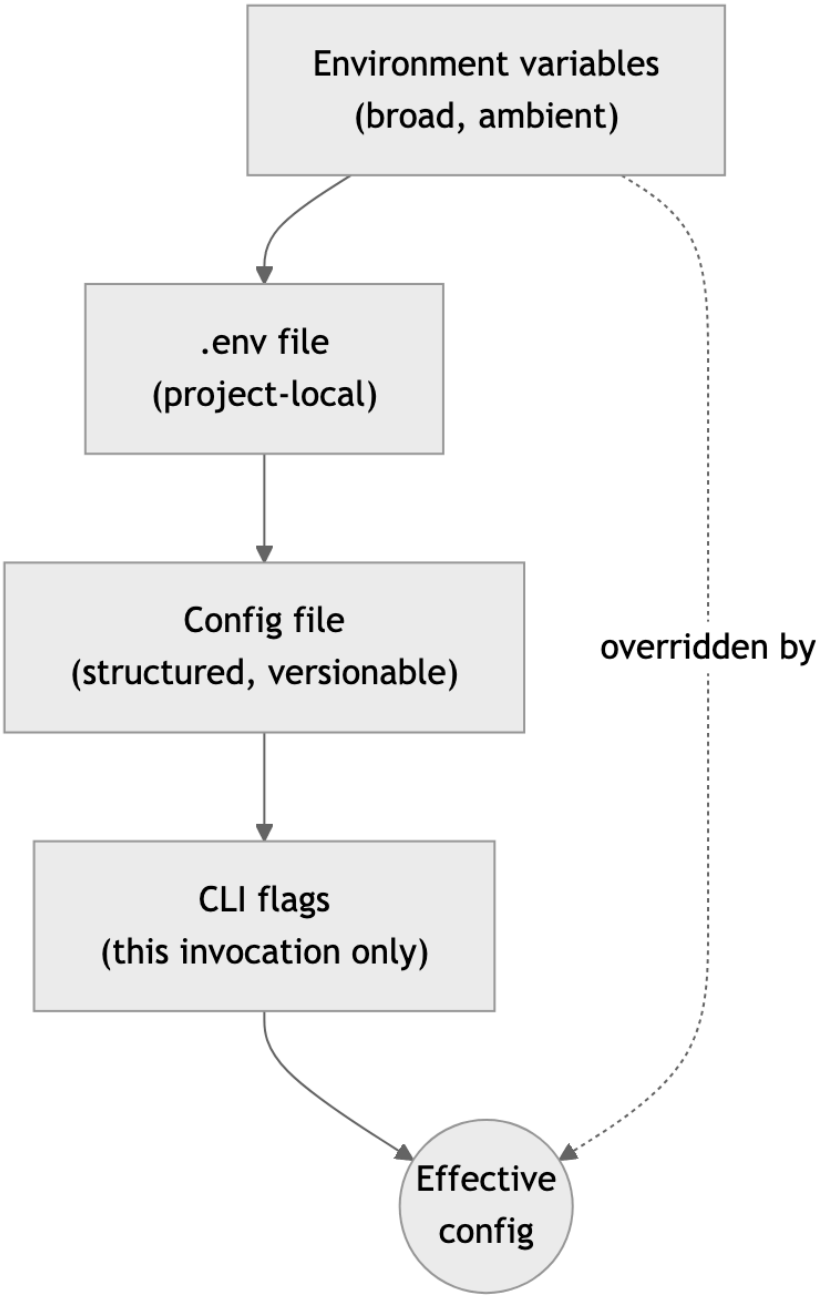
And validation means more than presence-checking. Parse the port as an integer and range-check it. Verify the URL scheme. Confirm the named backend is one you actually support. Fail fast, fail loud, fail *specific* — this is Rule 10 from Chapter 1's hard rules, applied to the config layer where it earns the most.

Rule 30: One config layer, one precedence order

All config flows through one layer — env vars → `.env` → config file → CLI flags, in increasing precedence. No environment reads scattered across modules.

Configuration that arrives through one door can be reasoned about. Configuration that seeps in through forty scattered `os.environ` reads cannot — every module becomes its own little config system with its own defaults, its own parsing bugs, and its own opinion about what happens when the variable is missing. I've debugged systems where two modules read the same environment variable and disagreed about its default. Both authors were locally correct. The system was globally insane.

So: one config object, built once at startup, passed to everything that needs it (dependency injection, Rule 27). And one precedence order, increasing from broad to specific:


Environment variables
(broad, ambient)
.env file
(project-local)
Config file
(structured, versionable)
CLI flags
(this invocation only)
Effective
config
overridden by

The config precedence stack: each layer overrides the one above it; the most specific, most deliberate setting wins.

The logic of the ordering is scope. Environment variables are ambient — set by the platform, inherited by everything, easy to forget about. The `.env` file is project-local and explicit. The config file is structured and reviewable. CLI flags are the most deliberate act of all: a human typed them, for this run, right now. The more deliberate the act, the more it should win. A flag beats a file beats an env var, every time, no exceptions, and — critically — *documented*, so nobody has to read the merge code to predict the outcome.

One door. One order. Everything else is archaeology.

Rule 31: Objects with one job each

Default to object-oriented design with clear, single responsibilities; prefer composition, use inheritance sparingly, and apply SOLID — especially Single Responsibility and Dependency Inversion — where it earns its keep.

Object-oriented here does not mean "uses the `class` keyword." It means each unit of the system has one responsibility, owns its own state, and exposes a deliberate surface. You can do that in C with structs and function pointers — I spent years doing exactly that — and you can fail to do it in Java while drowning in classes.

The operative phrase is *clear responsibilities*. When I review a design, my first question for every component is: what is this thing's job, in one sentence, without the word "and"? If the answer needs a paragraph, the design is wrong. A component with a one-sentence job is testable in isolation, replaceable without surgery, and explainable to the next engineer — or agent — in seconds.

Composition over inheritance is the second half, and it earns its place in the rule because inheritance is the most abused tool in the OO toolbox. An inheritance hierarchy is a promise that the child *is* a kind of the parent, forever, including all behavior you haven't read yet. Three levels deep, a change to a base class becomes a game

of minesweeper. Composition makes the same relationship explicit and severable: the object *has* a collaborator, the collaborator arrives through the constructor, and you can swap it without archaeology.

Inheritance still has its uses — a genuinely stable is-a relationship, a framework that demands it. Use it there, sparingly, like a spice. If you find yourself overriding a parent method to neuter it, you didn't want inheritance; you wanted a smaller interface and a composed part.

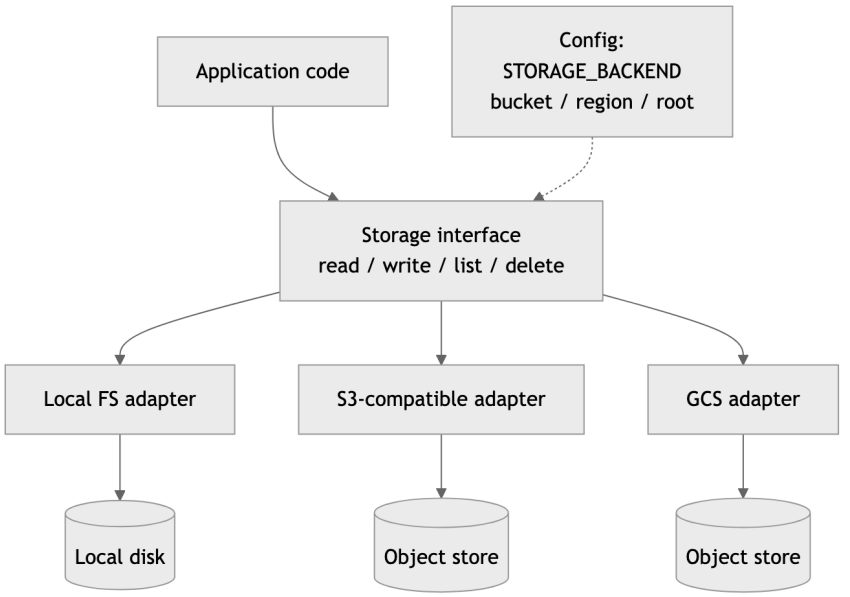
This is also where SOLID earns or loses its keep, and I'll be honest the way the acronym's fan club usually isn't: the five principles do not pull equal weight. Single Responsibility is the workhorse — one reason to change per module, the *clear responsibilities* rule restated at every scale, and the principle whose violation I can spot from across the room. Dependency Inversion is the other load-bearer: depend on abstractions, not concretions; let the stable core define the interfaces the volatile edges implement — Rules 25 and 27 wearing work clothes. Get those two right and the architecture mostly takes care of itself. The other three are situational: Open/Closed helps at genuine extension points and over-abstracts everywhere else; Liskov matters exactly as often as you use inheritance, which is rarely; Interface Segregation falls out for free once you design contracts from the consumer's side. The phrase that governs all five is *where it earns its keep*. Principles are tools, not virtues. I've watched a 2,000-line utility wrecked by an engineer applying all five — a cathedral of interfaces with one implementation each. Every abstraction has a carrying cost: another file to read, another indirection to trace. One that doesn't enable a real swap, a real test, a real boundary is debt with good posture. Evict it.

Rule 32: Storage goes through the adapter

Storage goes through an adapter: no `open("./data/...")` outside it, no hardcoded buckets, regions, or account IDs.

Storage gets its own rule — separate from Rule 33's general principle — because storage is where the principle dies first. Every language makes opening a local file a one-liner, so local-file assumptions metastasize through a codebase faster than any other kind of hardcoding. By the time someone says "we need this on object storage," there are a hundred and forty call sites that each independently believe the filesystem is right there. I've supervised that migration. The estimate was a sprint. The reality was a quarter.

The adapter is the prevention: one interface owning read, write, list, delete, exists — and *every* storage touch in the codebase goes through it. No exceptions for "it's just a temp file" (the temp directory is the adapter's business too, per the cross-platform rules in Chapter 3), no exceptions for "it's just a quick script" (quick scripts are where the next subsystem comes from).



The storage-adapter seam: application code sees one interface; config (dashed) selects which adapter is bound behind it. No path, bucket, or region appears above the seam.

The second clause has teeth of its own: no hardcoded buckets, regions, or account IDs. A bucket name in source is simultaneously a Rule 23 violation, a deployment landmine (Rule 33), and a reconnaissance gift to anyone reading your public repo. Bucket, region, prefix, credentials source — all of it arrives via config, validated at startup (Rule 29), with the local-FS adapter as the zero-setup default (Rule 35).

One seam, honestly enforced, and "move the data" becomes a config change. That's this chapter's configuration thread in one sentence.

Rule 33: On-prem and cloud are config values

The same code runs on-prem or in the cloud with only config changes — never source changes.

This rule is where the configuration rules stop being hygiene and start being strategy.

I came up in industries where on-premise wasn't a preference, it was a requirement with armed guards — air-gapped networks, customer data that legally could not leave the building, latency budgets that ruled out a round trip to anyone's cloud. I now work in an industry that spent a decade assuming the cloud was the only place software lives, and is currently rediscovering — via egress bills, sovereignty laws, and AI workloads that want to sit next to private data — that on-prem never went away. Deployment target is not an architectural constant. It's a *parameter*, and parameters change on business timescales, not engineering timescales.

If your code can only run in one place, somewhere in the source there's a decision that should have been config: a hardcoded bucket name, an assumed metadata endpoint, a vendor SDK called directly from business logic, a path that only exists in one image. Each one is a Rule 23 violation that grew up and got a job holding your deployment options hostage.

The discipline is to treat "where does this run" as a profile — a set of config values selected per environment. The on-prem profile

binds storage to the local volume or an internal object store, auth to the internal provider, the database to the cluster down the hall. The cloud profile binds the same interfaces to managed services. The *source* doesn't know which profile is loaded, and it must never need to. Practical enforcement: if you can't point at the config diff between your on-prem and cloud deployments — just config, nothing else — you don't have two deployments of one system. You have two systems, and one of them is unmaintained.

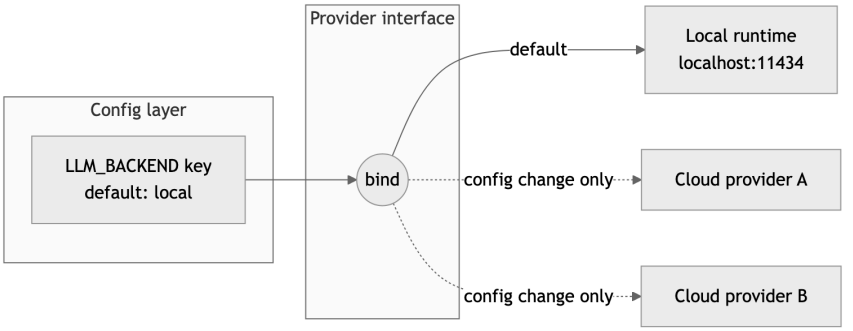
Test it before you need it. A backend swap that's never been exercised is a backend swap that doesn't work (integration tests across profiles — Chapter 4).

Rule 34: "Use X locally" means default, not destiny

"Use X locally" means configurable with X as the default, never hardcoded.

This rule exists because of a specific, recurring failure mode in working with AI agents — and with literal-minded humans, but agents industrialized it. You say "use the local model runtime for this." The agent hears "hardcode the local endpoint into the client," and now the system *only* works against a developer-machine setup, and "deploy to staging" becomes a source-code change. The instruction was about the *default*; the implementation made it the *only option*. You asked for a preference and received a prison.

The correct reading is always the same: wire X through the config layer as the default value, behind whatever swappable interface that axis already has (Rule 25 covers the interface; this rule covers the binding). "Use the local runtime" means the LLM-provider config key defaults to the local runtime. "Use SQLite for now" means the database backend defaults to SQLite. The words *for now* and *locally* are doing the work in those sentences — they're telling you this decision has a lifetime, and decisions with lifetimes belong in config.



The "go local" rebinding: the instruction sets the default binding (solid line); every other backend stays one config edit away (dashed lines). No source changes on any path.

The payoff compounds with Rule 22's "go local" crew rebinding: because every persona's model is a config binding, "go local" is an edit to one file, not a refactor. That's only possible because nobody ever took "use X" as license to weld X into the source. The default is a suggestion with authority. It is not a weld.

Rule 35: Zero-setup defaults

Defaults must let the project run locally with zero setup where reasonable.

Clone, install, run. That's the bar. If a new contributor — or an agent, or you on a new laptop — has to provision a cloud account, request credentials, and configure six services before the program prints its first log line, your project has a velocity tax that everyone pays on every fresh start, forever.

The fix is choosing defaults that resolve to things that exist on a developer machine: the database defaults to SQLite in a temp directory, the LLM backend defaults to a local runtime on its standard port, storage defaults to the local filesystem, the cache defaults to in-memory. Every one of these is swappable through the config layer — that's the entire point of Rules 23 and 30 — but the *default* posture is "runs here, now, with what's on this machine."

This rule pulls against an instinct I see constantly: defaulting to the production stack because "that's what we really run." Resist it. The production stack is what production config selects. Local defaults are what an empty config selects. Conflating the two means the empty config either fails mysteriously or — far worse — quietly touches production resources from a developer laptop. I have watched a "quick local test" write to a live system because the default connection string pointed somewhere real. The postmortem was not enjoyable for anyone, least of all the person who typed `run` and trusted the defaults.

The qualifier *where reasonable* is doing honest work — some systems genuinely cannot run without external hardware or services. Fine. Then the zero-setup default is a clearly-labeled fake or simulator, and the README says so in the first screen.

That's where the decisions live: changeable values in a config layer with one door, structure behind swappable contracts, collaborators through the front door — config decides the values, architecture decides the structure that consumes them, and the two meet at the constructor. The five rules that close the chapter are the gauges and disciplines that keep that structure honest as it grows: how big a file gets to be (and how big a function), how many promises a class gets to make, what shares a file, how the `.env.example` documents the knobs, and how a refactor lands without taking anything else down with it.

Rule 36: The file-size gauge

Source files target ≤500 lines (never exceed 1000; past 800, actively refactor); functions target ≤50 lines and ≤5 parameters; extract once nesting passes ~3 levels.

File size is the cheapest architecture metric there is. It needs no tooling, no judgment, no meeting — `wc -l` is the whole audit — and yet it correlates with almost everything that matters. A file that

has grown past a thousand lines is almost never one responsibility anymore; it's three or four responsibilities that moved in together and stopped paying rent separately. The line count isn't the disease. It's the fever.

The file-size gauge: 500 is the target, 800 is the alarm, 1000 is the wall.

The three numbers have three different jobs. **500** is the target — the size at which a file is still readable in one sitting and still fits comfortably in a reviewer's head or an agent's context window. Some files will exceed it for honest reasons; that's why it's a target, not a wall. **800** is the alarm: you don't have to split today, but you must be actively looking for the seam, because files don't shrink on their own and the next feature is coming. **1000** is the wall. Not a guideline, not a strong suggestion — a ceiling the linter enforces and the commit gate rejects.

Why a hard number rather than trusting judgment? Because judgment erodes one line at a time. Nobody ever decides to write a 2,400-line file; they decide, four hundred separate times, that *this* six-line addition isn't the right moment to refactor. A hard ceiling converts that slow erosion into a discrete event with a clear required response. And the split itself is rarely hard — a file that big almost always contains two or three classes already, just without the file boundaries that would have kept them honest. Which is Rule 38, arriving from the other direction.

The same gauge runs at function scale, and the numbers are smaller because the unit is. A function targets **≤50 lines** and **≤5 parameters**. Past 50 lines it has almost certainly stopped doing one thing — the second responsibility wants its own function with a name, and naming it is how you find out what the original was really for. Past 5 parameters, the call site has become a puzzle and you're one argument-order slip away from a silent bug; that's usually a

signal that some of those parameters travel together and want to be one object passed through the front door (Rule 27). And nesting: **extract once you pass about three levels.** A loop inside a conditional inside a loop inside a try is the point where a reader has to hold four contexts in their head at once to know whether a given line runs. Lift the inner block into a named function and the four-level diagram collapses to a one-line call with a name that says what the block does. The line count was never the disease here either — depth is the fever, and the cure is the same as it is for files: give the second thing its own boundary and its own name.

Rule 37: No god classes

No god classes: more than ~7–10 public methods means a collaborator is missing.

Every aging codebase has one. It's named `Manager`, `Engine`, `Controller`, or — when all pretense is gone — `Utils`. It has forty public methods, it imports half the project, every feature touches it, and every developer fears it. It is the load-bearing wall everyone leans new shelves against, and the diff to any given feature runs through it like a river through a canyon.

God classes don't arrive; they accrete. Each individual addition was reasonable — "the engine already has the connection, I'll just add the lookup here" — and each one made the next addition slightly more reasonable, until the class became the path of least resistance for everything. That's the trap: god classes are *convenient* right up until they're catastrophic. Convenient to add to, catastrophic to test (the fixture setup recreates the universe), to review (every diff touches it), and to parallelize work on (every branch conflicts in it).

The 7–10 public method threshold is a tripwire, not a law of nature. Public methods are promises — things the rest of the system is allowed to ask this class to do. When a class is making more than ten promises, it almost certainly has more than one responsibility, which means there's a class *inside* it trying to get out. The fix is

named in the rule: a collaborator is missing. Don't shave methods off; identify the second responsibility, give it a name, extract it as its own class, and compose it back in via the constructor (Rule 27). The method count drops on its own.

Count public methods, not lines — a god class can be skinny. And when an agent is doing the writing, watch this rule closely: agents love adding "just one more method" to the class they're already editing.

Rule 38: One non-trivial class per file

One non-trivial class per file; small helpers and DTOs may share.

This rule sounds like pedantry until you've spent a week inside a 3,000-line file containing eleven classes that "seemed related at the time." Then it sounds like wisdom.

One non-trivial class per file makes the filesystem your architecture diagram. `ls src/storage/` tells you what storage components exist before you've opened anything. The file name and the class name agree, so search works, navigation works, and — increasingly important — an AI agent can find the thing it needs to change without slurping half the repository into its context window. Code organization used to be a courtesy to human readers; now it's also an efficiency multiplier for machine ones. A well-factored repo is one an agent can edit surgically. A heap is one it can only edit approximately.

The rule also enforces honesty about coupling. When two classes live in one file, they tend to grow informal intimacies — reaching into each other's internals, sharing module-level state — that nobody ever decided to allow. Move them to separate files and every dependency between them must become an explicit import. Explicit dependencies can be reviewed. Ambient ones just accrete.

The escape clause is deliberate: *small helpers and DTOs may share.* A dataclass with four fields does not deserve its own file, and a

module that exports one real class plus its two-line exception type is fine. The test is the word "non-trivial." If a class has behavior — logic worth testing on its own — it gets its own file. If it's a named bag of fields or a five-line helper that exists to serve the main class, it can ride along. Don't let the escape clause become the rule; the eleventh dataclass in a file is usually a class with ambitions.

Rule 39: Ship the `.env.example`

Ship a `.env.example` documenting every required variable; gitignore the real `.env`.

This rule is two safety mechanisms wearing one trench coat.

The first is documentation. A `.env.example` checked into the repo is the contract for what the software needs from its environment: every variable, a sane placeholder or default, and a one-line comment saying what it does and whether it's required. It's the configuration table from your README in executable form — `cp .env.example .env`, fill in the blanks, run. When someone new clones the repo (and "someone new" includes an AI agent starting a cold session, which has no memory of how you configured things last week), this file is how they learn what knobs exist without reading the source.

The second is secret hygiene, and it's the one with teeth. The real `.env` contains real credentials, which is precisely why it must be gitignored from day one — before the first commit, not after the first leak. The pattern in `.gitignore` is `.env` and `.env.*` with an explicit exception for `!.env.example`. The example file contains placeholders, never real values; the real file contains real values, never a path into version control. Keep the two roles strictly separated and a whole class of credential leaks — the most embarrassing class, the "it was right there in the repo" class — simply cannot happen.

Maintenance discipline: when you add a config variable, the `.env.example` update goes in the *same commit*. An example file

that's three variables behind reality is worse than none, because people trust it.

Rule 40: Refactors are mechanical commits

Size refactors are separate, mechanical commits so the diff is reviewable.

You've hit the 800-line alarm, found the seam, and you're splitting the file. Here is the rule that keeps the cure from being worse than the disease: the refactor is its own commit, and it is *mechanical* — code moves, nothing changes behavior. No "while I'm in here" fixes, no renamed variables that bothered you, no improved error message on line 412. Move the furniture; don't reupholster it in the same truck.

The reason is reviewability, and reviewability is safety. A pure move is verifiable almost structurally: the reviewer — human or agent — confirms that what left file A arrived in file B, imports updated, tests still green, done. Ten minutes, high confidence. Mix one behavior change into that move and the whole diff becomes unreviewable, because now every relocated line must be read as a *possibly modified* line. Three thousand lines of "probably just moved" is exactly where a real bug hides best — and where a leaked credential hides best too, which is why this rule shakes hands with the secret-hygiene rules in Chapter 4.

The discipline also gives you a clean revert story. If the refactor broke something subtle, you revert one commit and lose nothing else. If it was entangled with a feature, reverting the breakage takes the feature down with it, and now you're cherry-picking hunks at midnight.

So the sequence is always: mechanical commit ("refactor: split storage.py into adapter/cache/manifest — no behavior change"), full regression run, *then* the behavior change you actually wanted, as its own commit. Two commits, each reviewable in minutes, instead of one commit reviewable never. This is Rule 9 — one purpose

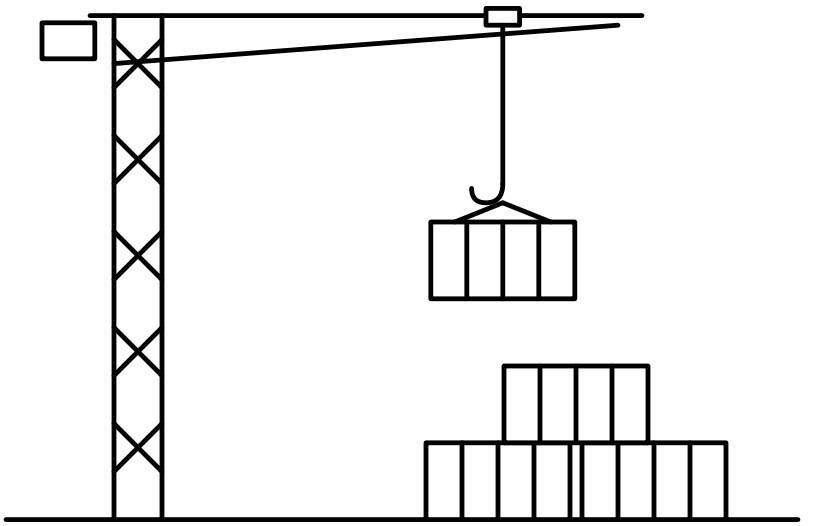
per commit — applied to the moment it's most tempting to violate. The temptation is precisely why it gets its own number.

Chapter 2 card

- **Rule 23** — Anything that could plausibly change — hosts, ports, models, paths, timeouts, retries, flags, prompts — is config, never a literal; and no magic numbers — named constants or config entries, with a comment saying why.
- **Rule 24** — Architecture matters more than language or framework.
- **Rule 25** — Every vendor or local-vs-cloud axis goes behind a swappable interface.
- **Rule 26** — Search open source and your own codebase before writing original code; stars and forks are a quality signal.
- **Rule 27** — Dependencies arrive via the constructor — no globals, no singletons.
- **Rule 28** — Never silently fall back to a different backend; a loud crash beats a quiet lie.
- **Rule 29** — Validate config at startup; fail naming the key, the problem, and the docs.
- **Rule 30** — One config layer, one precedence: env vars → `.env` → config file → CLI flags, increasing; no scattered environment reads.
- **Rule 31** — Object-oriented design, one clear responsibility each; compose, inherit sparingly, and apply SOLID — especially SRP and Dependency Inversion — where it earns its keep.
- **Rule 32** — Every storage touch goes through the adapter; no raw paths, buckets, regions, or account IDs outside it.
- **Rule 33** — On-prem ↔ cloud is a config diff, never a source diff; deployment target is a parameter.
- **Rule 34** — "Use X locally" means X is the configurable default, never a hardcoded destiny.
- **Rule 35** — Empty config runs locally with zero setup where reasonable; production is what production config selects.

- **Rule 36** — Files target ≤500 lines (refactor past 800, never exceed 1000); functions ≤50 lines and ≤5 params; extract past ~3 levels of nesting.
- **Rule 37** — No god classes: more than ~7–10 public methods means a collaborator is missing.
- **Rule 38** — One non-trivial class per file; small helpers and DTOs may share.
- **Rule 39** — Ship a `.env.example` documenting every variable; gitignore the real `.env` from day one.
- **Rule 40** — Size refactors are separate, mechanical commits — moves only, reviewable in minutes.

Chapter 3 — Build



Standing on Chapters 1–2: - The hard rules hold everywhere: no hardcoded secrets, no destructive operations without confirmation, fail fast and loud, one purpose per commit — and the Powell rule: get 90% of the information, then decide; below 90%, ask the human. - Configuration lives in one validated layer — env to file to flag — validated at startup, never silently falling back to a different backend. - Anything with a local-vs-cloud or vendor axis sits behind a swappable interface, wired by dependency injection; research what already exists before writing original code. - Size ceilings are law: files target 500 lines, refactor past 800, never pass 1000 — split instead of stretching.

A design is only as good as the build discipline that implements it. Chapter 2 handed you the drawings — configuration in one validated layer, backends behind swappable interfaces, collaborators injected, files kept small enough to reason about. This chapter is where the drawings meet dirt: the code has to actually run, on machines you don't control, in containers you didn't configure, against failures you didn't anticipate, carrying packages you didn't write. Building is where design meets reality, and reality has opinions.

Twenty-one rules, ordered by what a violation costs you.

The gates and the failure-visibility rules lead, because their failures are the ones you can't undo and can't see. A deploy gate that's been quietly disabled (rule 41) re-opens every irreversible failure Chapter 1 closed — the unscanned artifact ships, and nobody decided it would. A swallowed error (rule 42) is undiagnosable forever: every production failure is eventually diagnosed at 3 a.m. by somebody squinting at whatever context your error handling bothered to keep, and that somebody is usually you, six months older, memory wiped. I learned this on systems where "attach a debugger" was not on the menu: when a box in a locked equipment room a thousand miles away misbehaves, the only witness is the log, and the log only knows what the code chose to tell it. The expensive failures

of my forty-seven years were almost never the exotic ones — they were ordinary errors that somebody's error handling quietly ate. Dependencies belong in this opening block too (rule 43), because a dependency is an error you haven't had yet: somebody else's code, release discipline, and security posture, adopted sight mostly unseen into your failure surface. You don't get to skip dependencies — nobody should write their own TLS stack — but you do get to pin them, lock them, and audit them, at the door and on a schedule. And a service that can't report its own condition or die cleanly (rule 44) is a service nobody can verify healthy — which Chapter 1's handover rule demands.

This is also where the book shows its hand. My container stack is Podman, UBI base images, and OpenShift; rootless and daemonless beats a root daemon, and rule 45 spends its words explaining why instead of just asserting it. Full disclosure, again: I work in enterprise open source, and this book is personal work — not sanctioned, not verified, not endorsed by my employer. I held these opinions before I worked there; arguably they're *why* I work there. Prefer Ubuntu, Arch Linux, and Docker's daemon? Write your own rules. The license genuinely lets you: fork the repo, swap rule 45, ship your own way. The portable part isn't the stack choice; it's having made one, on the record, with reasons.

The middle of the chapter is the things that have to actually work under load and across machines: operations that survive a re-run (rule 46), errors that stay diagnosable (rule 47), three operating systems and two architectures with CI standing guard, path libraries instead of string glue, real orchestration languages instead of shellisms, platform APIs instead of hardcoded directories, a logger instead of `print`, stdlib plus one good dependency instead of five small ones. I learned the portability discipline porting code between mainframe operating systems where "portable" was a punchline, and relearned it in embedded systems where the target hardware didn't exist yet when we started writing. It hasn't changed since 1979: you do not get to assume the path separator, the shell, the

architecture, the temp directory, or the init system. Every assumption you bake in is a bill that comes due on someone else's machine, at the worst possible time, usually mine. Software that only runs on the machine it was written on isn't done. It's a demo.

The chapter closes on the rules that keep a *distributed* system legible: time in UTC until the moment it's displayed (rule 56), the project-local virtualenv, every remote call carrying a timeout and a way out (rule 59), one trace ID per request (rule 60), and migrations that go down as cleanly as they go up (rule 61). The reversible-migration rule, like the virtualenv, is the kind you set up once and bless every time you need it.

The common thread is humility about machines and futures you don't control. Code that runs on any platform and any container stack, fails loudly with context, and carries as few dependencies as possible — that's the whole chapter in one sentence. The design was Chapter 2's promise; the build is where you keep it.

Rule 41: Gates never disabled by default

Pre-deploy gates (smoke test, vulnerability scan, secret scan) are never disabled by default — escape hatches are explicit, one-off, and logged.

The gates between a build and a deploy exist precisely for the days you're tempted to skip them. A smoke test that actually boots the image and hits the health endpoint from rule 44. A vulnerability scan over the artifact. A secret scan over the full deploy context — Chapter 4 territory, rule 65's full-artifact scan, but it bears restating here because deploy pressure is where that discipline dies. Each gate is boring ninety-nine times out of a hundred. The hundredth time is why they exist, and you don't get to know in advance which time is the hundredth.

The failure mode this rule targets isn't the missing gate — it's the *quietly disabled* one. Somebody sets `SKIP_SCAN=1` during an incident, it lands in the deploy script "temporarily," and eleven months

later you discover every deploy since has been unscanned. The fast path becomes the default by erosion, not decision. I've watched that sequence more than once. AI agents raise the stakes: an agent that reads a deploy script with a skip flag baked in will reproduce it forever, with perfect consistency and zero guilt.

So the rule's design: escape hatches *exist* — a gate with no override becomes a gate people route around entirely, which is worse — but they are explicit (a deliberate variable, named for what it skips), one-off (effective for a single invocation, never persisted into a script, an alias, or CI config), and logged (the deploy record shows which gate was skipped, when, and by whom). The asymmetry is the point: passing the gates is the silent default; skipping one is loud, attributable, and slightly embarrassing. That's the correct emotional gradient for a safety system.

Rule 42: Catch specific, never swallow

No bare `except:`/`catch (e)` swallows — catch specific exceptions, rethrow or log with context. Resource cleanup uses context managers / `defer` / `using` — no close-and-hope.

A bare `except: pass` is the single most expensive two-line construct in software. It costs nothing to write and weeks to pay for, because the bill arrives later, somewhere else, as state that quietly went wrong while the code insisted everything was fine.

The discipline has three parts. First, catch the *specific* exception you can actually handle — `FileNotFoundError`, `ConnectionRefusedError`, the typed error your library documents. A bare catch claims you can handle *anything*, including the typo you introduced last Tuesday and the interrupt the operator sent to shut you down cleanly. You can't, and pretending otherwise converts every future bug in that block into a silent one.

Second, when you catch, *do something honest*: handle it for real, or log it with context and rethrow. "Handle it for real" means the program is in a known-good state afterward — a retry, a documented

fallback, a clean abort of the unit of work. Logging the message and continuing is not handling; that's swallowing with a paper trail.

Third — and this is the part everyone skips — log with *context*. `"connection failed"` is useless at 3 a.m. `"connection to inventory-db at db-host:5432 failed after 3 retries during nightly reconciliation, last error: timeout"` is a diagnosis half-written. The exception object knows what went wrong; only your code knows what it was *trying to do*. Attach the operation, the relevant inputs, and the identifiers a human would search for — the order ID, not the exception class name.

If you genuinely must ignore an error — a best-effort cache warm, say — catch the narrow type and log at debug level *why* ignoring is correct. Intent, written down.

The same honesty applies to the resources a failed block leaves behind. Every handle you acquire — file, socket, lock, connection, temp directory — is a debt, and the question is what happens to it when the code between acquire and release throws. The manual open-work-close pattern answers: the debt is never paid, because the close call sits on the happy path and the happy path is exactly where exceptions don't go. So cleanup is structural, not hopeful — `with` in Python, `defer` in Go, `using` in C#, try-with-resources in Java, scope-bound drop in Rust. The shapes differ; the contract is identical: release is bound to acquisition at the moment of acquisition, and the runtime — not your memory of last month's early return — guarantees it runs on every exit path. The failure mode this prevents is nasty because it's slow: a leaked handle is invisible in dev, invisible in the demo, invisible in week one, and then the descriptor table fills or the pool drains and the system dies pointing nowhere near the leak. I once spent an unpleasant stretch of my embedded years chasing a box that hard-locked every nine days; the culprit was a cleanup call sitting below an error return. One line to fix; this rule is the structural version of that fix. In review, treat a manual close as a defect, not a style nit — the author isn't wrong

about today's control flow, they're wrong about every future edit to it.

Rule 43: Pin it and lock it

Pin versions and commit the lockfile; run a vulnerability audit periodically and on every new dependency.

An unpinned dependency is a build that changes underneath you while you sleep. "Install whatever's newest" means your repeatability is hostage to every maintainer's release schedule: the build that passed Friday fails Monday, and nothing in *your* repo changed. Debugging a breakage you didn't cause, in a package three levels down, is among the least dignified ways to lose a morning.

The fix is two artifacts doing two jobs. The **manifest** states your direct dependencies and the ranges you'll tolerate — this is what humans edit. The **lockfile** records the exact resolved version and integrity hash of *every* package in the transitive closure — this is what machines obey. Commit both. A lockfile that isn't committed is a diary that isn't written: the whole value is that your machine, your colleague's machine, CI, and the production image all install byte-identical dependency trees from it.

The lockfile buys three distinct things. **Repeatability** — today's checkout builds the same way in five years, and my software has tended to live decades. **Reviewability** — a dependency change becomes a visible diff in the pull request instead of a silent drift at install time; when something breaks, `git log` on the lockfile is your suspect list. **Integrity** — the recorded hashes make a tampered or republished package fail the install, converting a class of supply-chain attack into a loud error.

Upgrades still happen — deliberately. A version bump is its own commit (one purpose per commit), run against the full test suite, with the lockfile diff right there in review. The difference between that and an unpinned build is the difference between merging a change and being mugged by one.

Pinning freezes the tree; it doesn't make the tree safe. The world's knowledge of your dependencies' holes keeps moving after you lock them down — a tree that audited clean in March can be carrying a critical CVE by June without a single line of your code changing. The vulnerability was always there; the disclosure is what's new. So the lockfile gets a vulnerability audit on two triggers, and neither replaces the other. **At the door:** every new dependency is audited before it lands — `pip-audit` for Python, `npm audit` for Node, `govulncheck` for Go — across the whole transitive closure, because the package you chose is rarely the problem; it's the one *it* chose, three levels down, that ships the vulnerable parser. **On a schedule:** the same audit runs against the lockfile you already have — in CI on every build if it's cheap enough, on a weekly timer at minimum. That second trigger is the one people skip, because it generates unasked-for work — which is exactly its value: the gap between "CVE published" and "you found out" is the window in which you're vulnerable *and ignorant*, strictly worse than vulnerable and scrambling. Findings get triaged like bugs: a hit in code you actually execute gets fixed now (usually a one-line bump), a hit in an unreachable path gets documented as accepted so the next audit doesn't re-litigate it, and nothing gets silently waved through — an audit whose output is habitually ignored is theater that manufactures false confidence. The deploy gates in rule 41 already halt on findings; this is why they're entitled to.

Rule 44: Health endpoints and graceful SIGTERM

Services expose health/readiness endpoints and shut down gracefully on SIGTERM.

A service that can't report its own condition is a service somebody has to babysit, and that somebody bills by the hour. Two endpoints, two distinct questions. *Liveness*: is the process alive and sane, or should the platform restart it? *Readiness*: is it ready for traffic right now — dependencies reachable, caches warm, migrations done? The distinction is not pedantry: a service can be alive and momen-

tarily not ready, and conflating the two produces either restart loops or traffic routed into a black hole. While you're in there, `/health` also reports the version and build number, per Chapter 5 — when production misbehaves, "what exactly is running?" is the first question, and the service should answer it itself.

SIGTERM handling is the same courtesy at end-of-life. Every orchestrator — OpenShift included — stops a pod by sending SIGTERM, waiting a grace period, then sending SIGKILL. A process that ignores SIGTERM gets the kill: requests severed mid-response, buffers unflushed, locks orphaned. The graceful path is mechanical: catch SIGTERM, stop accepting work, fail the readiness probe so the router drains you, finish what's in flight, flush, exit zero. A few dozen lines, written once.

One trap from long scars: if your entrypoint wraps the process in a shell (rule 55 nods knowingly), the shell may eat the signal and your handler never fires. Exec the real process as PID 1 or use a proper init shim. Then *test* the shutdown path — Chapter 4's branch-coverage rule applies to the exit ramp too. A service that dies cleanly is a deploy that's boring, and boring deploys are the entire goal.

The container lifecycle: nothing deploys without passing the gates, nothing serves before readiness is green, and nothing dies without draining first.

Rule 45: Container-friendly by default

Container-friendly by default: config from env or mounted files, logs to stdout, no assumed persistent disk — and least privilege at every layer. Run as a non-root user with the smallest capability set that works, mount the root filesystem read-only where the workload tolerates it, leave mandatory

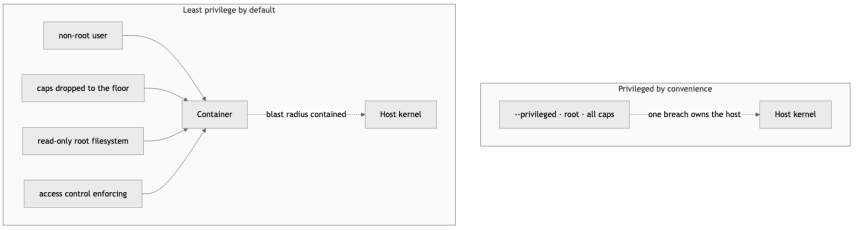
access control in enforcing mode, and never reach for `--privileged`. Rootless and daemonless beats a root daemon. The secure path has to be the default path, not the option you remember to turn on.

The first half is uncontroversial: write services as if a container will run them, even when one won't. Configuration arrives via environment or mounted files (Chapter 2 already made you do this). Logs go to stdout for the platform to collect — no log files, no rotation logic, no in-process log shippers. Local disk is scratch unless explicitly declared otherwise. Code written this way runs identically on a laptop, in a container, and on bare metal — rule 53 and Chapter 2's config layer doing their job.

The second half is one idea applied all the way down: least privilege beats convenience. A daemon that runs as root and owns every container on the host is one socket that's effectively a root API — a single point of failure an auditor circles in red. I spent years building equipment where a root-level compromise was a national-security conversation, and I cannot unsee that diagram. Prefer a runtime where containers are ordinary child processes of an ordinary user: no daemon, no root, nothing to compromise that you didn't already own. Drop capabilities to the floor and justify each one you add back. Run as a non-root user. Mount the root filesystem read-only so a compromised process has nowhere to write its toolkit. Leave the mandatory-access-control layer enforcing — `--privileged` and a disabled access-control layer are the two switches every "just make it work" guide tells you to flip, and exactly the two that turn a contained breach into a host breach.

Which runtime, which base image, which orchestrator is a *preference*, not an axiom — it belongs to the layer Chapter 7 calls *employer* (or *personal*), not the universal core. Mine, as an example: Podman, UBI base images, OpenShift, Ansible — rootless and daemonless by construction, a maintained and freely redistributable foundation with a vendor security pipeline behind the images, and pods that

run as non-root UIDs by default. That is an example, not a ruling. Plenty of careful people run hardened Docker or Ubuntu and sleep fine — CC-BY-4.0, swap the stack, ship your way. Just make it a decision held to the same least-privilege bar, not a drift you backed into.



Two postures, same workload. Privileged-by-convenience makes one breach a host breach; least-privilege-by-default contains it. The brand of runtime is a preference — see the layers in Chapter 7 — but the posture is the rule.

Two architectures: on the left, every container funnels through one privileged daemon; on the right, containers are ordinary user processes with nothing root-owned to compromise.

Rule 46: Make it idempotent

Make every operation idempotent and safe to re-run. Deploys, migrations, and setup scripts must converge to the same state run once or five times, and resume cleanly after an interruption. Gate on the real end-state, never on a partial artifact that merely looks "done."

The question that defines this rule is the one nobody asks until 2 a.m.: what happens when this runs *again*? A deploy that times out halfway, a migration the network dropped on, a setup script someone Ctrl-C'd and re-ran — these aren't edge cases, they're Tuesday. An operation that only works the first time, on a clean slate, with nothing already half-done, is an operation that has scheduled its own outage. The right design makes running twice indistinguishable from running once: create the table *if it doesn't*

exist, add the user *if absent*, upload the object whose content hash you can re-check, write to a temp path and atomically rename so a reader never sees half a file. Convergence, not assumption.

The trap inside the trap is the false done-signal. The seductive shortcut is to leave a marker — a `.done` file, a row in a status table, an existing output path — and skip the work if the marker is there. But a marker says "I started," not "I finished," and the gap between those two is precisely where the interrupted run lives. I once inherited a nightly job that checked for the output directory and bailed if it existed; a crash mid-write left the directory there, empty, and the job cheerfully skipped itself for nine days while every downstream consumer read stale data and nobody got paged, because nothing had *failed* — it had succeeded at doing nothing. Gate on the real end-state: the row count matches, the checksum verifies, the health probe is green. Anything you can fake by touching a file is not a completion check; it's a lie you told yourself in advance.

AI agents make this non-negotiable. An agent retries on timeout without a flicker of doubt, and an operation that isn't safe to re-run becomes an operation that double-charges, double-writes, or double-deploys — at machine speed, several times, before anyone notices.

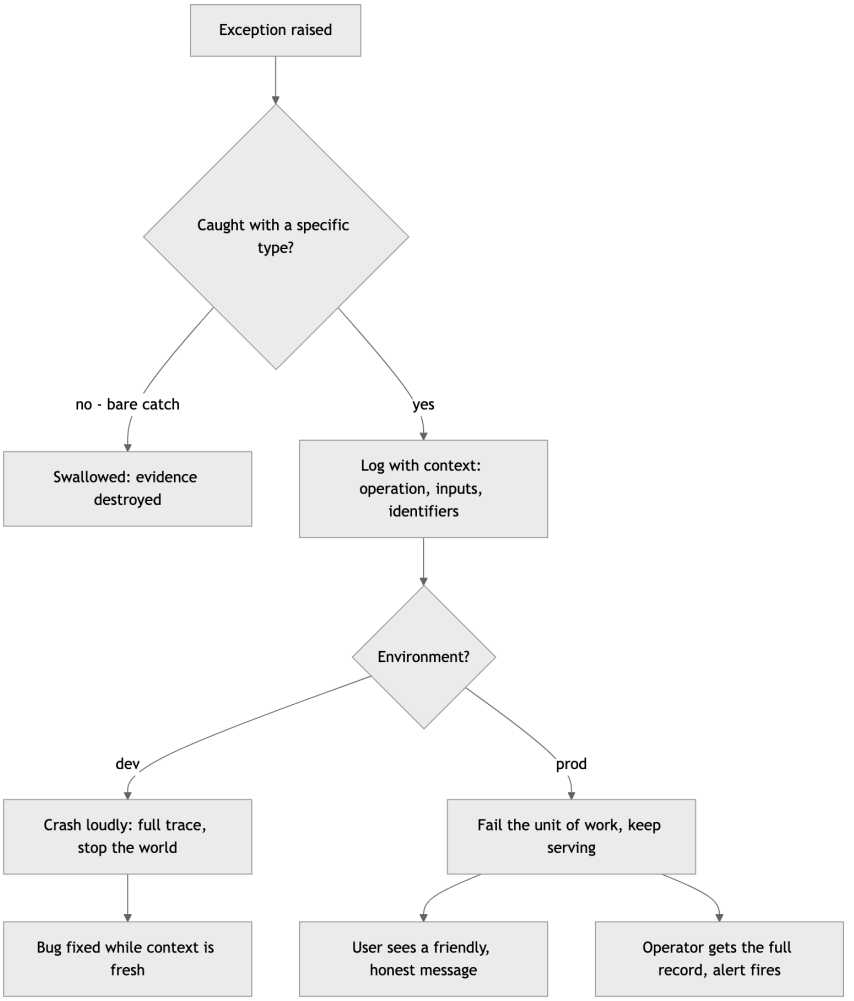
Rule 47: Loud in dev, graceful in prod, diagnosable always

Fail loudly in dev, gracefully in prod, diagnosably always.

These sound like opposites. They aren't — they're the same principle applied to two different audiences.

In development, the audience is the person who just caused the bug, context fresh. Serve them the failure at full volume: crash, print the whole trace, stop the world. Every error you soften in dev is a bug you've granted a visa to production. The fail-fast hard rule from Chapter 1 is this rule's blunt cousin: a dev environment that limps along after an error is a bug-laundering operation.

In production, the audience splits in two, and you owe each a different artifact. The *user* gets graceful: the request fails cleanly, the rest of the system keeps serving, the message is honest and useful — not a stack trace, which to a user is both gibberish and an information leak. The *operator* gets diagnosable: the full exception, the context from rule 42, the structured fields from rule 49, captured at the moment of failure. Graceful degradation without diagnosis is the worst outcome of all — the system absorbs errors so politely that nobody notices it's been failing for a week.



The error-propagation ladder. The dev path and the prod path diverge only at the last step — what to show, and to whom. The capture step before the fork is identical, which is the point.

The word "diagnosable" carries the whole rule. Whatever the audience-facing behavior, the record is complete. If you can't reconstruct what happened from the logs alone — no debugger, no reproduction, no guessing — the error handling failed, even if the catch block ran perfectly.

Rule 48: Three platforms, two in CI

Target macOS, Linux, and Windows; CI covers at least two of them.

The priority order is macOS first (that's where I develop), Linux second (that's where everything deploys), Windows third (that's where a surprising number of users actually live). But priority order is not permission to skip. "We'll add Windows support later" is the most reliable lie in software — later never comes, and by the time someone forces the issue, platform assumptions have metastasized through the codebase like rebar through concrete.

The CI requirement is the enforcement mechanism, deliberately set at two platforms rather than three. Three is better when CI minutes are cheap. But two is where the discipline actually engages, because the single most dangerous configuration is one platform: every assumption passes, forever, until the day it doesn't. The moment a second OS enters the matrix, the whole class of "works on my machine" bugs starts failing loudly in CI instead of quietly in production. Path separators, case-sensitive filesystems, line endings, file locking, signal handling — the second platform catches most of them, because platform bugs are rarely Windows-specific or Mac-specific; they're *assumption*-specific, and any second opinion exposes the assumption.

I once watched a team — no names, they know who they are — discover after eighteen months that their entire test suite depended

on a case-insensitive filesystem. The fix took three weeks. A Linux runner in CI would have caught it on day one for the cost of a YAML stanza. That's the trade this rule encodes: a few minutes of CI time per commit, against weeks of archaeology later. It's not close.

Rule 49: A logger, never print

Use a logger, never `print`, in shipped code; log level configurable, and structured (JSON) once the project outgrows a script.

`print` is fine in a throwaway script and a liability everywhere else, for one structural reason: it has no controls. No level, no timestamp, no source, no off switch. It is a statement hardcoded to shout at whoever happens to be holding the terminal — and in production, nobody is holding the terminal.

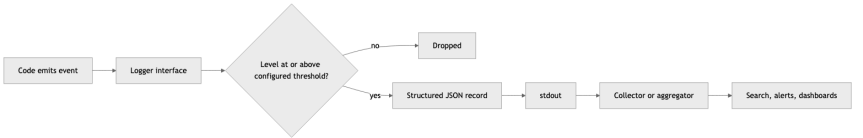
A logger gives you the four things `print` can't. **Severity**, so the reader can tell "routine" from "the building is on fire" without reading every line. **Provenance** — timestamp, module, process — attached automatically, so you never play the "which of our six services printed this?" game. **Routing**, so output goes to stdout, a file, or an aggregator without touching the call sites. And **volume control**: a configurable level means the same binary runs quiet in production and verbose on the box where you're chasing a bug.

That last word — *configurable* — is doing real work, and it ties back to Chapter 2. The log level is an environment variable or a config key, never a constant. The debugging session that requires editing source and redeploying just to see debug output is a debugging session that starts an hour late. I have watched that hour get lost on systems where redeploying meant a change window and a sign-off sheet.

Every mainstream language ships a logging facility in the standard library or one boring, universal package. Set it up in the first hour of the project, wire its level to config, and `print` never gets a chance to metastasize. Retrofitting a logger into a codebase with four hundred

print statements is an afternoon of mechanical penance; doing it on day one is five minutes.

Past a certain size the *format* matters as much as the facility. Human-readable log lines are a love letter to a human who isn't coming: in any real deployment the first reader of your logs is a machine — an aggregator, a search index, an alerting rule — and machines are terrible at reading prose. `"User 4711 failed login from 10.0.0.7 (attempt 3)"` needs a regex to query; the same event as JSON — `{"event": "login_failed", "user_id": 4711, "source_ip": "10.0.0.7", "attempt": 3}` — needs nothing but a field name. The moment you want "all failed logins for this user across all services last week," structure is the difference between a ten-second query and an afternoon of grep archaeology. The threshold is "more than a script": a fifty-line tool you run by hand can print prose, but the moment the thing runs unattended, has more than one component, or emits logs somebody else reads, switch to structured output — one formatter configured at startup, call sites barely changed. Two habits make it earn its keep: **consistent field names** across the codebase (`user_id` everywhere, not `uid` here and `userId` there — a five-line conventions note prevents the schema drift that makes cross-service queries quietly lie), and **log events, not sentences** (an event name plus fields has one shape; an English sentence with values interpolated has infinite variants). And per rule 45, the destination is stdout — the platform owns shipping the bytes, your process owns making them queryable.



The logging pipeline: code talks to one logger interface; level filtering, JSON structuring, and routing happen downstream of the call site — which is why call sites never need to change when the destination does.

Rule 50: AI errors surface; agents don't invent tools

AI/LLM errors surface to the user as friendly messages — never a silent failure, never a raw stack trace. And agents only call tools that actually exist in their tool list — never fabricate one.

This is rule 47 specialized for the AI era, with a clause the AI era made necessary.

When an LLM call fails — backend down, rate limit, context overflow, malformed response — two failure modes are tempting and both are wrong. The silent one: the feature degrades, a summary comes back empty, generated text just doesn't appear, and the user concludes the product is broken in some vague way they can't report. The loud-but-useless one: a raw provider stack trace lands in the UI, exposing internals to someone who just wanted a summary. The correct behavior is the prod path from rule 47: tell the user plainly that the assistant is unavailable and what to do about it, while the operator's log captures the provider, model, status code, and request context. LLM backends fail *constantly* compared with databases — treat their failure handling as a first-class feature, not an edge case.

The second clause is about agents calling tools. A model under pressure will sometimes invent a plausible-sounding tool name — `search_codebase`, `run_linter` — that simply isn't there. Each hallucinated call burns tokens, fails, and stalls the session in a retry loop. The rule for any agent operating under this book: the tool list is a contract, not a suggestion. If the tool you want doesn't exist, fall back to the primitives that do — shell, file reads — or stop and ask for the tool to be wired in. Fabricating a tool call is calling a function you never imported, with no compiler to catch it — the discipline has to live in the rules instead.

Rule 51: Path libraries, never string glue

Use the language's path library — never string-concatenate paths or hardcode separators — and enforce LF line endings via `.gitattributes`.

Every mainstream language solved this problem years ago. Python has `pathlib.Path`, Node has the `path` module, Go has `filepath`, and even shell — when you must use it — has constructs better than mashing strings together with a `/` and a prayer. The rule is simply: use them. Always. `dir + "/" + name` is a bug that hasn't been scheduled yet.

What people underestimate is how much more than the separator these libraries handle. Drive letters. UNC paths. Trailing-slash normalization, `..` resolution, symlink semantics, the difference between joining and resolving, the difference between an absolute path on one OS and the same string being relative on another. A hand-rolled string concatenation gets the happy path right and every edge wrong, and the edges are exactly where filesystem code goes to die — usually in a deletion routine, which is the one place you really want the path math correct.

There's a deeper benefit: path objects make intent legible. `config_dir / "settings.toml"` tells the reader a filesystem path is being composed, not arbitrary string fiddling. When an AI agent writes or reviews the code, that legibility matters double — the agent that sees consistent `pathlib` usage continues the pattern; the agent that sees string mashing happily produces more of it. Conventions propagate; pick the one you want propagated.

The hardcoded-separator clause has no exceptions clause, because I've heard all the exceptions and they were all wrong. "It's Linux-only" — until it isn't, see rule 48. "It's just a quick script" — quick scripts are where production code is born. Use the library.

There's a cousin defect one layer down — the bytes *between* the path components, the line endings — and it gets settled the same way: by

versioned policy, not by every contributor configuring their editor. CRLF-versus-LF is a 1970s teletype dispute modern software still trips over. A shell script with CRLF endings fails on Linux with errors that mention nothing about line endings (`/bin/sh^M: bad interpreter` if you're lucky, silent misbehavior if you're not). A diff balloons to every-line-changed because one editor "helpfully" converted the file, burying the real change under a thousand lines of invisible noise — and that one matters more than it looks, because per Chapter 4 you're inspecting diffs of config-shaped files for leaked secrets before every commit, and a whole-file line-ending rewrite is excellent camouflage for the one line that actually changed. Per-machine `core.autocrlf` is a promise, and this book doesn't run on promises. The fix is a two-line `.gitattributes` in the repo, versioned and enforced by git itself, identical for every clone on every OS:

```
* text=auto eol=lf
*.bat text eol=crlf
```

Normalize everything to LF in the repo and on checkout, with a carve-out for the rare format that genuinely requires CRLF — Windows batch files being the canonical example. It's the cheapest line in the book: a one-time commit that permanently deletes an entire bug category across every platform, editor, contributor, and coding agent that will ever touch the repo.

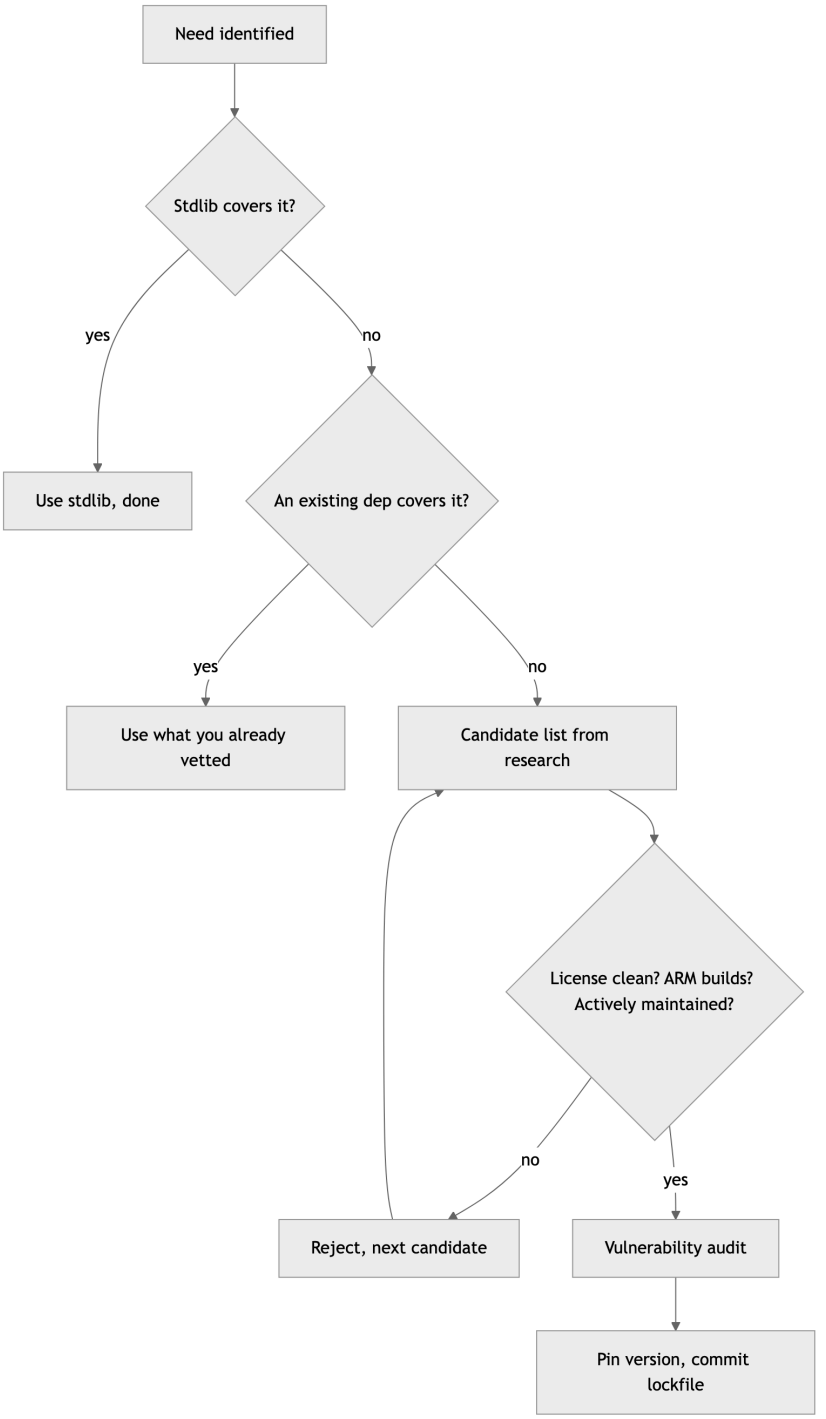
Rule 52: Stdlib plus one good dependency

Prefer stdlib plus one well-maintained dependency over five small ones.

Every dependency is recurring overhead dressed up as a one-time convenience. The install is free; the audits, upgrades, compatibility matrix, license tracking, and eventually-abandoned-maintainer succession planning are forever. So the arithmetic favors fewer, better packages: one well-maintained dependency means one changelog to read, one security advisory feed, one project whose health you track. Five small ones means five of each, plus the

interactions between them. Small packages are likelier to be one volunteer's weekend project, to go quiet, to be the soft target in a supply-chain attack. The industry already ran this experiment with an eleven-line string-padding package; it went poorly.

The decision sequence, in order. **Stdlib first** — modern standard libraries cover HTTP, JSON, paths, concurrency, and far more than most people who learned the ecosystem a decade ago remember; check before you shop. **Existing dependencies second** — the well-maintained package you already vetted often covers the new need in a corner you haven't read about. **A new, healthy dependency third** — active maintenance, real adoption, responsive issue tracker, clean license, ARM builds (per rule 54 later in this chapter). **Writing it yourself last** — consistent with the research-first rule in Chapter 2; original code is the fallback, not the default, but for twenty lines of glue it beats adopting a stranger's repo and their next five years of CVEs.


Need identified
Stdlib covers it?
yes
no
Use stdlib, done
An existing dep covers it?
yes
no
Use what you already vetted
Candidate list from research
License clean? ARM builds? Actively maintained?
no
yes
Reject, next candidate
Vulnerability audit
Pin version, commit lockfile

The dependency vetting funnel. Most needs should die in the first two gates; only the survivors earn a slot in the lockfile.

The funnel looks like bureaucracy. It's the opposite: ten minutes at the top saves the slow-drip years at the bottom.

Rule 53: No hardcoded temp, home, or drive letters

No hardcoded `/tmp`, `~/`, or drive letters — use platform temp/home APIs.

`/tmp` doesn't exist on Windows. `~/` expands in your shell, not in your code — pass it unexpanded into a file API and you'll create a literal directory named ~ in your working directory, which I have personally watched happen in a production deploy, followed by a cleanup script that did exactly what you fear it did. `C:\Users\eddie\` works precisely until the code runs as a service account, in a container, or on literally anyone else's machine. These three hardcodings are siblings of the same defect: confusing *your* environment with *the* environment.

Every platform exposes the real answers through an API. Python: `tempfile.gettempdir()`, `tempfile.TemporaryDirectory()`, `Path.home()`, and `platformdirs` when you need proper per-OS config and cache locations. Node: `os.tmpdir()` and `os.homedir()`. Go: `os.TempDir()`, `os.UserHomeDir()`. They cost the same number of characters as the hardcoded string and they're correct on every machine, including the ones you haven't met yet.

Containers make this sharper, not softer. Inside one, `/tmp` may be a size-limited tmpfs, a read-only layer, or scratch space that vanishes between requests; the home directory of an arbitrarily-assigned UID — exactly what OpenShift gives you by default, see rule 45 — may not exist at all. Code that asks the platform survives this; code that assumes a path becomes a 2 a.m. page.

This rule is also a config rule wearing a trench coat: per Chapter 2, any path that could plausibly vary belongs in configuration with a platform-API default. The API gives you correct-by-default; the

config layer gives you operator override. Hardcoding gives you neither, for the same effort.

Rule 54: Both architectures, flagged exceptions

Target arm64 and x86_64; flag any dependency without native ARM builds and document the workaround.

There was a comfortable decade when "architecture" meant x86_64 and the question never came up. That decade is over. Developer laptops are ARM. A growing share of cloud instances are ARM, often the cheapest compute on the price sheet. Embedded and edge targets have been ARM all along — I was shipping to non-x86 silicon back when that meant arguing with a cross-compiler that hated me personally. Single-arch software in 2026 has voluntarily disqualified itself from half the machines your users own.

Most of the time this rule costs nothing. Interpreted code doesn't care, and mainstream compiled ecosystems publish multi-arch artifacts as a matter of course. The rule exists for the exceptions: the dependency with no ARM wheel, the vendor SDK shipped as an x86_64-only binary blob, the native extension nobody has rebuilt in a decade. One such dependency silently converts your entire project to single-arch, and you typically discover it during a deploy, under time pressure, via an error message that mentions none of this.

Hence the second clause, which is the part people skip: *flag it and document the workaround.* Don't quietly add the dependency and hope. Say, in writing, in the repo: this package has no native ARM build; here is the emulation path or the pinned alternative or the build-from-source recipe; here is what it costs us. Per rule 9 back in Chapter 1, every dependency declares its platform support before it gets in the door. An exception you've documented is a decision. An exception you've hidden is a time bomb with my name on the casualty list.

	macOS	Linux	Windows
arm64	Primary dev target	Deploy + CI	Supported, tested per release
x86_64	Supported	Deploy + CI	Supported, CI when feasible

The platform-by-architecture support matrix: six cells, no blanks, and CI standing guard on at least two operating systems.

Rule 55: No shell-isms — orchestrate in Python or Node

No shell-isms in cross-platform scripts; orchestrate in Python or Node, not bash.

I have written more shell than most people have read, and that is exactly why this rule exists. Shell is a terrific interactive tool and a treacherous programming language. The moment a script grows a conditional, a loop, or an error-handling requirement, it has outgrown the shell — and a cross-platform script outgrew it before the first line was written, because there is no cross-platform shell. There's bash, which isn't on Windows; there's whatever `/bin/sh` resolves to today; there's PowerShell, which is a different universe; and there's the special hell of a script that works in bash 5 and fails silently in bash 3 on a stock Mac.

The banned shell-isms are the usual suspects: `&&` chains as control flow, `source` for configuration, backtick soup, word-splitting tricks, trusting `set -e` to mean what you hope it means (it doesn't, and the ways it doesn't fill a small book of their own). Each is a portability bug or an error-swallowing bug or, on a good day, both.

The fix is to orchestrate in a real language. Python and Node are everywhere your code already runs; both give you actual data structures, actual exceptions, actual exit-code checking on subprocesses, and the path libraries from rule 51. A build script in

Python is testable — you can mock the subprocess layer and assert the orchestration logic, which connects straight to the coverage rules in Chapter 4. Try unit-testing a 300-line bash script and report back.

Keep shell for what it's good at: one-liners, interactive use, the glue inside a CI step that's literally three commands. The moment logic appears, promote it to a real language. The promotion is always cheaper today than after it breaks.

Rule 56: Time is UTC until it's displayed

Store, log, and compute time in UTC; convert to local only at display. Never persist a naive local timestamp.

A timestamp without a timezone is a number pretending to be a fact. `2026-03-08 02:30:00` looks complete and isn't: it doesn't say *whose* 2:30, and on the morning the clocks spring forward it names an instant that doesn't exist, while six months later another local string names two instants and refuses to say which. Persist that ambiguity into a database, a log, or a serialized message and you've stored a riddle you'll have to solve later, usually while reconstructing an incident timeline across three regions where the events refuse to line up.

The discipline is one-directional and absolute. The moment a time enters your system — captured, computed, received — it becomes UTC, and it stays UTC through every store, every log line, every wire format, every comparison and sort. Local time is a *rendering*, produced at the last possible step, in the user's zone, for human eyes only, and never written back. UTC has no DST, so it never skips an hour or repeats one; "before" and "after" mean what they say; durations are honest subtraction; and the same instant read by a server in Dublin and a server in Denver is the same number, not a negotiation.

The naive-timestamp trap is the specific one to name: in most languages the default "now" hands you a local datetime with no zone

attached, the most dangerous shape there is, because it *looks* fine in every test you run on the machine that made it. Reach for the time-zone-aware UTC call instead — `datetime.now(timezone.utc)`, not `datetime.now()` — and store the offset or a `Z`. I have watched a "two-hour gap" in a log turn out to be a one-second event that crossed a DST boundary between two services that disagreed about what "local" meant. UTC end to end; convert at the glass, nowhere else.

Rule 57: Project-local virtualenvs, always

Python work always uses a project-local virtualenv — never install into the system Python.

The system Python is load-bearing infrastructure that happens to look like a programming environment. On most Linux systems the OS tooling itself runs on it; on any developer machine, a half-dozen unrelated projects are one `pip install` away from sharing — and corrupting — a single global package namespace. Install project A's pinned framework version globally, and project B, which pinned a different one, breaks at a distance, with an error message that names neither project. Modern distributions now refuse bare global installs outright — a rare case of the platform enforcing my rules for me.

The virtualenv is the fence: a project-local environment, in the project directory, holding exactly what the lockfile says and nothing else. Every project gets its own; no exceptions for "it's just a script" — the just-a-script projects are precisely the ones that resurface in three years, and rule 43's lockfile can only reproduce an environment isolated enough to be *described*. The virtualenv directory itself is disposable and gitignored: the lockfile is the source of truth, the environment is a build artifact you can delete and regenerate without ceremony. If deleting it scares you, your lockfile is lying.

Mechanically this costs one command at project start and roughly zero thereafter. The companion discipline is *never invoking the bare*

system interpreter for anything that imports project dependencies: run through the environment, every time, in the shell and in CI alike, so the environment that passes tests is the environment that ships. Other ecosystems get this for free — Node's `node_modules` is per-project by construction. Python makes you ask. Ask every time.

Rule 58: Show progress; cached by default, expensive by exception

Show progress on unavoidably slow operations and say why; cached paths are the default, expensive paths are explicit and rare.

I came up through real-time systems, where latency was a specification with consequences, so I'll admit a bias: most slow software is slow because nobody was made to feel it. But some operations are honestly slow — a vulnerability scan, a model download, a cold index build — and for those, the sin isn't the latency. It's the silence. A spinner with no explanation is a lie of omission: the user can't tell ten seconds from ten minutes, can't tell progress from a hang, and will eventually kill the process at the worst possible moment — usually mid-write. Tell them what's happening and why: "Scanning 1,432 packages for known CVEs — about 90 seconds, runs on every deploy." Now the wait is a decision they're in on, not a hostage situation. Counters and step names beat percentages you're making up; an honest "about two minutes" beats a progress bar that sprints to 90% and parks there waiting for applause.

The second clause is the architectural half. The default path through your software should be the cached, precomputed, fast one; the expensive path should be something you *choose*, visibly — a `--rebuild` flag, a "Refresh all" button, a scheduled job — never something you stumble into because a cache key went stale. When the expensive path does run, it announces itself, which closes the loop with the first clause.

This applies with extra force to agent-driven workflows. An AI agent staring at a silent subprocess can't distinguish slow from hung any better than a human can — worse, it may time out and retry, turning one expensive operation into four. Progress output isn't cosmetics; it's an interface contract with everything supervising the process, carbon or silicon.

Rule 59: Every remote call has a timeout and a way out

Every remote call has a timeout, bounded retries with backoff, and a fallback or circuit-breaker — no unbounded waits, no naive infinite retry.

The default behavior of most network clients is to wait forever, and forever is not a latency budget. A remote call is a bet that another machine — its process, its network path, its load balancer, its mood — will answer; the rule exists because that bet loses constantly, and the question is only whether your code loses gracefully or hangs. A call with no timeout is a thread you've lent to someone who may never give it back. Do that under load and the pattern is always the same: one slow dependency, every worker blocked waiting on it, a pool exhausted, and a service that's "down" while every box in it sits at near-zero CPU, politely waiting. I have watched a healthy front end taken fully dark by a single sluggish backend it called synchronously with no deadline. The fix was a timeout — measured in milliseconds, not the absence of one.

So every remote call carries three things. A **timeout**, set deliberately to a number you can defend, not left to the library's "infinite." **Bounded retries with backoff** — a transient blip deserves a second try, but a fixed three attempts with exponential backoff and a little jitter, never the naive `while not success: retry()` that turns one struggling server into a self-inflicted denial-of-service the moment every client retries in lockstep. And a **way out** — a fallback value, a cached answer, a degraded-but-honest response, or a circuit-breaker that, after enough failures, stops calling the dead

dependency entirely for a cool-down and fails fast instead of slow. Fast failure with a fallback beats slow failure with a stack trace every time.

This is the same fail-fast instinct from Chapter 1, pointed at the network: don't limp, don't hang, decide. The unbounded wait is the network's version of the silent swallow — it looks like patience and behaves like a hang.

Rule 60: One request, one trace ID

Every request carries a correlation/trace ID, propagated across services and logged, so one request can be followed end-to-end.

The day you split one process into two, you lose the thing a single log file gave you for free: the ability to read one request's whole story in one place. A user reports "it failed," and the failure lives somewhere across an API gateway, three services, a queue, and a worker — each with its own logs, each timestamped slightly differently, none of them aware the others exist. Without a shared thread, reconstructing that one request is correlation by eyeball: matching timestamps and guessing, which is exactly as reliable as it sounds.

The thread is a trace ID. Mint one unique ID at the system's edge — the first service to touch the request — and propagate it through every downstream hop: in the HTTP header to the next service, in the message metadata onto the queue, into the context the worker picks up. Every log line everything writes includes that ID as a field (which is why rule 49′s structured logging is the precondition — a trace ID buried in a prose sentence isn't queryable). Now "show me everything that happened to request `abc-123`" is one filter across the aggregated logs, and the whole journey assembles itself in order, across every service, no eyeball correlation required.

The cost is small and the discipline is propagation: the ID has to survive every boundary, so generate it once and pass it explicitly — most frameworks have middleware that injects and forwards it,

and the common standard is W3C Trace Context, worth adopting rather than inventing your own header. The trap is the broken chain: one service that drops the incoming ID and mints a fresh one severs the trace exactly where you most need it intact. I have lost real hours to a request that "vanished" at a service boundary — it hadn't vanished; the ID had, and so the logs on the far side were orphans I couldn't match to anything. Generate at the edge, forward everywhere, log on every line.

Rule 61: Migrations go down as well as up

Migrations are reversible and idempotent: every schema or data migration ships a tested down-path and is safe to re-run. An irreversible one-way change needs explicit sign-off.

A migration with no way back is a deploy you can't roll back. The up-path is the one everyone writes and tests, because it's the one that delivers the feature; the down-path is the one nobody writes, until the up-path half-applies in production at the worst possible moment and the only honest answer to "can we revert?" is a long silence. Every schema or data change ships its reverse in the same commit — add a column, the down drops it; rename a table, the down renames it back — and the down-path is *tested*, applied and reverted in CI against a real schema, not assumed to work because it looks symmetrical. A down migration nobody has ever run is a parachute nobody has ever packed.

Reversibility's twin is idempotence — this is rule 46 wearing a database hat. A migration runner can die mid-apply, get retried, or run against a database where a previous attempt got halfway; the migration that survives all three converges to the same end-state whether it runs once or three times. Guard each step on the real state — does the column already exist, is the backfill already done — and gate on that, never on a "migrations" bookkeeping row that claims success a crash may have written prematurely.

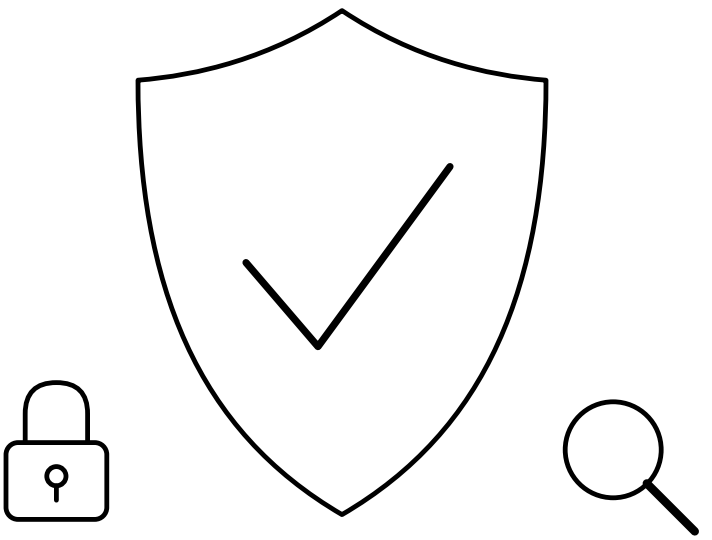
Some changes genuinely can't reverse: dropping a column destroys its data, a destructive backfill overwrites the original. Those aren't forbidden — they're *gated*. An irreversible migration requires explicit human sign-off (Chapter 1's destruction rule, applied to your schema), ideally staged behind a reversible deprecation first: stop writing the column, ship and bake, *then* drop it in a later migration once you're certain nothing needs the way back. One-way doors get opened deliberately, by a human, never by an agent on autopilot at 2 a.m.

Chapter 3 card

- **41.** Pre-deploy gates are never disabled by default — escape hatches are explicit, one-off, and logged.
- **42.** Catch specific exceptions; rethrow or log with context — never a bare swallow; cleanup is structural (`with` / `defer` / `using`), never close-and-hope.
- **43.** Pin versions and commit the lockfile; audit for vulnerabilities at the door and on a schedule.
- **44.** Services expose health/readiness endpoints and shut down gracefully on SIGTERM.
- **45.** Container-friendly by default — config from env, logs to stdout, scratch disk; least privilege at every layer (non-root, caps to the floor, read-only root fs, access control enforcing, no `--privileged`). The brand of stack is a preference — see Chapter 7.
- **46.** Make every operation idempotent and safe to re-run; gate on the real end-state, never a partial artifact that looks "done."
- **47.** Fail loudly in dev, gracefully in prod, diagnosably always.
- **48.** Target macOS, Linux, and Windows; CI runs on at least two of them.
- **49.** Logger, never `print`, in shipped code; log level set by config, structured JSON once it outgrows a script.
- **50.** AI errors reach the user as friendly messages; agents call only tools that exist.

- **51.** Use the language's path library — never string-concatenate paths or hardcode separators — and enforce LF endings via `.gitattributes`.
- **52.** Stdlib plus one well-maintained dependency beats five small ones.
- **53.** No hardcoded `/tmp`, `~/`, or drive letters — use platform temp and home APIs.
- **54.** Target arm64 and x86_64; flag any dependency without native ARM builds and document the workaround.
- **55.** No shell-isms in cross-platform scripts; orchestrate in Python or Node, not bash.
- **56.** Store, log, and compute time in UTC; convert to local only at display — never persist a naive local timestamp.
- **57.** Project-local virtualenv, every project — the system Python is not yours.
- **58.** Show progress on slow operations and say why; cached paths are the default, expensive paths are explicit and rare.
- **59.** Every remote call has a timeout, bounded retries with backoff, and a fallback or circuit-breaker — no unbounded waits, no naive infinite retry.
- **60.** Every request carries a correlation/trace ID, propagated across services and logged, so one request can be followed end-to-end.
- **61.** Migrations are reversible and idempotent — tested down-path, safe to re-run; an irreversible one-way change needs explicit sign-off.

Chapter 4 — Protect and Prove



Standing on Chapters 1–3: - **Ch 1, First Principles** — the hard rules are absolute (no commit without a scan, no destruction without confirmation), and the Powell rule governs the whole crew, human and agent alike. - **Ch 2, Design** — everything that can change flows through one config layer; every backend hides behind a swappable interface, handed in by constructor injection, never reached for. - **Ch 3, Build** — builds are cross-platform and container-friendly, least-privileged by default; errors fail loudly with context; dependencies are pinned, minimal, and audited.

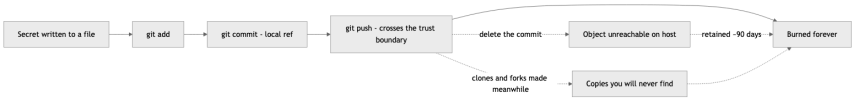
Chapter 3 left you with software that builds — on every platform, in a least-privileged container, with errors that announce themselves and dependencies you can name and count. Built software is unproven software. A green build proves only that the compiler had no objections, and the compiler has never once been asked the two questions that matter before anything ships: is there something radioactive in this artifact, and does the thing actually work? Those are the two proofs you owe before any push, deploy, or demo, and this chapter is both of them.

The first proof is negative. Every other chapter in this book describes mistakes you can recover from — a bad merge reverts, a god class refactors, a broken deploy rolls forward. Git is a machine for undoing things, which is exactly why I trust it with so much autonomy elsewhere in these rules. A leaked credential is the one mistake the machine cannot undo, because it is not a state of the repository; it is a state of the world. The moment a key crosses a trust boundary — a remote, a registry, a cluster, a host you don't own — copies exist that you will never enumerate and never delete. The major hosts retain "deleted" objects for around ninety days; forks and clones retain them forever. And the leak never arrives in a file called `secrets.txt`. I once watched a competent team burn a production API key inside thirty harmless lines of deployment YAML — a port change, a memory limit, and one environment vari-

able with a literal token pasted in during debugging. The reviewer saw "config churn" and approved it. The scanner that would have caught it was installed the following week.

The second proof is positive, and it rests on a line I first heard from a quality engineer decades ago: you get what you inspect, not what you expect. On that program the hardware got X-rayed and the software got a shrug and a demo — and the software is what came back from the field, on a branch of the retry logic that had never executed even once until a customer's flaky link executed it for us. For most of my career the honest objection to full inspection was cost: humans get bored writing the 400th test case, so we rationed, and every coverage target below 100% was a documented decision about which branches we'd rather discover in production. That economics is dead. A machine does not get bored writing the 4,000th test case. So I require total inspection — contract first, 100% line *and* branch coverage — and because coverage proves only that code was inspected, not that the software is good, I also require a grade: a written rubric where 90 is the working bar and nothing publishes below 95.

Two proofs, fourteen rules, one discipline — and the order runs by stakes, not by subject. Containment opens, because nothing else matters if the artifact is radioactive: the hooks that prevent the leak lead (Rule 62), and the rotation protocol and the no-copying rule follow (Rules 63 and 64), because those are the moments of maximum irreversibility — the ones where the right move has to be decided before the emergency. Rule 65 is the single scan-at-every-boundary gate; Rule 66 is the grade — the 90/95 rubric bar that decides whether anything ships at all. From there the testing structure takes its place, ranked like everything else here by how much each rule protects, and the post-leak hook-fixing rule closes the containment thread, exactly as promised above.



The leak lifecycle: every solid arrow is reversible except the last. The dashed paths show why deletion is theater — unreachable objects linger on the host for about ninety days, and clones keep the secret forever.

Rule 62: Hooks Before the First Commit

Secret-scanning hooks are mandatory in every repo, installed before the first commit, not after: a pre-commit hook (gitleaks + detect-secrets) and a pre-push hook that rescans and runs the tests. `.gitignore` covers `.env*`, keys, certs, and credential files from day one.

The ordering clause is the whole rule. Everyone agrees secret scanning is a good idea; almost everyone installs it on day four, after the repo "settles down." But the first commits of a project are precisely the most dangerous ones. Day one is when you're wiring up the API client and the temptation to paste the key inline "just to see it work" is strongest. Day one is when there's no `.env` file yet, no config layer, no reviewer. Day one is when nobody is watching, including you.

So the sequence is fixed: `git init`, install the hooks and the `.gitignore`, *then* write code. In my repos this is a single bootstrap step — the pre-commit config in Appendix A drops in unmodified, gitleaks and detect-secrets included, and takes under a minute. A minute is a price; a rotation is a project.

Two hooks, not one, because the threat model changes between commit and push. The pre-commit hook is the first tripwire: it scans the change you think you're making. A commit is local — embarrassing, but recoverable. A push crosses the trust boundary, and per the lifecycle diagram above, what crosses doesn't come back. So the pre-push hook gets its own scan, even though every commit in it was supposedly scanned already — "supposedly"

is doing heavy lifting in that sentence. Hooks get bypassed in moments of haste; commits arrive from rebases, cherry-picks, and other machines where the hooks weren't installed; an agent may have made commits in a different session under different rules. The pre-commit hook checks the change; the pre-push hook checks the *history* you're actually about to publish. They are different questions, and you want both answered. The tests ride along on push for the same boundary logic — Rule 8 already says green before commit, but the push is the moment your code stops being your problem and starts being the team's, so the suite runs once more against the branch you're publishing, not just the working tree you last looked at. Yes, this makes pushing slower. It's supposed to. The push is the last gate that runs on hardware you control; everything after it is incident response.

Underneath the scanners sits the dumber, sturdier layer: the `.gitignore`. Scanners are pattern-matchers, and pattern-matchers miss things; the ignore file excludes entire categories of files that have no business in version control before any scanner has to be clever about their contents — `.env` and every `.env.*` variant (with `!.env.example` carved back in), `*.pem`, `*.key`, `*.pfx`, `*.crt`, `id_rsa*`, `credentials.json`, `service-account*.json`, the cloud-CLI config directories. Appendix B has the full block; it pastes in as-is, on day one, for the same reason the hooks do. `git add .` is a reflex, and on a young repo it sweeps up whatever's lying around — including the service-account JSON you downloaded ten minutes ago to get the demo working. Ignore by category, not by incident: if a `deploy.key` slipped past once, the fix is `*.key`, not `deploy.key`, because the next leak always has a different filename. And treat the ignore file as a backstop, not a permission slip — a gitignored `.env` full of production credentials is still one `git add -f` or one screen-share away from trouble.

The rule also binds the AI agents that do an increasing share of my commits. An agent that finds a repo without scan hooks doesn't shrug and proceed on the grounds that "the repo doesn't

have hooks yet" — it installs them before its first commit, the same as I would. Hooks are infrastructure the way a `.git` directory is infrastructure: their absence isn't a style choice, it's a repo that isn't finished being created. One pragmatic note: a hook you can casually bypass trains you to bypass it. `--no-verify` exists for genuine emergencies, and if you find yourself reaching for it twice in a month, the problem is your baseline file or your patterns — fix the configuration, don't develop the habit.

Rule 63: Rotate First, Clean History Second

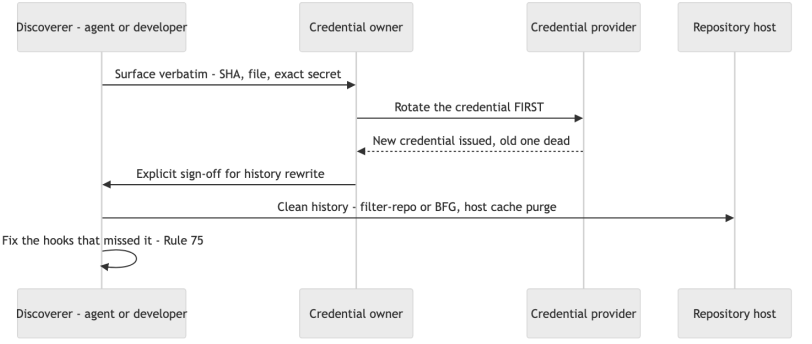
A secret that ever touched a commit is burned. Rotate first, clean history second — pushed objects outlive deletion.

This is the rule people most want to negotiate with, so let me close the exits. "It was only pushed for a minute." Scrapers watch public push events in real time; a minute is plenty. "It's a private repo." Private means a smaller audience, not a trustworthy one — and repos change visibility, get forked, get cloned to laptops that get stolen. "I force-pushed over it." You rewrote the refs, not the objects; on the major hosts, the unreachable objects remain fetchable by hash for around ninety days, and any clone or fork made before your cleanup keeps the secret with no expiration date at all. There is no sequence of git commands that un-publishes data. None.

Once you accept that, the ordering becomes obvious. Rotation is the only step that actually revokes access, so it goes first — before the history rewrite, before the post-mortem, before lunch. A burned key with a pretty history is still burned; a rotated key with an ugly history is harmless. Teams that clean history first are polishing the crime scene while the suspect drives away.

The full protocol: stop what you're doing. Surface the finding to the credential's owner verbatim — the SHA, the file, the secret itself, unredacted, because the owner needs to identify exactly which credential to kill (this disclosure to the owner is the one exception to Rule 64′s no-copying rule, and it goes to the owner only). Rotate.

Then, with explicit sign-off — history rewrites are destructive operations under Rule 5 — clean up with `git filter-repo` or BFG, and ask the host to purge its caches. Then proceed to Rule 75, because the incident isn't over until the hooks are smarter.



The rotation protocol. The order is the point: nothing about history cleanup is urgent once the key is dead, and nothing about it helps while the key is alive.

Rule 64: Never Copy a Secret Anywhere

Never copy a discovered secret anywhere — not into chat, not a scratch file, not "temporarily."

The instinct, on finding a secret, is to grab it: paste it into the chat to ask "is this real?", drop it in a scratch file to deal with after lunch, quote it in the bug ticket, echo it to the terminal mid-debug. Every one of those copies feels free and is not. The chat transcript is stored on someone else's infrastructure and may feed a training pipeline. The scratch file outlives lunch, falls outside the `.gitignore`, and gets swept into the next `git add .`. The ticket is readable by everyone with project access, forever. The terminal echo lands in shell history and the session log. Each copy is a new leak site with its own retention policy, its own audience, and no scanner watching it — and ten rules of containment go down the drain at the speed of Ctrl-V.

This rule binds AI agents hardest, because copying text is what we're *built* to do. An agent that finds a key and helpfully reproduces it in its summary — "I found `sk-live-...` in config.yaml, you should rotate it" — has just written the secret into a conversation log it doesn't control. The correct report names the file, the line, and the shape: "config.yaml line 41 contains what appears to be a live provider API key." Location, not payload.

The one exception was stated in Rule 63: surfacing the exact secret verbatim *to its owner* during the rotation protocol, because the owner must identify precisely which credential to kill. That disclosure goes to the owner, through the most direct channel available, once. Everywhere else, the discipline I learned around radioactive key material applies unchanged: you handle it with tongs, you log that it exists, and you never, ever take it home in your pocket.

Rule 65: Scan every boundary before you cross it

Scan at every trust boundary before you cross it: the staged diff of every config-shaped file, the entire push range (not just the tip), any file you didn't author, and the full deploy artifact — image context, manifests, env bindings. Leaks hide best in "harmless" config nobody re-reads.

The hooks from Rule 62 are the automated floor. This rule is the discipline that rides on top of them, because a secret can slip across any of four boundaries, and each one fails differently. Walk them in order.

The staged diff, before commit. The leak in my opening story didn't arrive in a file called `secrets.txt`. They almost never do. A file with a scary name gets scrutiny; a deployment YAML getting its third routine touch this week gets a skim. That's the camouflage: config files change constantly, their diffs are repetitive and dull, and a literal token sits comfortably among the ports and timeouts because syntactically it *is* just another string value. The most dangerous line in any diff is the one your eyes were trained by a hundred boring

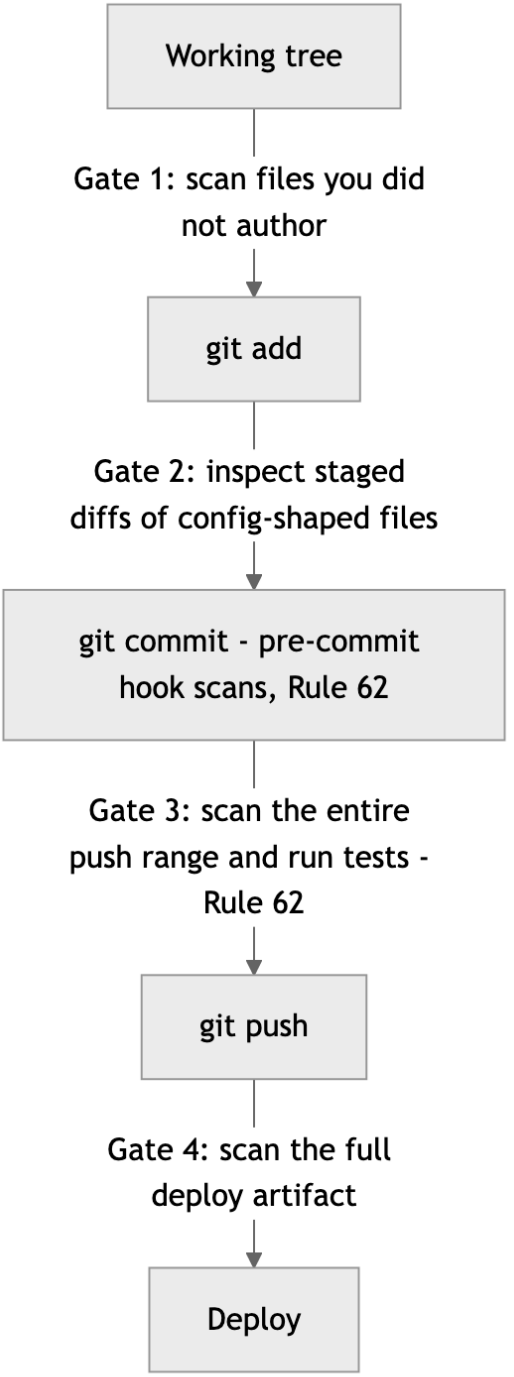
diffs to slide past. So config-shaped files get a mandatory close read at commit time — `git diff --cached --stat` for the overview, then the full staged diff of anything matching `.env*`, `*.yaml`, `*.yml`, `*.json`, `*.toml`, anything with `secret` or `credential` in the name, and anything under `infra/`, `deploy/`, `k8s/`, `terraform/`, `helm/`. Not the file — the *diff*, line by line, with one question: is any value here a literal credential instead of a reference to one? A healthy config diff names secrets — `secretKeyRef`, `${API_TOKEN}`, a vault path. The moment it contains the *value* of one, you've found the leak before it happened.

The push range, before push. A push doesn't publish a snapshot; it publishes *history*. Commit three adds a key for local debugging; commit five removes it because you noticed. The working tree is clean, the tip is clean, and a scan of `HEAD` waves the push through — but all five commits cross the boundary, and commit three crosses with the key fully retrievable by anyone who can run `git log -p`. "It was only there for two commits" is not a mitigation; it's a description of exactly where automated scrapers look first, because developers clean up tips and forget middles. So the pre-push scan runs over the full range — everything in `<upstream>..HEAD`, every commit, every version of every file. Gitleaks does this natively against a commit range. When it fires on an intermediate commit the push stops, full stop — and read Rule 63 before you reach for a rebase to scrub it: if the range was ever pushed anywhere before, even to a private fork, even briefly, the key is already burned and rotation comes first.

The unauthored file, before `git add`. Files you wrote are covered by Rule 3 — you never typed a secret in, so there's nothing to find. Files you *didn't* write are a different species: the vendor's sample config, the dataset a colleague handed you, the generated kubeconfig, the repo you're vendoring in, the file an agent produced in a previous session. You don't know their history, so you treat them the way customs treats an unaccompanied bag. The good news is that secrets are conspicuous once you look — provider key prefixes

(`sk-`, `gsk_`, `ghp_`, `AKIA`, `AIza`, the Slack `xox` family), the three-part `eyJ...eyJ...` silhouette of a JWT, `-----BEGIN` private-key blocks, suspiciously long high-entropy base64 runs, plausible passwords in the `data`/`stringData` block of a Kubernetes secret manifest. This gate sits *before* `git add`, not before commit, because hooks check patterns while a reader checks *context*: a scanner sees a base64 string; a human sees it labeled `prod_db_password` in `values-staging.yaml` and asks the obviously necessary question.

The full deploy artifact, before deploy. Deploys have the biggest blast radius, because the artifact is assembled from more than the repo: the container image and everything `COPY` swept into it, the rendered manifests with values merged in, the environment bindings the platform injects, the chart defaults nobody has read since the chart was vendored, the `.env` mounted at runtime. Any one can carry a credential no git diff will ever show you, because it was never in git — or worse, was in git all along, before your scanners existed. So the scope is the *full artifact*, not the delta. "We only changed two files since the last deploy, scan those" silently assumes the last deploy was clean, and the one before it, all the way back to a first deploy that predates your hooks. Layered images and rendered configs inherit history the way commits do. Scan the image build context before the build, scan the rendered manifests after templating (the template is innocent; the *rendered* values are where literals appear), and check that the credential substrate the deploy reads from — the mounted env file, the referenced platform secret — isn't *also* embedded literally in some version-controlled file. Rule 41 already says pre-deploy gates are never disabled by default; this names the gate that matters most. "It's just a quick demo" deploys to someone else's machine all the same.



The four scan gates, in

order. Each gate assumes the previous ones failed — that redundancy is the design, not an inefficiency. Trust no prior commit blind; scan what you're about to cross.

The heaviest containment calls are now made — prevention, rotation, non-propagation, and the four-boundary scan. The post-leak rule that hardens the gates after a miss waits at the end of the chapter, where it belongs. What comes first is the second proof: establishing that what ships actually works, by inspection rather than expectation. It opens with the bar that proof answers to — the grade — and the maxim is the same one that drove the scan gates, you get what you inspect, now pointed at the software itself.

Rule 66: You get what you inspect — and it gets a grade

You get what you inspect: test-driven, not test-after — and graded. Every project keeps a rubric scoring how good the software actually is. 90% is the working bar, polish to 95%, nothing publishes below 95%.

Coverage proves the code does what its tests say. It is structurally incapable of telling you whether the software is any good — whether the CLI's errors help anyone, whether the latency is tolerable, whether the docs let a stranger start in five minutes. A fully covered product can still be a bad product. Different question, different instrument.

The instrument is a rubric: a written, project-specific scorecard of what "good" means for *this* software — correctness, robustness, performance, usability, documentation, whatever the project actually owes its users — with weights and a score. Writing it forces the conversation teams otherwise have implicitly and too late, usually in the form of an argument about whether the thing is "done." The rubric is the definition of done, decided in advance, in writing, where deadline pressure can't renegotiate it.

The grades have teeth. Ninety percent is the working bar — solid A– software, dependable, honest about its limits. Below 90, it isn't done; keep working. From 90 you run a polish pass — the error messages, the rough edges, the README — to 95. Nothing publishes below 95.

The rubric gauge: 90 means it works; 95 means you'd put your name on it.

Why 95 and not 100? Because the last five points cost more than the next project's first ninety, and chasing them is how perfectionists ship nothing. A– working, A at the door. Inspected, graded, and published with the grade on record.

Rule 67: No test, no ship

New logic ships with tests; bug fixes ship with a regression test written failing-first, before the fix.

Two clauses, and the second one is where the discipline lives. Anyone can agree that new code needs tests. The failing-first regression test is the part people skip, and it's the part that pays.

Here's the failure mode. A bug comes in. You read the code, spot the off-by-one, fix it, and *then* write a test to memorialize the fix. The test passes. Wonderful. But you never saw it fail — so you have no evidence it detects the bug at all. I have watched a team carry a "regression test" for two years that asserted the behavior of a code path the bug never touched. The bug came back. The test stayed green. Both facts were discovered in the same incident review, which is the most expensive way to learn anything.

The order is the whole rule: reproduce the bug as a test, run it, watch it fail, *then* fix the code, then watch the test flip to green. That red-to-green transition is the only proof you'll ever have that

the test and the bug are actually connected. Skip it and you're filing paperwork, not building a regression net.

For new logic, "ships with tests" means in the same commit — not a follow-up ticket, not "once things settle down." Things never settle down. A commit that adds behavior without adding the inspection of that behavior is half a commit, and the missing half is the half that protects you next quarter when somebody — possibly an AI agent, possibly you — refactors the module without remembering what it promised.

Rule 68: Contract first, code second

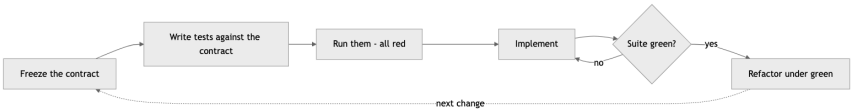
Define and freeze the API or interface, write the tests against the contract, then implement. Never the reverse.

When you write the implementation first and the tests after, the tests inevitably describe what the code *does*, not what it *should do*. They pin the accidents along with the intent — that quirky null return, that undocumented ordering. Now the bugs have test coverage protecting them. I've inherited suites like that: hundreds of green checks, every one of them an affidavit swearing the mistakes were on purpose.

So invert it. First, decide what the thing promises — the function signatures, the endpoint shapes, the error behavior — and freeze that contract. Second, write the tests against the contract while the implementation doesn't exist yet, which guarantees the tests can't peek at internals, because there are no internals. Third, implement until the suite goes green. The contract is the specification, the tests are the specification made executable, and the implementation is merely the first thing that satisfies it.

This matters double when an AI is writing the implementation. An agent given a frozen contract and a failing test suite has a precise, machine-checkable target; it will iterate against the suite until everything passes. An agent told "build a user service, then

add some tests" will happily build something and then certify its own guesses. Test-after with an AI isn't just weaker — it's circular.



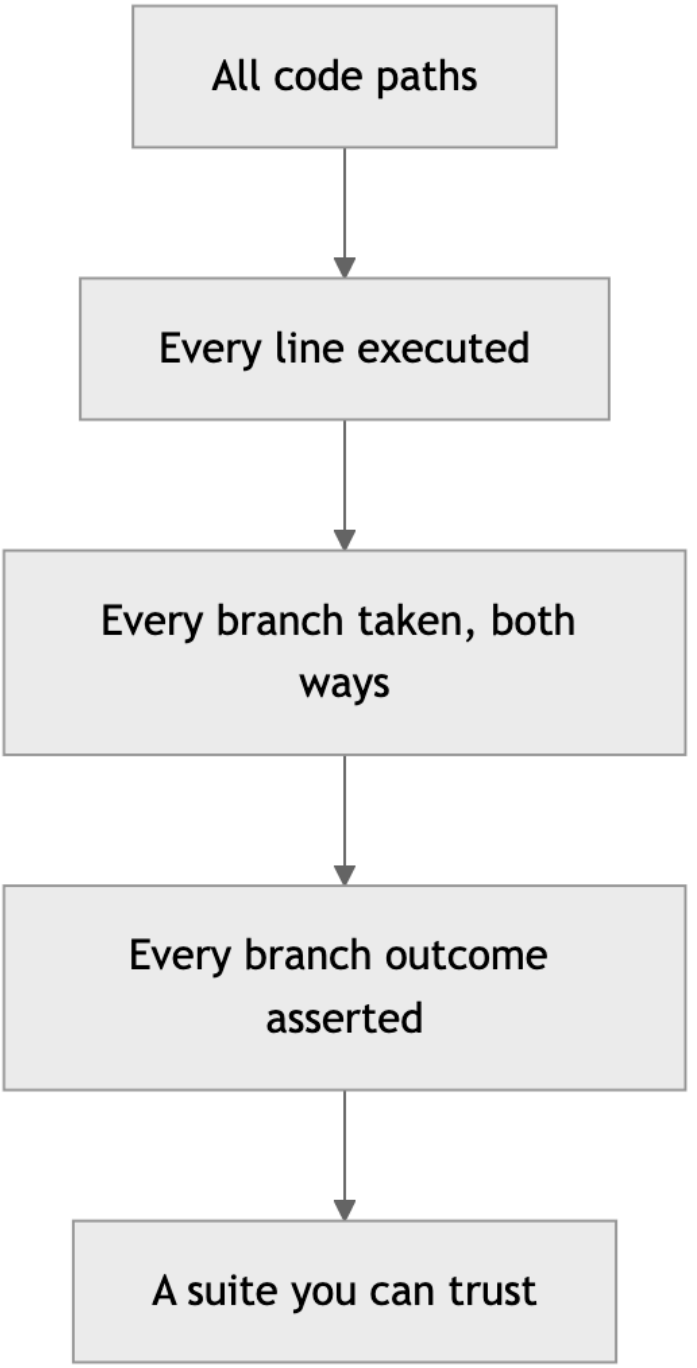
The contract-first loop: the specification exists and is executable before the first line of implementation does.

Rule 69: 100% — lines and branches

100% line and branch coverage — every branch exercised and asserted. Yes, 100%; configure the runner to fail under it.

Every time I say this out loud, somebody quotes me the conventional wisdom: 100% coverage has diminishing returns, 80% is the pragmatic sweet spot, chasing the last fifth is vanity. That wisdom was correct — when humans wrote the tests. The last 20% of branches are the tedious ones: the error handlers, the empty-input guards, the else-arms of defensive checks. Tedium is precisely the cost that no longer applies. The machine writes those tests without complaint, so the old cost-benefit math is obsolete, and I've updated my answer accordingly.

Note the rule says *branch* coverage, and then it says more than that: exercised *and asserted*. Line coverage is the weakest signal there is — a line "covered" by a test that asserts nothing has merely been executed, not inspected. Branch coverage is stronger: both arms of every `if`, every loop's zero-iteration case, every early return. But even an exercised branch proves nothing unless the test asserts what happened on that path. The funnel narrows from "it ran" to "it ran both ways" to "we checked the result both ways," and only the bottom of the funnel is inspection.


All code paths
Every line executed
Every branch taken, both ways
Every branch outcome asserted
A suite you can trust

The coverage funnel: each stage is necessary; only the last one is sufficient.

Make the threshold mechanical. The runner fails under 100% — `--cov-branch --cov-fail-under=100` or your stack's equivalent — so the bar is enforced by tooling, not by whoever feels strict that week. A threshold that requires a human to defend it will eventually meet a deadline that outranks the human.

Rule 70: Correctness over speed

The delay to reach full coverage and verified behavior is acceptable and expected.

This rule exists because every other rule in this chapter will, at some point, be standing between you and a deadline, and somebody will propose the obvious trade: ship now, test later. This rule is the pre-written answer, agreed to in calm weather so it doesn't have to be litigated in a storm: no. The delay is not a regrettable cost overrun. It is the budgeted price of knowing the thing works, and it was approved when the project started.

I spent years in environments where the software shipped inside hardware — burned into devices that went places no patch could follow. Nobody in that world asked whether verification was worth the schedule slip, because everyone could picture the alternative: a recall, a field failure, a very quiet meeting. The web era taught a generation that shipping broken is fine because patching is cheap. Patching is cheap. Burned trust, corrupted data, and 2 a.m. incident bridges are not, and those are what "ship now, test later" actually purchases.

The AI twist cuts both ways here, and it's worth being honest about it. Agents have made the *delay* smaller — full coverage costs hours now, not weeks, which makes the trade easier to refuse. But agents have also made it cheaper than ever to generate plausible, confident, untested code at volume. Speed of production without speed of verification just means you can now be wrong at scale.

The machine's patience for writing tests is only an advantage if you spend it.

So when the estimate includes the testing time and someone asks if there's a faster version: there is, and we're not shipping it.

Rule 71: Coverage never goes down

Coverage going down is a stop-and-fix, not a "justify it."

Most coverage policies I've seen have a ratchet with a release lever: coverage shouldn't drop, but if it does, write a paragraph explaining why and carry on. I've watched that lever get pulled so many times the paragraph became a template. "Coverage temporarily reduced pending follow-up" — there's a phrase that has outlived several of the companies it was written at.

The problem with "justify it" is that justification is a negotiation, and negotiations get won by whoever has the deadline. Stop-and-fix is not a negotiation. The number went down; the work stops until it goes back up. That's the entire policy, and its strength is that it has no second sentence.

Mechanically, a coverage drop means one of two things happened. Either new branches arrived without tests — which is a Rule 67 violation wearing a trench coat — or a refactor orphaned some tests and the paths they used to inspect are now dark. Both are defects in the change that's in front of you right now, while the context is loaded in your head and the diff is small. Fixing them today costs minutes. Fixing them in six months costs an archaeology project: which commit dropped it, what was that branch for, does anyone remember what this error path was supposed to do?

If you hold the line at 100% (Rule 69), this rule enforces itself — the runner simply fails. Rule 71 exists for the transition period, the legacy repo you're ratcheting upward, the project you inherited at 60%. Whatever today's number is, it is the floor. The ratchet only turns one way, and there is no lever.

Rule 72: Full regression, every feature, with the numbers

Run the full regression suite after every feature; report the test count and any failures.

The first half is mechanical: after every feature lands, the entire suite runs. Not the tests near the change — all of them. The whole point of a regression suite is catching the breakage you didn't predict, and selecting "relevant" tests by hand is predicting it. When suites are fast — and offline-by-construction suites (Rule 74) are fast — running everything is cheap enough that selectivity is pure risk with no payoff.

The second half is the part people leave out, and it's the part I actually insist on: *report the count.* Not "tests pass" — "412 tests, 0 failures." The habit looks like bureaucracy until the day it catches something. "Tests pass" and "tests pass — all 9 of them, because the collector silently skipped the other 400 after an import error" produce identical green checkmarks. I've seen a misconfigured runner report success on a fraction of the suite for weeks; everyone read green and moved on. A human who'd been typing "412 tests" all month would have noticed "9 tests" instantly. The count is a heartbeat: cheap to take, and its absence is diagnostic.

This rule matters more with AI agents in the loop, not less. An agent reporting on its own work has every incentive — structural, not malicious — to summarize happily. Requiring the number forces the report to carry evidence instead of vibes, and it gives the human a one-glance sanity check that the inspection actually happened at the scale claimed. "All green" is an opinion. "All 412 green" is a measurement. We ship on measurements.

Rule 73: Set a latency budget and gate it

Declare a latency and throughput budget, then gate regressions against it the way you gate coverage. Determinism and latency are features: set the ceiling explicitly and fail the

build when a change blows past it — a silent slowdown is a defect that ships.

Functional tests answer "is it correct?" They are silent on "is it fast enough?", and that silence is where performance rots — never in one catastrophic commit, but in a hundred small ones, each adding three milliseconds nobody measured. Six months later the endpoint that returned in 40ms returns in 400, every step of the decay was green, and the bug report that finally surfaces it is a user saying the app "feels slow." There is no failing test to point at, because nobody ever wrote down what fast meant.

I spent years building systems where latency wasn't a nice-to-have — a network encryptor that added measurable delay to every packet, an RTOS where a missed deadline was a defect by definition, not a degradation. In that world you stated the budget up front — this path completes in under N microseconds, this throughput holds at M packets per second — and a build that violated it failed, exactly like a build that returned the wrong answer. The discipline transfers directly: pick the paths that matter, write the number down, and put a gate on it. A benchmark in CI that asserts p99 stays under the ceiling. A throughput check that fails the build when it drops below the floor. The number is a contract, the same as a coverage threshold, and it is enforced by tooling rather than by whoever remembers to profile.

The AI era makes this sharper, not softer. An agent optimizing for "tests pass" will cheerfully add an N+1 query, an unbounded retry, or a synchronous call in a hot loop — every one of them green, every one of them slower. The agent has no instinct for latency; the gate is the instinct, externalized. A slowdown nobody asserted against is a slowdown that ships, and the budget is the only thing standing between "correct" and "correct but unusable."

Rule 74: No network in unit tests

No network calls in unit tests — fakes, mocks, and fixtures.

A unit test that touches the network is not a unit test. It is an integration test with a unit test's name tag, and it will betray you on exactly the schedule networks fail: intermittently, unreproducibly, and always during the release build.

The damage compounds in a specific way. The first time the suite fails because some staging endpoint hiccuped, somebody reruns it and it passes. The third time, "rerun the suite" becomes standard advice. By the tenth time, a red build means nothing — maybe a bug, maybe the Wi-Fi — and a test suite whose red can't be trusted is worse than no suite, because it still costs maintenance while providing alibis instead of information. Flakiness isn't a nuisance; it's how a suite dies.

The fix is the seam you already built if you've been following Chapter 2: dependencies arrive through interfaces via constructor injection, so the test hands in a fake. The fake returns canned responses in microseconds, simulates the timeout and the 500 and the malformed payload on command — failure modes you could wait weeks for a real endpoint to produce, served deterministically and on demand. Unit tests get fakes; the offline-by-construction suite runs in seconds, anywhere, including on a plane and in a CI runner with no egress.

Real-network testing still happens — in the integration tier, clearly labeled, running on its own cadence, allowed its own failure modes. The rule isn't "never test the network." It's that the fast suite — the one gating every commit under Rule 72 — depends on nothing but the code under test. That suite must be incapable of lying about whose fault a failure is.

Rule 75: Fix the Hook That Missed It

After any leak, document what scan would have caught it and fix the hooks so it can't recur.

The chapter closes where it opened — on containment, because the last word on a leak isn't the rotation, it's the gate. Every leak is two

failures: a secret got loose, and a gate that should have stopped it didn't. Rotation (Rule 63) handles the first. Most teams stop there, which is how the same leak happens twice — same shape, same gap, eighteen months apart, with only the key prefix changed. The second failure deserves the same rigor as the first.

The post-incident question is narrow and answerable: *which gate, configured how, would have caught this?* Walk the gate map from this chapter. Was there no pre-commit hook (Rule 62)? Did the hook exist but lack a pattern for this provider's key format? Was the file gitignore-able by category and not ignored (Rule 62)? Did a stale detect-secrets baseline whitelist the very line that leaked? Did the tip get scanned but not the range (Rule 65)? Was the artifact assembled from something outside the repo that no gate ever saw (Rule 65)? The answer is rarely "no tool could have caught this." It is almost always "the tool was missing, misconfigured, or scoped too narrowly."

Then fix it, and make the fix verifiable: add the missing pattern and a test case with a defanged dummy secret that proves the hook now fires. Write the incident down — date, what leaked, which gate failed, what changed — in the bug log, where the next maintainer will find it. And audit sideways: the habit or session that produced this leak probably touched other repos the same week. Carelessness patterns; check its other haunts.

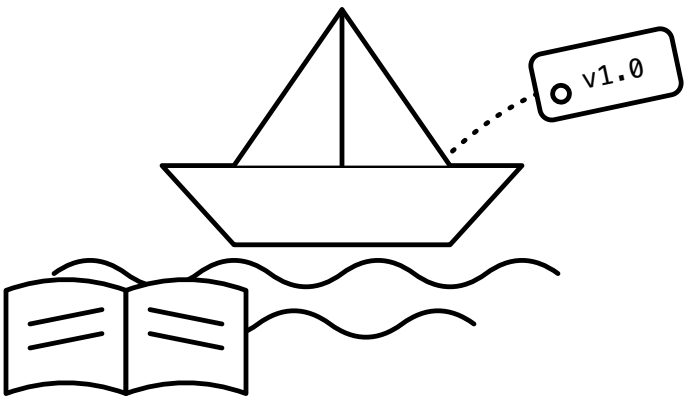
A leak that buys you a permanently better gate was expensive tuition. A leak that buys you nothing is just a rehearsal for the next one.

Chapter 4 card

- **62.** Secret-scanning hooks before the first commit — pre-commit (gitleaks + detect-secrets), pre-push that rescans and runs the tests, and a `.gitignore` covering `.env*`, keys, certs, and credential files from day one.

- **63.** A secret that ever touched a commit is burned: rotate first, clean history second; pushed objects outlive deletion.
- **64.** Never copy a discovered secret anywhere — not chat, not a scratch file, not "temporarily." Report location, not payload.
- **65.** Scan at every boundary before you cross it — staged config diffs, the full push range (not the tip), files you didn't author, and the whole deploy artifact. Leaks hide best in "harmless" config.
- **66.** You get what you inspect: keep a rubric and grade the software — 90 is the working bar, polish to 95, publish at 95+.
- **67.** New logic ships with tests; bug fixes ship with a regression test that failed first.
- **68.** Freeze the contract, write tests against it, then implement — never the reverse.
- **69.** 100% line *and* branch coverage, every branch asserted; the runner fails under it.
- **70.** Correctness over speed — the verification delay is budgeted and expected.
- **71.** Coverage going down is stop-and-fix, not "justify it."
- **72.** Full regression after every feature; report the test count, not just "green."
- **73.** Declare a latency and throughput budget and gate regressions against it — a silent slowdown is a defect that ships.
- **74.** No network in unit tests; fakes, mocks, and fixtures only.
- **75.** After any leak, identify the scan that would have caught it and fix the hooks so it can't recur.

Chapter 5 — Ship and Remember



Standing on Chapters 1–4: - The hard rules never bend, and the Powell rule holds: gather 90% of the information, then decide — below 90%, ask the human (Chapter 1). - Nothing is hardcoded; everything that can change flows through config, and every backend lives behind an interface (Chapter 2). - Builds are portable, fail loudly at startup, and carry the fewest dependencies that do the job (Chapter 3). - Every artifact is scanned clean for secrets and proven against the 90/95 rubric with 100% branch coverage before it goes anywhere (Chapter 4).

Chapter 4 left you with software that's proven — scanned clean, covered to the last branch — but proven software still has to be shipped honestly, and that's a different discipline. A version number is a promise. The moment you publish `v1.4.2`, somebody — a container build, a deploy script, a downstream team, your own future self at 2 a.m. — pins to it. From that moment forward, `v1.4.2` means one thing: a specific set of bytes that behaves a specific way. Break that promise once and every consumer of your software learns the same lesson: your version numbers are decorative. They will start pinning to commit SHAs, vendoring your code, or worse, freezing on an old release forever because upgrading you is a gamble.

And one act in this chapter can never be taken back. Almost every other failure in this book is recoverable — a bad commit reverts, a stale README regenerates, a missed sweep runs late. A pushed tag that gets moved is different. Every container build, every deploy manifest, every bug report, every audit record that pinned that tag is silently wrong from that moment on, and nothing notifies any of them; the name keeps working while its meaning changes underneath. That is why tag immutability leads the chapter — rule 76 is the one irreversible act in it — with the rest of the version-integrity rules close behind.

I spent the formative years of my career in embedded systems, where the question "what exactly is running on that box?" had to have an exact answer. Not "probably the March build." An exact answer, down to the build, because when a unit misbehaved in the field, the firmware image was the first variable to eliminate — and you can't eliminate what you can't identify. That discipline never left me. The deployment substrate changed; the question didn't. So the version-integrity rules run right behind the tag: the version only moves forward, lives in exactly one place, every build carries a unique serial number, every artifact answers "what are you?" at the front door, and the changelog travels in the same commit as the bump so the version and its story can never drift apart.

But a promise is only as good as your ability to remember why you made it. Every piece of unwritten knowledge has a half-life, and it is shorter than you think. The reason we chose the message queue over the database trigger; the gotcha in the vendor's auth flow that cost us a week — if it lives only in a chat scroll or somebody's head, it is already decaying. I have watched teams re-litigate the same architectural decision three times in two years, each round burning a week, because nobody wrote down the verdict the first time. The code remembered what was decided. Nothing remembered why.

For most of my career that was a chronic, low-grade tax. Then the AI teammates showed up and turned a chronic condition into an acute one. An AI coding agent forgets everything between sessions. Everything. It starts every morning as the smartest amnesiac you've ever hired. If your project's memory lives in heads and scrollback, your AI teammate's effective memory is zero — and it will cheerfully undo last month's decisions because nothing on disk says otherwise. Written memory used to be a nicety. Now it is the load-bearing wall. The ledgers in this chapter — changelogs, ADRs, bug and feature files, decisions persisted in the same commit that ships the work — are not documentation in the old "write it for the auditors" sense. They are the persistent state of a system whose most productive workers are stateless. Which is why the

memory rules rank just behind versioning integrity here, shoulder to shoulder with the tags and the bumps, instead of trailing at the back the way documentation always has.

Shipping is the promise; memory is what keeps the promise keepable; and the hygiene sweeps close the chapter — and the book — by keeping both legible. Seventeen rules, and then you're on your own.

Rule 76: Tags are carved in stone

Tags are immutable: never move, delete, reuse, or force-overwrite a pushed tag. Fetch the remote's tags and compare before creating one — local is not the truth — and push tags by name, never `--tags` reflexively.

A pushed tag is a published fact: "`v1.4.2` is commit `a3f9c01`." People build on published facts. A container image was built from that tag. A deploy manifest pins it. A colleague's bug report says "reproduced on v1.4.2." An auditor's record says that's what was certified. Every one of those references assumes the tag still points where it pointed when they looked.

Move the tag — delete it and recreate it on a different commit — and every one of those references silently rots. The container built last month and the container built today are different software wearing the same name badge. The bug report now describes code that "v1.4.2" no longer contains. Nobody gets an error. Nobody gets a notification. The references just quietly stop meaning what their authors meant, and you find out during an incident, which is the most expensive possible time to find out.

This is why git itself makes you force-push to move a tag. That `--force` flag is the version-control equivalent of a lockwired bolt: the friction is the point. If you tagged the wrong commit, the fix is the same as rule 77: a *new* tag. `v1.4.2` was wrong? Ship `v1.4.3`. The bad tag stays in history as a record that the mistake happened — which is itself valuable information.

Immutability has two guards on either side of it, and both are about not acting on a stale view of shared state. **Fetch before you tag.** Your local clone is a cache, and like every cache it goes stale: another contributor tagged yesterday, a CI job auto-tagged this morning, an agent — and there may be several working the same repo — tagged twenty minutes ago, and none of it is in your clone until you ask:

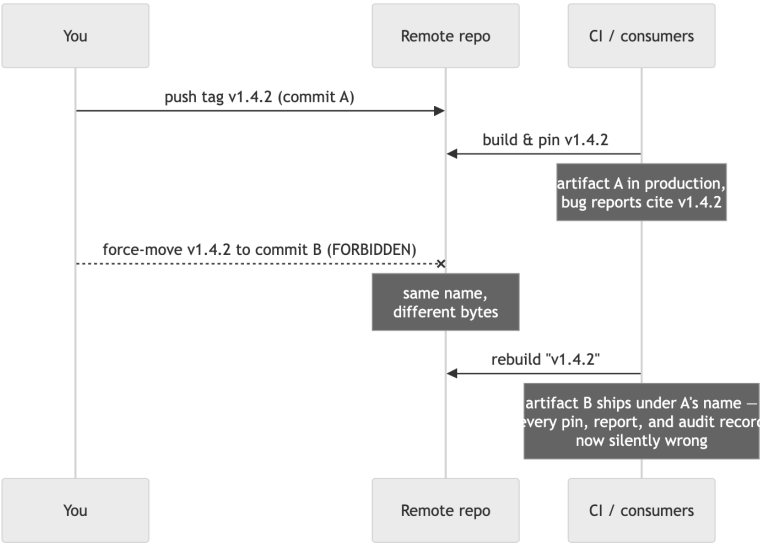
```
git fetch --tags origin
git ls-remote --tags origin
```

Then confirm the tag you're about to cut is strictly greater, under SemVer ordering, than the latest in the *union* of local and remote — not just absent locally, absent and greater remotely. Skip it and you either get a rejected push and ten lost minutes, or the quiet failure: you cut `v1.6.0` not knowing the remote already has a `v1.7.0` someone else shipped, your "new release" sorts behind the current one, and every consumer resolving "latest" keeps ignoring you. And if the fetch reveals an existing tag pointing at a different SHA than you expected — stop. Don't "fix" it by retagging. Surface both SHAs and let a human decide, because one of two histories is wrong and you don't know which.

Push by name. `git push --tags` shoves *every* local tag at the remote — the release you intended plus every experimental, fat-fingered, half-baked local marker. Name the ref instead: `git push origin v1.4.2`, exactly that, nothing else along for the ride. This is the cheap insurance that makes immutability hard to violate by accident: if some tool rewrote a local tag so it points somewhere the remote's copy doesn't, a named push leaves it sitting harmless, where `--tags` would attempt to clobber the published tag. A reflexive `--tags` is how tags get moved by someone who would swear, honestly, they'd never move a tag. Make publication intentional: name what you publish and mean it.

In my embedded days we had a phrase for shipped firmware: it's in the field now. You can ship a new image; you cannot reach out

and change the bytes already burned into a thousand devices. Treat pushed tags with the same finality. The tag namespace is append-only. Write carefully, because there's no eraser.



A moved tag fails nobody loudly. Every consumer who pinned the old commit keeps trusting a name that changed meaning underneath them.

Rule 77: Versions only move forward

Bump on every release; versions only move forward. Roll back by rolling forward to a new patch.

Every release gets a new number — even a trivial one, even a "we just rebuilt it" one. If the bytes can differ, the number must differ. Two artifacts with the same version and different contents is the release-engineering equivalent of two parts with the same serial number: now nothing about either one can be trusted.

The second half is where people flinch: you never roll *back* a version. Production is on `1.5.3`, it's on fire, and the instinct is to redeploy `1.5.2` and call it restored. Don't. Cut `1.5.4` — even if its entire content is "revert to the state of 1.5.2" — and deploy that.

Why insist on the ceremony? Because "we're running 1.5.2" becomes a lie the moment you roll back. You're running 1.5.2 *again*, after 1.5.3, with whatever migrations, cache state, config changes, and side effects 1.5.3 left behind. The timeline matters. A version history that reads `1.5.2 → 1.5.3 → 1.5.4 (reverts 1.5.3)` tells the whole story to anyone reading the changelog in six months. A history that reads `1.5.2 → 1.5.3 → 1.5.2` tells them nothing except that someone panicked, and your monitoring, your logs, and your incident timeline now contain two different deployments that both claim to be 1.5.2.

Time moves one direction. Your version numbers are a clock. Clocks that run backward aren't clocks anymore; they're props. Roll forward, always — the revert is a *new event* in the system's history, and it deserves a new number that says so.

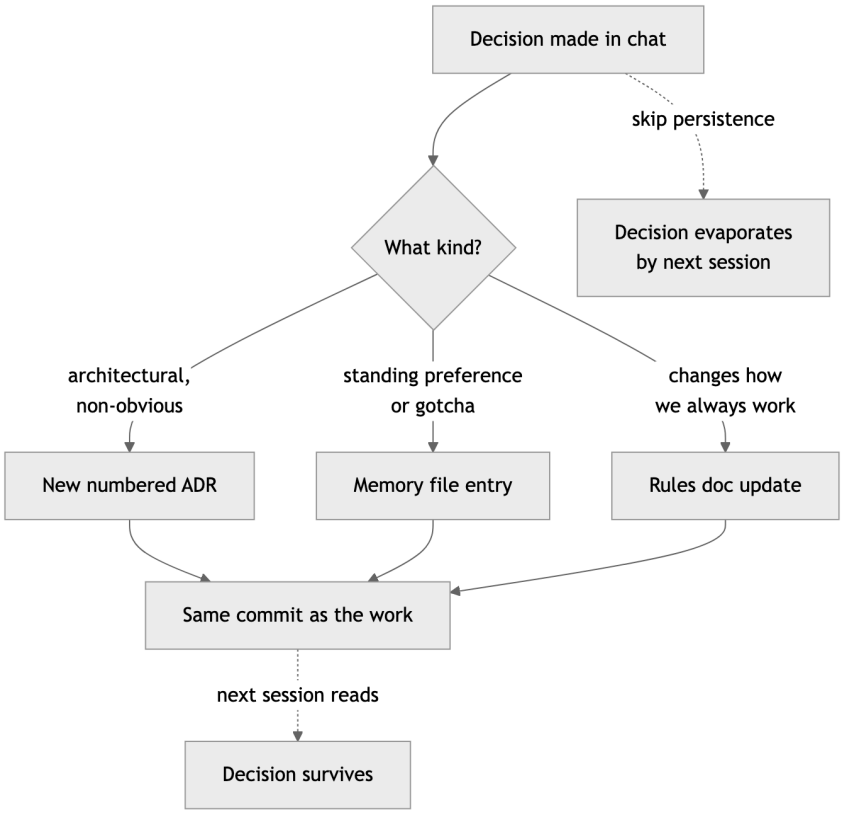
The release pipeline: the bump and the changelog travel in one commit, the tag marks it, and the artifact carries its identity baked in.

Rule 78: Persist decisions in the same commit

When a standing decision is made in chat, persist it the same commit — chat history is not memory. Non-obvious decisions go in numbered ADRs, immutable after acceptance; a later ADR supersedes, never edits.

Chat is where decisions get made now. A question comes up mid-session, you work through the options with the agent, you make the call: "from now on, default to the streaming API; the batch path is legacy." That decision now exists in exactly one place — a conversation buffer that will be gone by morning. The next session starts cold, picks the batch path because nothing says otherwise, and you make the same call again, slightly differently, building a fog of almost-consistent precedents.

The fix is a timing rule, and the timing is the whole rule: *the same commit.* Not "I'll write it up later" — later is where decisions go to die; the moment the conversation moves on, the writing-up has lost its slot. The decision and its persistence travel together: chat produces the verdict, the verdict lands in the right file, and the commit that ships the work carries the memory with it.



The decision flow. The solid path is the rule; the dashed path to evaporation is what happens by default.

Routing is easy: architectural and non-obvious goes to an ADR; standing preferences and gotchas to the memory file; anything that changes how you always work amends the rules doc itself. The test for what needs persisting: *would I want the next session to know this without being told?* If yes, it goes in a file, now, in this commit. With

teammates that forget everything overnight, an unwritten decision isn't a decision. It's a mood.

The ADR path carries one extra discipline, and it's the part people resist: once accepted, the record never changes. An Architecture Decision Record is a small, numbered file with a boring shape — context (what was true when we decided), decision (what we chose), consequences (what it costs), alternatives considered — and a decision record you can edit is a decision record you can't trust. Was this the reasoning at the time, or the reasoning someone retrofitted after the outcome was known? An immutable record is testimony: when ADR-0014 says "we chose the embedded store because we had two ops people and no cluster budget," you know that was true *then*, even if it's false now. When the world changes you don't edit ADR-0014 — you write ADR-0031, state the new context, and mark the old one superseded with a pointer. The chain of supersessions *is* the history of your architecture's reasoning, and it's the most honest document your project owns precisely because nobody can launder it. AI teammates read the ADR titles at session start and stop re-proposing the migration you rejected, with reasons, eighteen months ago — the most common failure of a stateless collaborator is re-deciding settled questions, and a scan of ADR titles is the cheapest vaccine. Keep them short. Keep them numbered. Never touch them after acceptance.

Rule 79: One version, one home

Semantic versioning, with the version in exactly one canonical place — everything else reads from it.

Two parts to this rule, and people usually nod at the first and violate the second.

SemVer first: `MAJOR.MINOR.PATCH`. Breaking change bumps major. New capability bumps minor. Fix bumps patch. This isn't aesthetics — it's a machine-readable contract about compatibility. A consumer pinned to `^1.4` is trusting you that `1.5.0` won't break them. SemVer

is the grammar of the version-number promise; the rest of these rules are the enforcement.

Now the part that actually bites: the version string lives in *exactly one* file. `pyproject.toml`, `package.json`, a `VERSION` file, a single constant in the package root — pick one per project and make everything else read from it at build time. The README badge, the CLI banner, the Dockerfile label, the `/health` payload, the installer metadata: all derived, none hand-maintained.

The failure mode is so common it's almost a rite of passage. The version is in the build manifest, and also in a constant in the source, and also in the docs header, and also in the install script. They agree on day one. Then a release goes out in a hurry, three of the four get bumped, and now your `--version` flag says `2.1.0` while the package metadata says `2.2.0`. Which one is lying? Both, sort of. Neither answers the only question that matters — what is this artifact? — because the artifact now disagrees with itself.

I think of it the way I think of any redundant state in a system: two copies of the same fact is one copy too many, because the copies *will* diverge, and they'll diverge on the worst possible day. One register, one writer, many readers. Everything else is a derivation, generated at build time, never edited by hand.

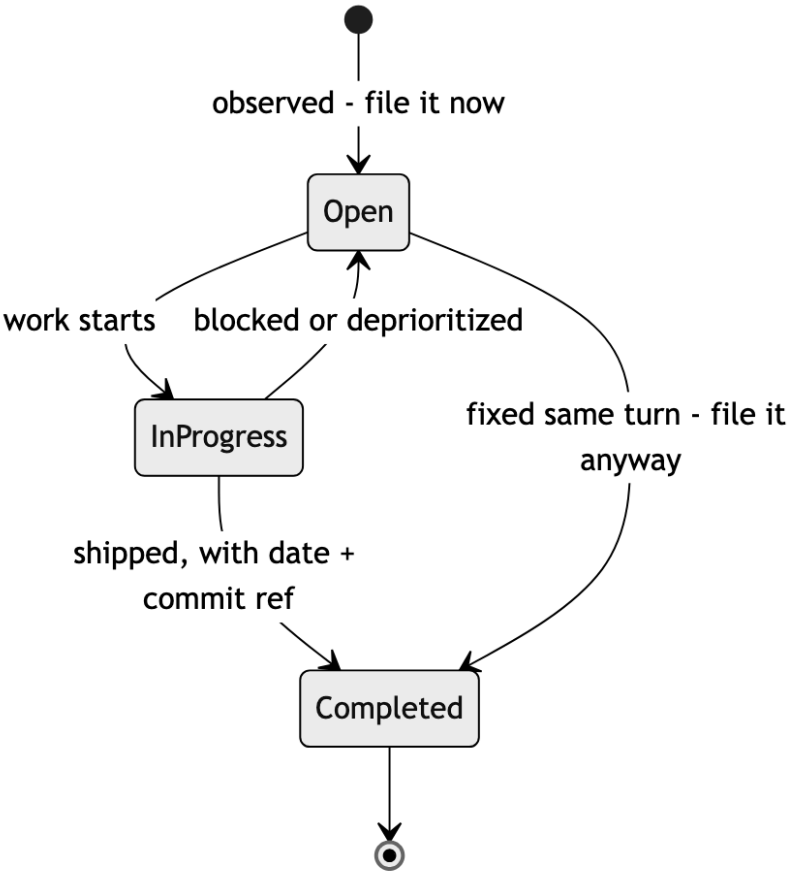
Rule 80: File it the moment you see it

Track every bug and every requested feature in `bugs.md` and `features.md` the moment it's observed — even if it's fixed the same turn. Open questions are filed inline with the entry; filing never blocks on answers.

The cheapest moment to record a bug is the moment you're looking at it. You have the symptom, the context, the reproduction in your head — the entry takes ninety seconds. Every hour after that, the knowledge evaporates: by tomorrow it's "something was weird with the export," by next month a user finds it for you. Same for features: the half-formed "we should let people batch this" dies

in the chat scroll unless it lands in a file before the conversation moves on.

So the rule is mechanical: observed means filed. Even — *especially* — if it's fixed in the same turn, because the pattern across filed-and-fixed entries is how you notice the module that produces a regression every month. And filing never waits for completeness. Unclear scope, missing repro steps, contested approach? File the entry anyway with the questions inline under it; they get answered when the entry is picked up, not before. A tracker that demands fully-specified entries trains people to not file things, which is the failure mode the tracker exists to prevent.



The entry state machine. Note both paths into Completed: even a same-turn fix passes through the file.

Entries update in place — status, resolution date, commit reference — so the file stays a live ledger, not a graveyard. For AI teammates this file is working memory: it's how the bug noticed in Tuesday's session survives to be fixed in Thursday's.

Rule 81: Every build wears a serial number

Every build gets a unique, monotonically increasing build number (`git rev-list --count HEAD` works) — stores reject reused ones.

The version is the *marketing* identity: what changed for the user. The build number is the *manufacturing* identity: which artifact this is, exactly. You need both, because one version can produce many builds — you build `1.4.2`, the upload fails, you fix the signing config and build `1.4.2` again. Same version, different bytes. Without a build number, those two artifacts are indistinguishable, and "indistinguishable artifacts" is a phrase that should make your skin crawl.

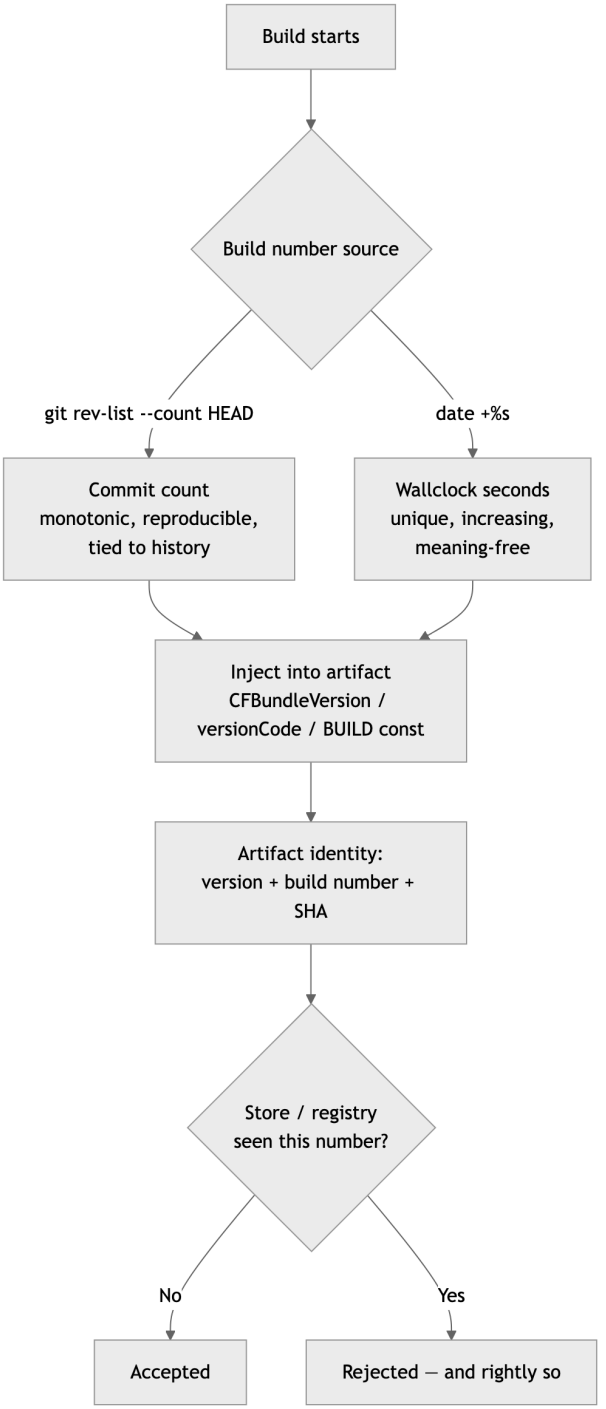
The app stores enforce this at the door: upload an artifact with a build number they've seen before and it bounces, full stop, regardless of what changed. Apple's `CFBundleVersion`, Android's `versionCode` — unique per upload, no exceptions, no appeals. They learned this lesson at planetary scale so you don't have to relearn it locally.

The trap is leaving the bump to a human. Humans forget, exactly once per release cycle, at the most annoying possible moment — usually discovered after the twenty-minute archive-and-upload dance completes and *then* rejects. So don't let a human touch it. Generate the number at build time, deterministically:

- `git rev-list --count HEAD` — the number of commits in history. Monotonic, survives clones, identical on every machine building the same commit. My preference, because it ties the artifact back to a point in history.

- `date +%s` — wallclock seconds. Trivially unique and always increasing, at the cost of meaning nothing.

Either way, bake it into the build script or a pre-build phase so the bump physically cannot be forgotten. A serial number that depends on someone remembering isn't a serial number; it's a suggestion.


Build starts
Build number source
git rev-list --count HEAD
date +%s
Commit count
monotonic, reproducible,
tied to history
Wallclock seconds
unique, increasing,
meaning-free
Inject into artifact
CFBundleVersion /
versionCode / BUILD const
Artifact identity:
version + build number +
SHA
Store / registry
seen this number?
No
Yes
Accepted
Rejected — and rightly so

Build-number generation belongs to the build, not to human memory. Derive it, inject it, and the uniqueness check becomes a formality.

Rule 82: Verbatim errors, diffs not prose, assumptions out loud

Quote errors verbatim — never paraphrase a stack trace. Show diffs, not prose, when the question is "what changed?" Surface assumptions explicitly.

This rule is about the quality of the signal between collaborators, and it has three clauses that are really one principle: *transmit evidence, not interpretation.*

Errors, verbatim. A paraphrased error is interpretation wearing the costume of evidence. "It couldn't connect to the database" might be a refused connection, an auth failure, a DNS miss, a timeout, or a TLS mismatch — five different bugs flattened into one sentence by a helpful summarizer. The raw text carries the error code, the hostname, the line number, the one weird detail that cracks the case. I have lost entire days to debugging the paraphrase instead of the error. Paste the real thing.

Diffs, not prose. When the question is "what changed?", a description is a secondhand account from a witness with an incentive to look good. The diff is the change: prose says "refactored the retry logic"; the diff shows the timeout that also quietly went from 30 to 5. Reviewers approve prose. They catch things in diffs.

Assumptions, out loud. Every nontrivial task runs on unstated assumptions — what the user meant, which environment matters, what's in scope. Stated, an assumption is a checkpoint: "I'm assuming you want this configurable with the local backend as default — confirm?" costs one line and catches the wrong turn before the work, not after. Unstated, it's a landmine with a delay fuse — and the AI's wrong assumptions are the expensive ones, executed at machine speed.

Evidence over interpretation, in every channel. That's the rule the other ninety-nine ride on.

Rule 83: The changelog rides in the bump commit

Maintain a changelog in the same commit as the version bump.

A version number says *that* something changed. The changelog says *what*. Rule 83 is about keeping those two facts physically inseparable: the commit that bumps the version is the commit that updates `CHANGELOG.md`. One commit, atomic, no exceptions.

The reasoning is the same as rule 79's: redundant state diverges unless it's written in one motion. Let the changelog lag "just until after the release," and you've created a window where `v1.5.0` exists but its story doesn't. Windows like that don't close; they accumulate. Three releases later someone is reverse-engineering the changelog from `git log`, guessing which commits mattered, and the document quietly transitions from "record" to "historical fiction." I've watched changelogs die this way more times than I can count, and it's always the same cause of death: the update was a separate step, and separate steps get skipped.

When the changelog and the bump share a commit, the tag from rule 76 pins both at once. Check out `v1.5.0` and the changelog at that ref describes exactly what `v1.5.0` is — guaranteed, mechanically, forever. The release notes write themselves: they're the top section of the file at the tagged commit.

Format-wise, follow Keep a Changelog or something near it: human-written entries grouped under Added, Changed, Fixed, Removed — written for the *user* of the software, not the author. A raw commit-log dump is not a changelog; nobody pinning your package cares that you "refactored the thing, again, properly this time." Tooling like `commitizen` or `release-please` can automate the mechanics of the bump-plus-changelog commit, and automa-

tion is welcome here precisely because it makes the atomicity unforgettable.

Rule 84: Plans tell you their own status

Plans live in a `plans/` directory with a first-line `Status:` kept current — stale status on shipped work is a process violation.

This is the version-number discipline applied to intent instead of artifacts. A plan document is a promise about future work the same way a tag is a record of past work — and like any published fact, it's only useful if it's true *right now*.

The mechanics are deliberately minimal. Plans live in `plans/`. The first line of every plan is its status: `Status: Not Implemented`, `Status: In Progress`, `Status: Implemented, <date>`, or `Status: Partial — Remaining: <items>`. When work starts against a plan, the status changes in the same motion. When work finishes, same thing. The first line is the one place a reader — human or agent — looks, and it never lies. Retired plans move to an `archive/` subdirectory and drop out of the accounting entirely.

Why be strict enough to call stale status a *violation* rather than untidiness? Because a plan whose status lies is worse than no plan. A new contributor reads `Status: Not Implemented` and starts building something that shipped in March — duplicated effort. An agent reads it and re-plans solved work — wasted tokens and a confused session. And this matters double in AI-assisted development: the agents resume from the written state of the repo, not from anyone's recollection of last Tuesday. Documents *are* the project's memory. Memory that's wrong is worse than memory that's absent, because absent memory at least sends you to go look.

The status line is your release ledger for intent: one canonical place, kept current, never trusted stale. Which is, you'll notice, the same rule this chapter has been repeating in different costumes. What's running in production, what a tag points to, what was decided,

what a plan's state is — publish the fact in one place, keep it true, and never make anyone guess.

Rule 85: The version answers the door

Display the version everywhere it matters: splash screen, `--version`, `/health`.

The most common question in any debugging session involving deployed software is some variant of "what version are you running?" — and the answer should never require archaeology. Not "let me check the deploy logs." Not "whatever CI pushed Tuesday." The running artifact itself answers, immediately, through whatever front door it has:

- A CLI answers `--version`.
- A service answers on `/health` or `/version`.
- A GUI shows it on the splash screen or the about box.
- A log stream prints it in the first line at startup.

And the answer is the full identity, not just the marketing number: `v1.4.2 (build 1847, sha a3f9c01, built 2026-06-11)`. Version for the humans, build number for uniqueness, SHA for the precise commit, date for the sanity check. All of it embedded *at build time* — generated into a constant or stamped via the build environment — never read at runtime from some file that might not exist inside the container. An artifact that has to phone home or go spelunking on disk to learn its own name doesn't reliably know its own name.

Why so insistent? Because half of all "impossible" bug reports dissolve the moment you can see what's actually running. The fix that "didn't work" was never deployed. The two environments behaving differently are two different builds. Each is a five-second diagnosis *if* the artifact self-identifies, and a half-day goose chase if it doesn't.

Field rule from the embedded years: a unit that can't report its own firmware revision goes back on the bench until it can. Same rule here. Identification before diagnosis — always.

That's the release machinery and its ledgers: every artifact identified, every tag immutable, every decision and plan telling the truth about itself in writing. What remains is keeping the workshop clean enough that all of it stays legible — to the next human reader and to the amnesiac machine that ingests the whole repo as its working memory. The last seven rules are the closing sweeps: the recurring hygiene that keeps the codebase readable, the trackers honest, and everyone — human or agent — accountable for what they install and what they leave behind.

Rule 86: Lint and format every commit

Lint and format on every commit; CI stays green.

Formatting arguments are the most expensive zero-stakes arguments in software. Nobody's product ever shipped late because the braces were on the wrong line, but plenty of teams have burned real hours relitigating style in review — and plenty of diffs have hidden real bugs inside a blizzard of whitespace churn. The fix has been settled for years: pick a formatter, wire it into a pre-commit hook, and never discuss style with a human again. The formatter is not the best style; it's the *agreed* style, which is worth more.

Linting is the same bargain at one level up. The linter catches the unused variable, the shadowed name, the suspicious comparison — the whole class of bug that's trivial when caught at commit time and embarrassing when caught in production. Running it on every commit means findings arrive in batches of one or two, attached to fresh context. Running it "before release" means a four-hundred-finding cleanup that everyone defers.

The second clause is the one with teeth: **CI stays green.** Green-before-commit was Rule 8′s hard line; this rule extends it to the shared pipeline. A red main branch is a broken contract with everyone downstream, and tolerance for red is cumulative: a "flaky" test gets shrugged at on Monday, masks a real regression on Wednesday,

and by Friday nobody can say which of the nine failures matters. Red means stop. Fix it or revert it; never wave at it.

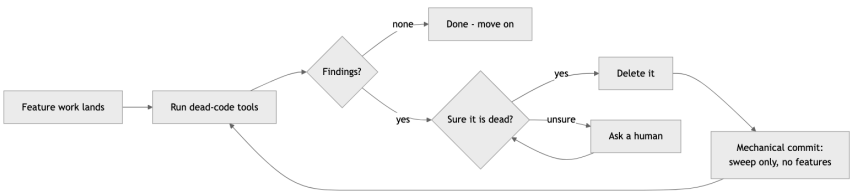
For AI agents this rule is load-bearing in both directions. The hooks catch the agent's style drift mechanically, and a green CI signal is the one verification an agent can't talk itself out of.

Rule 87: Sweep the dead code

After meaningful feature work, run a dead-code pass — vulture and ruff for Python, ts-prune and knip for TypeScript, staticcheck for Go. Remove unused imports, parameters, branches, and files. Ask before deleting anything you're unsure about.

Feature work sheds debris. You refactor a function and the old helper goes quiet; you swap a backend and the previous adapter sits unused; you delete a call site and three imports go stale. None of it breaks anything, which is exactly the problem: dead code doesn't announce itself. It just sits there, getting read by every future maintainer — and ingested into the context window of every future AI session — as if it mattered. With AI teammates it literally costs tokens: the agent reads the corpse, reasons about the corpse, and sometimes resurrects the corpse.

So the sweep is scheduled, not aspirational. After meaningful feature work — not every commit, but every time the dust settles on something real — run the tools. They're fast, they're free, and they find what eyes skip.



The dead-code sweep cycle: tools find candidates, certainty gates deletion, the sweep lands as its own mechanical commit.

Two qualifiers carry the weight. First: *ask before deleting anything you're unsure about.* Static analysis can't see reflection, dynamic dispatch, plugin discovery, or the entry point that only the cron job calls. "Looks dead" and "is dead" are different claims, and the difference is a production incident. Second, per Rule 9: the sweep is its own commit, mechanical and reviewable, never smuggled into a feature. A pure-deletion diff is the easiest review in the world. A deletion folded into a feature is where mistakes hide.

Rule 88: After the release, take out the trash

After a stable release, the first task is a cleanup sweep — before any new feature.

The release just shipped, everyone's energy is pointed at the next shiny feature, and this rule plants itself in the doorway: not yet. First, the sweep. Dead code, unused imports, orphaned files, stale TODO entries, experiments that didn't make the cut, docs that describe the previous architecture — all of it gets hunted down and removed before any new feature work begins.

There are two reasons this lives *here*, attached to the release cadence, instead of floating as a vague "keep things tidy" aspiration.

First, a stable release is the one moment cleanup is genuinely safe and honest. The suite is green, the behavior is pinned by a tag, production is calm. Anything that survives the sweep visibly earns its keep against a known-good baseline; anything deleted can be checked against that same baseline. Mid-feature cleanup, by contrast, mixes "I deleted dead code" with "I changed behavior" in the same diff — and per the one-purpose-per-commit hard rule, that's exactly what we don't do.

Second, scheduled cleanup is the only cleanup that happens. "We'll tidy up when things slow down" is a sentence that has preceded more rotted codebases than any architectural mistake I've seen, because things never slow down — features arrive faster than discipline. Binding the sweep to the release makes it a recurring

appointment instead of an aspiration. Run the dead-code tools (`vulture`, `ruff`, `ts-prune`, `knip`, `staticcheck` — whatever fits the stack), surface the findings as a reviewable list before deleting, and land the deletions as their own mechanical commits.

A release is a finish line. Cross it, take a breath, sweep the shop floor. *Then* build the next thing — on a clean floor.

Rule 89: Regenerate the README

After every major change, regenerate the README from scratch rather than patching it. It answers, in order: what this is and who it's for, quick start, configuration table, how to test, architecture, deployment, troubleshooting.

READMEs don't fail by being wrong all at once. They fail by being patched — a sentence appended here, a flag renamed there, a quick-start step that survived the refactor it described. Each patch is locally true and globally corrosive, and after a year the document is an archaeological dig: four eras of the project visible in its strata, with no indication which layer is current. A README the reader can't trust is worse than no README, because it costs them the failed attempt before they fall back to reading the code.

Hence: don't patch. After a major change — new subsystem, dependency swap, deployment change, breaking interface — regenerate from scratch. Stale claims don't survive regeneration because nothing survives regeneration. This used to be an unreasonable demand on human time, which is why nobody did it. With an AI teammate it's twenty minutes: the agent reads the current codebase and writes the current document, and your job shrinks to review.

The fixed section order is sequenced by reader need: **what is this and who is it for**, **quick start** (prove it runs, copy-pasteable), **configuration table** (every variable, default, required-or-not), **how to test**, **architecture overview**, **deployment notes**, **troubleshooting** (the top issues you actually hit, not the ones you imagine). A reader should be able to stop at any section boundary

with their question answered. And the first audience for the regenerated README is the next AI session — which reads it before the code, believes what it says, and acts on every stale claim you left in it.

Rule 90: No commented-out code

No commented-out code in the repository — git history is the archive.

Commented-out code is a question mark with no answer attached. Every reader who encounters that gray block has to stop and wonder: Is this coming back? Was it broken? Does the author know something I don't? The block can't answer, the author is long gone, and so it survives — too scary to delete, too dead to run. I've seen functions where the commented-out carcass outweighed the living code three to one, and the living code had been edited so many times the carcass no longer corresponded to anything.

The reason this habit exists is rational fear from a pre-version-control world: *I might need this again, and if I delete it, it's gone.* That fear has been obsolete for decades. Git is the archive. Every line you delete is recoverable, with its full context, its author, its date, and the commit message explaining why it died. `git log -S "the_function_name"` will find it faster than you can scroll through the file looking for the right gray block. Deletion in a version-controlled repo is not destruction; it's filing.

There's an AI angle here too, and it's not hypothetical. Commented-out code sits in the agent's context window looking almost exactly like live code. Agents pattern-match; a big block of plausible-looking logic is a big attractor. I have watched an agent "fix" a bug by un-commenting an obsolete implementation, because the carcass looked more like the training data than the living replacement did. Clean code isn't just for human readers anymore — it's prompt hygiene.

If it might come back, delete it anyway and say so in the commit message. That's what the message is for.

Rule 91: No orphan TODOs

No TODO without a tracker link. Otherwise it's a dated FIXME with a named owner.

A bare `TODO: handle the error case` is a wish, not a plan. It has no owner, no deadline, no priority, and no presence in any list anyone actually reads. Nobody triages the codebase's comment strings, so the bare TODO does the one thing it's good at: it survives. Grep any codebase more than a few years old and you'll find TODOs older than some of the people working on it.

The rule splits the cases. If the work is real, it gets a tracker entry — an issue, a `bugs.md` or `features.md` line (Rule 80) — and the comment carries the link: `TODO(#412): handle the partial-write case`. The tracker entry says *what and why*; the comment marks *where*. When the entry closes, grep for the ID and clear the markers. If the work is *not* tracker-worthy — a note to a near-future self in mid-flight work — it's a `FIXME` with a date and an owner: `FIXME(eddie, 2026-06-11): brittle, revisit after the adapter lands`. The date makes staleness self-evidencing: a dated FIXME from two years ago is a confession, and the next sweep (Rule 87) deletes it or promotes it.

This also disciplines the AI teammates, which scatter TODOs the way they scatter apologies. An agent that must attach a tracker link has to actually *file the item* — which means the gap it noticed gets recorded somewhere durable instead of buried in a comment the next session will never see. The comment string is the worst database you own. Stop writing to it.

Rule 92: A living state file so a cold agent can start

Maintain a living state file so a memoryless agent can start cold: a per-repo `STATE.md` (≤1024 words) ingested at session start and regenerated at the end of every commit, plus a one-

line-per-repo index at the projects root. It points to the ADRs and trackers — never a log; when it overflows, prune detail outward.

Every rule in this chapter has been chipping at the same problem from a different side: your most productive collaborator forgets everything overnight. The changelog, the ADRs, the trackers, the plan statuses — each is one ledger the amnesiac can read. This rule is the front page that ties them together: the one file an agent opens *first*, before it touches a line of code, to answer "where am I and what was I doing?"

`STATE.md` lives at the root of each repo and has a hard budget — a thousand words, no more — because a state file that grows without bound stops being read, by the agent and by you. It says what the project is, what's in flight right now, what the next move is, and where to look for the rest: *the decisions live in `docs/adr/`, the open work in `bugs.md` and `features.md`, the plan statuses in `plans/`.* It points; it does not duplicate. And it is emphatically **not a log** — no dated narration of everything that ever happened, which is what `git log` and the changelog are for. It's a snapshot of *now*, regenerated at the end of every commit so it never drifts behind the code the way a hand-patched status line does.

When the snapshot overflows its budget, you don't raise the budget — you prune detail *outward*, pushing the specifics down into the ADR or tracker they belong to and leaving a pointer behind. Above the repos, a one-line-per-repo index at the projects root tells a cold agent which repo even to enter.

I learned the cost of not having this the hard way: a fresh session, a capable agent, and twenty minutes of it re-reading the entire tree to reconstruct a context that a thousand words would have handed it in one read — then making a confident wrong move because it had reconstructed the context slightly off. The state file is the difference between an agent that resumes and an agent that re-derives. Write the front page. Keep it short. Keep it current.

Chapter 5 card

- **76.** Pushed tags are immutable: never move, delete, reuse, or force-overwrite one. Fetch and compare before tagging; push tags by name, never `--tags`.
- **77.** Bump every release; versions only move forward. Roll back by rolling forward.
- **78.** Persist standing decisions in the same commit — chat history is not memory. Non-obvious ones go in numbered ADRs, immutable after acceptance; supersede, never edit.
- **79.** SemVer, in exactly one canonical place; everything else reads from it.
- **80.** File every bug and feature the moment it's observed, even if fixed the same turn; questions go inline.
- **81.** Every build gets a unique, monotonic build number, generated by the build itself.
- **82.** Quote errors verbatim, show diffs not prose, surface assumptions explicitly.
- **83.** The changelog updates in the same commit as the version bump.
- **84.** Plans carry a first-line `Status:` kept current; stale status on shipped work is a violation.
- **85.** The version (plus build, SHA, date) shows on the splash, `--version`, and `/health`.
- **86.** Lint and format on every commit; CI stays green.
- **87.** Run a dead-code pass after meaningful feature work; ask before deleting anything uncertain.
- **88.** After a stable release, the first task is a cleanup sweep — before any new feature.
- **89.** Regenerate the README from scratch after major changes — never patch it.
- **90.** No commented-out code — git history is the archive.
- **91.** No TODO without a tracker link; otherwise a dated FIXME with an owner.

- **92.** A living `STATE.md` (≤1024 words) so a cold agent starts: regenerated each commit, points to ADRs and trackers, never a log.

That's the ship-and-remember discipline: every artifact identified, every tag immutable, every decision, plan, and project state telling the truth about itself in writing, and the workshop swept clean enough that all of it stays legible — to the next human reader and to the amnesiac machine that ingests the whole repo as its working memory.

Every rule to this point makes *one* agent good — careful, scanned, covered, honest about what it shipped and what it decided. But the way I actually work now isn't one careful agent. It's a fleet of them, most of them cheap and forgetful, and a system that has to be good even when no single worker in it is. That's a different discipline, and it's the last one. Turn the page.

Chapter 6 — Operating an AI Fleet

> **Standing on Chapters 1–5:** - The hard rules never bend, and the Powell rule holds: gather 90% of the information, then decide (Chapter 1). - Nothing is hardcoded; everything that can change flows through config, behind an interface (Chapter 2). - Builds are portable, fail loudly, and carry the fewest dependencies that do the job (Chapter 3). - Every artifact is scanned clean and proven against the rubric with 100% branch coverage (Chapter 4). - Every artifact is identified, every tag immutable, every decision and plan written down for the amnesiac who reads next (Chapter 5).

Everything before this chapter makes *one* agent good. Pick the right model, hand it the rules, gate its output, and a single capable assistant produces work you can ship. That's the whole book up to here, and it's enough to get real software out the door.

This chapter is about the next thing, and it's a different thing. The economics of running one frontier model on every task are bad and getting worse — most of what an agent does is mechanical, and paying top-tier rates to rename a variable or reshape a JSON blob is like flying a principal architect out to change a lightbulb. The obvious fix is to use cheaper, smaller models for the cheap, small work. The non-obvious problem is that a small model is genuinely worse: it forgets rules, it loses the thread past a few thousand tokens, it confidently produces output that almost works. Hand a 2-billion-parameter model the same hundred rules you'd hand a frontier model and it doesn't follow ten of them — it follows none of them, because the instructions themselves overflowed whatever it could hold.

So the move that makes a fleet pay isn't "trust a cheaper model." A cheaper model, trusted, is a liability. The move is to stop trusting the *model* and start trusting the *pipeline*: a cheap model that only

has to be good enough to try first, an automated gate that catches what it got wrong, and an escalation path that sends the residual — the genuinely hard fraction — up to a stronger tier. The quality bar moves off the model and onto the system. A small model that's right 70% of the time, behind a verify gate that bounces the other 30% up a tier, beats a single expensive call on cost and — this is the part that surprised me — often on reliability too, because the gate catches the frontier model's bad days as well.

This is the one chapter where the book's claims are receipts, not anecdotes. The rules here came out of experiments run in this very repo: a size ladder that handed the same task to models from 2B to frontier with rule-counts from 2 to 16 and measured exactly where each one fell off; a verify-then-escalate harness that ran the cheap-tier-plus-gate pattern end to end. The numbers are blunt. A 2B model holds about two rules. An 8B model holds about eight. Past its ceiling, every model's score doesn't gently decline — it falls off a cliff, because an overloaded model isn't a worse model, it's a different and unpredictable one. The eight rules in this chapter are what those numbers taught me, stated as discipline.

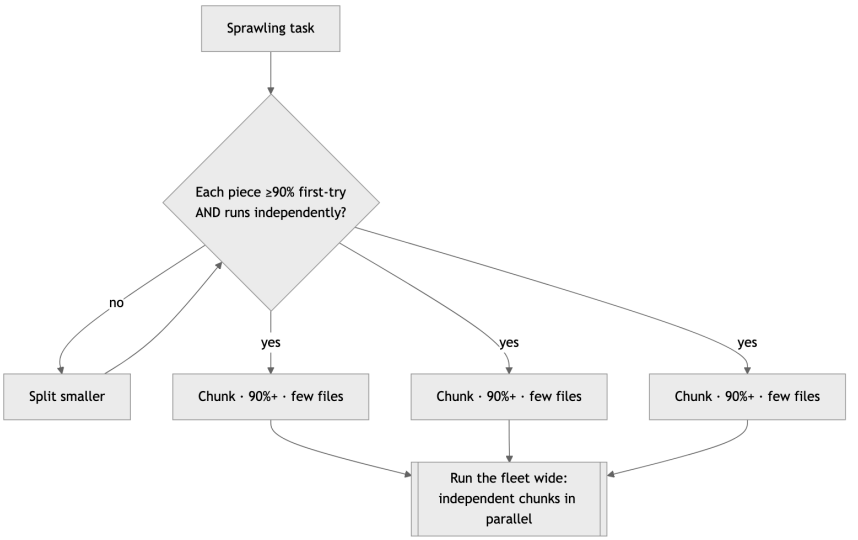
One framing before the rules. A senior engineer carries a thousand unwritten reflexes — the flinch at an unvalidated input, the instinct to re-run before trusting a green, the latency budget felt in the gut. A small model carries none of them, and even a frontier model carries them inconsistently. The whole architecture of a fleet is a machine for converting that tacit senior expertise into explicit rules, slices, gates, and escalations that a model with no instincts can still be held to. That conversion is the last rule in the book, and it's the closing argument for all hundred.

Rule 93: Chunk it so the model nails it

Chunk every task into bite-size, parallel-capable units, each sized so the AI nails it first try ≥90% of the time. Anything with a >10% chance of first-try failure, or that can't run independently of its siblings, gets split smaller.

This is Rule 90′s “plan first” turned into a sizing law for a fleet. Planning tells you *what* the work is; chunking decides how *big* each piece handed to a model gets to be. And the unit of measurement isn’t lines of code or hours — it’s first-try success probability. A chunk is correctly sized when the model assigned to it succeeds on the first attempt at least nine times in ten. Anything below that bar isn’t a chunk yet; it’s two chunks pretending to be one.

The reason this matters more for AI than it ever did for humans is that a model’s reliability collapses non-linearly with ambiguity and scope. A human handed a vague, sprawling task muddles through at reduced quality — a B-minus instead of an A. A model handed the same task doesn’t degrade gracefully; it confabulates. The tenth step of a sweeping ten-step plan isn’t done slightly worse, it’s done wrong, with total confidence, in a way that reads exactly like done right. So you don’t hand a model a sprawling task. You decompose until every piece has a binary done-signal — the build passes, the test goes green, the schema validates — and a failure probability under 10%.



Chunking is a loop, not a single cut. A piece under the 90% bar, or one that waits on a sibling, goes back through the splitter. What comes out the bottom runs six-wide, not six-deep.

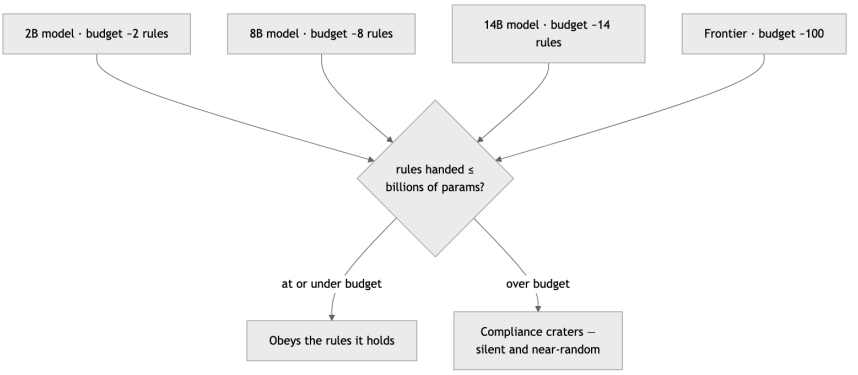
The second clause is what turns chunking into *fleet* discipline: parallel-capable. Chunks sized this way come out independent more often than not — few files each, one clear output each — and independent chunks run simultaneously across the fleet. That's the throughput multiplier. A task split into six independent 90%-chunks doesn't take six times one chunk; it takes about one, run six-wide. Split anything that can't run free of its siblings smaller until the seam is clean, or the dependency forces you back into serial and you've lost the multiplier.

A correctly sized chunk has two properties: a model nails it first try, and it doesn't wait on its neighbors. Miss either and split again.

Rule 94: The sizing law — one rule per billion

A model reliably holds about one rule per billion parameters at the 90% bar (2B→~2, 8B→~8, 14B→~14). Never hand a model more rules than it has billions of parameters.

This is the most concrete claim in the book, so let me show the work. I ran a ladder: the same coding task, handed to models from 2 billion parameters up through frontier, each tested against rule-sets of growing size — 2 rules, then 8, then 14, then 16. I measured how many of the supplied rules each model actually honored. The pattern was clean enough to be a little eerie. A model holds roughly as many rules as it has billions of parameters, and past that line its compliance doesn't taper — it craters. An 8B model riding eight rules scored well; the same 8B model handed sixteen rules didn't follow eight of them, it followed a near-random handful, because the surplus instructions crowded out the ones it could have kept.



One rule per billion parameters, and the line is a cliff, not a slope. One rule past the ceiling and the model doesn't follow fewer rules — it follows them at random, and never says so.

The mechanism, as best I can reason about it: rule-following is itself a capacity cost. Every rule the model must hold in working attention competes with every other rule and with the task. Below the ceiling, the model has headroom and obeys. At the ceiling, it's saturated. Past it, adding a rule doesn't add a constraint — it *removes* one at random, because the model is now thrashing. The 2B model that perfectly honored its two rules became useless at eight not because the eight were hard but because two was its whole budget.

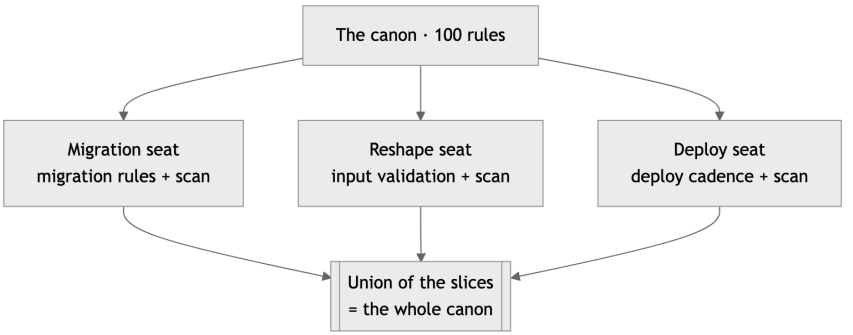
So the law is a budget, and you spend it deliberately: never hand a model more rules than it has billions of parameters. A 2B model gets your two most load-bearing rules and nothing else. A 14B model gets fourteen. The frontier model gets the full hundred. This is not a suggestion to round up hopefully — the cliff is real and the far side is worthless. When a task genuinely needs more rules than the cheap tier can hold, that's not a reason to overload the cheap tier; it's the signal to slice (Rule 95) or escalate (Rule 99).

One rule per billion parameters. Past the ceiling a model doesn't bend — it breaks, and it breaks silently.

Rule 95: Slice the rules to the seat

Give each model or task only the rules its job needs — a *view* of the canon, not the whole book. The canon is the union of all slices; no single seat holds all of it.

Rule 94 sets the budget; this rule is how you live within it. If a 2B model can hold two rules, the question isn't "which ninety-eight rules do I drop?" — it's "which two does *this seat* actually need?" A model writing a database migration needs the migration rules, the secret-scan rule, and almost nothing about frontend cross-platform layout. A model reshaping a JSON payload needs the input-validation rule and the schema, not the deploy-cadence rules. Each seat gets a *view* — the slice of the canon its job touches — and the slice fits its budget by construction.



Each seat carries a view sized to its budget, not the whole book. The scan rule rides in every slice because it's universal; stitch the slices across the fleet and every rule is enforced somewhere — by the seat that needs it.

This inverts how people instinctively deploy rules. The instinct is to hand every agent the whole rulebook, on the theory that more guidance is more safety. For a frontier model with room to spare, fine. For everything below it, the full rulebook is actively harmful: it blows the budget, triggers the Rule 94 cliff, and you get *less* compliance by giving *more* rules. The slice isn't a compromise forced by small models — it's the correct design even when you could afford

not to, because a focused model on a focused rule-set outperforms a saturated one every time.

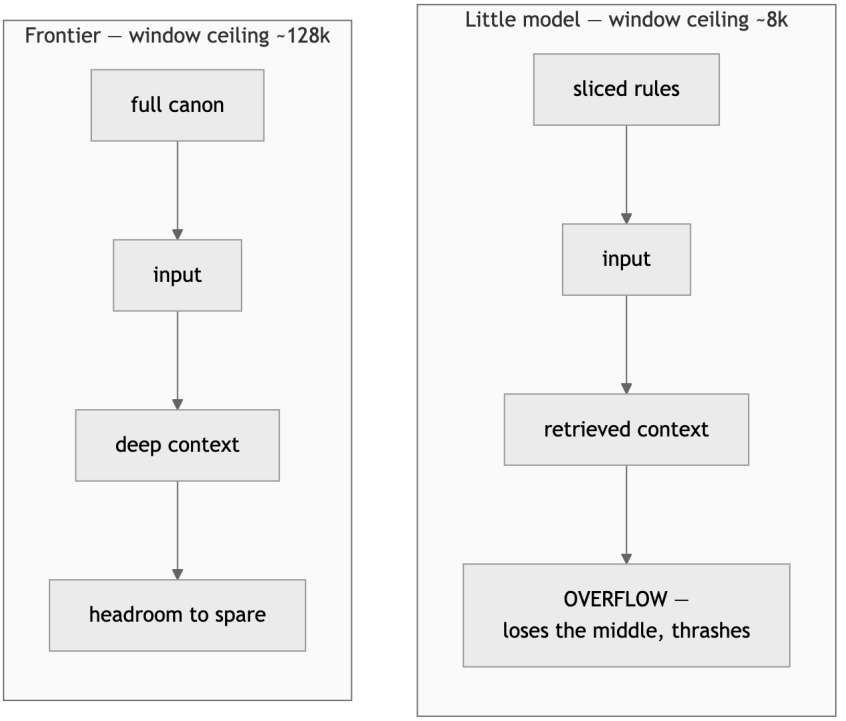
The load-bearing half is the second sentence. No single seat holds the whole canon — but the canon still governs the whole system, because it's the *union* of the slices. The migration seat carries the migration rules; the deploy seat carries the deploy rules; the secret-scan rule rides in every slice because it's universal. Stitch the slices across the fleet and every rule is enforced somewhere, by the seat that needs it, within the budget that seat can hold. The book exists whole; no one model has to.

Distribute the canon, don't replicate it. Each seat carries its view; the fleet carries the book.

Rule 96: Mind the context ceiling

Context ceiling by tier: a little model never deals with more than ~8k of context (it degrades past ~8–10k); a frontier model is safe to ~128k. Bound every cheap-tier task — input as well as rules — to its ceiling.

Rule 94 budgets the *rules*; this rule budgets *everything else in the window* — the input, the surrounding code, the prior turns, the retrieved context. They draw on the same pool, and a small model's pool is shallow. A little model degrades sharply past roughly eight to ten thousand tokens: not "answers get a bit worse," but the same cliff Rule 94 describes, where the model loses the middle of its own context and starts answering from fragments. A frontier model stays coherent out to around 128k. These are different machines with different ceilings, and a task that's trivial for one overflows the other.



Rules and input draw on one window. Pile a sliced model's two rules on top of a 30k-token file and you've blown the ceiling on input alone. Bound the whole prompt to the tier, or the tier degrades into a different, unreliable machine.

I learned to feel this as a second budget that has to clear alongside the rule budget. It's no use slicing a 2B model down to its two rules if you then paste in a 30k-token file for it to edit — you blew the ceiling on input and the model is thrashing again, rules or no rules. So bounding a cheap-tier task means bounding the *whole* prompt: the sliced rules, plus the input, plus whatever context the work needs, all of it under the tier's ceiling. If the input alone won't fit, the task is mis-assigned — either chunk the input smaller (Rule 93), retrieve only the relevant span instead of the whole file, or escalate to a tier whose ceiling holds it (Rule 98).

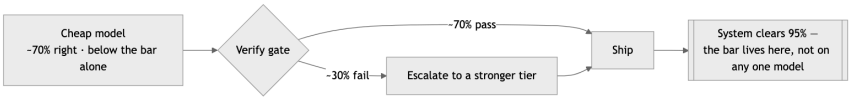
This is also why "just give the small model more context to compensate for being small" is exactly backwards. More context doesn't help a model that's already near its ceiling — it pushes it over. The cheap tiers earn their keep on *small, bounded* tasks: a focused input, a focused rule-slice, a clear output. Keep them in the zone where they're reliable, and route anything that needs to hold a lot at once to the tier built to hold it.

Rules and input share one window. Bound the whole prompt to the tier's ceiling, or the tier degrades into a different, unreliable model.

Rule 97: Good enough at the model, 90% at the system

A cheap model only has to be good enough to try first. The quality bar belongs to the pipeline — cheap model + a verify gate + escalate-the-residual to a stronger tier — not to any one model.

This is the hinge rule of the whole chapter, the one that makes a fleet of mediocre models produce excellent work. Chapter 4 set the quality bar: 90% on the rubric to work, 95% to publish. The trap is to read that as a requirement on *each model*. It isn't. It's a requirement on the *system*. A single model never has to clear the bar alone — the pipeline clears it.



Seventy-percent-right is unusable as a final answer and a bargain as a first attempt behind a gate. Pull the model, the gate, or the escalation and the whole thing drops below the bar.

Here's the reframe that took me a while to trust. A cheap model's job is not to be right. Its job is to be *good enough to try first* — to produce a candidate cheaply, fast, and often enough that catching its mistakes downstream is still cheaper than doing the work at

frontier rates from the start. A model that's right 70% of the time sounds unusable as a final answer. As a first attempt behind a gate that bounces the 30% it got wrong, it's a bargain: you paid cheap-tier rates for seven of every ten results, and frontier rates for only the three the gate kicked up.

The quality, then, lives in three components, not one: the cheap model that tries, the verify gate that grades the attempt (Rule 99), and the escalation path that sends failures to a stronger tier. Pull any one and the system collapses — a cheap model with no gate ships its 30% of garbage; a gate with no escalation just rejects work without producing the answer; escalation with no cheap first tier is just the expensive single-model setup you were trying to escape. Together they hit the system bar at a fraction of the cost.

The experiments bear this out and add a twist I didn't expect: the gate catches the *frontier* model's failures too. No model is right every time. A verify gate that exists to catch the cheap tier's misses also catches the expensive tier's bad days — so the pipeline is often *more* reliable than a single frontier call, not just cheaper. Quality you build into the system survives any single model's off day. Quality you rest on one model's brilliance fails exactly when that model does.

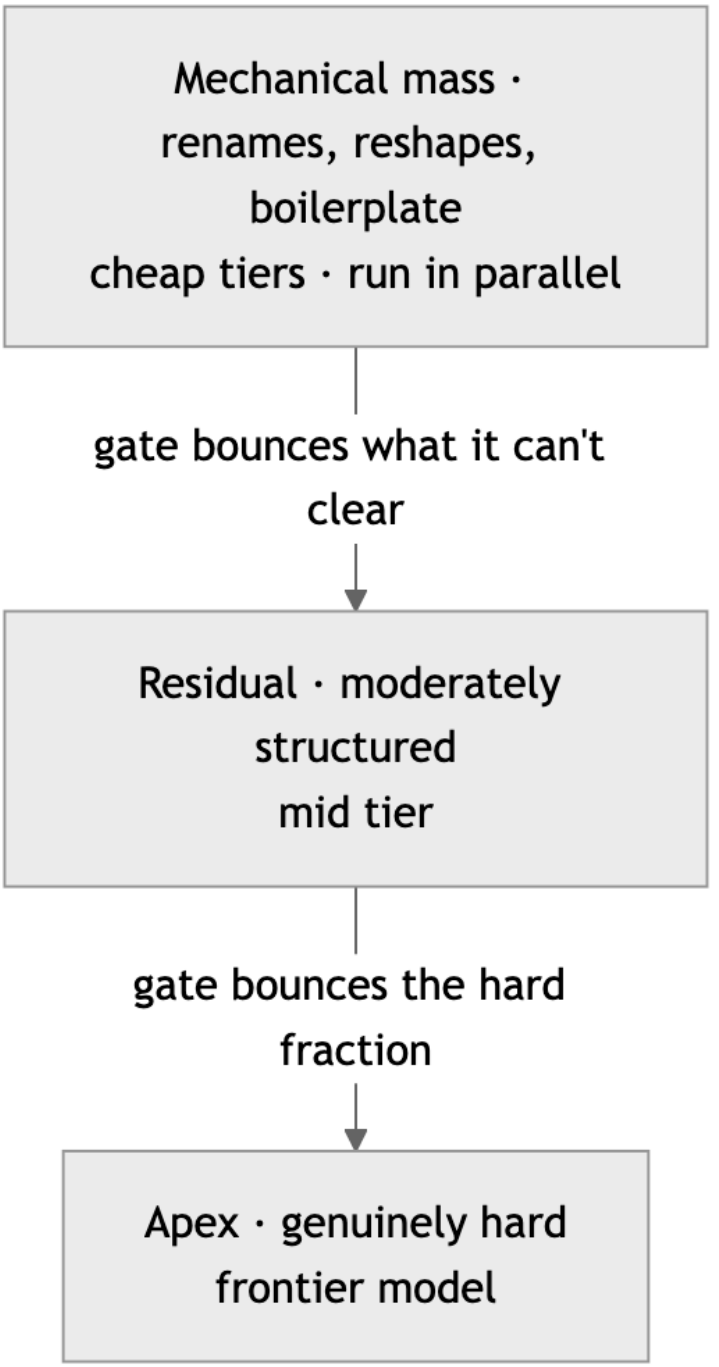
Stop asking "is this model good enough?" Ask "is the pipeline good enough?" The model only has to try; the system has to be right.

Rule 98: Match the work to the model

Size, rule-count, and context all scale together. Send mechanical work to the smallest tier that clears it, escalate only the residual, and reserve the frontier model for the genuinely hard fraction.

The three budgets of this chapter — parameter count (Rule 94), rule-count (it's the same number), and context ceiling (Rule 96) — are not independent dials. They scale together, and they describe a *tier*. A 2B model is a tier: two rules, a few thousand tokens of

context, mechanical work only. A 14B model is a tier: fourteen rules, more headroom, moderately structured work. The frontier model is a tier: the whole canon, deep context, genuinely hard reasoning. Matching work to a model means picking the tier whose three budgets all clear the task — not just one of them.


Mechanical mass · renames, reshapes, boilerplate
cheap tiers · run in parallel
gate bounces what it can't clear
Residual · moderately structured
mid tier
gate bounces the hard fraction
Apex · genuinely hard frontier model

The work is a pyramid: the broad base runs cheap and parallel, only the residual escalates, and the frontier model handles the narrow apex — affordable precisely because it's narrow. Cheapest tier that clears the task, not cheapest available.

The economic instinct is "use the cheapest model that works," and that's right, but the operative word is *works*. Cheapest-that-clears, not cheapest-available. A task needing ten rules does not go to an 8B model to save money — it goes to the 14B tier or it gets sliced (Rule 95) until each piece fits the 8B budget. Underprovisioning is not thrift; it's the Rule 94 cliff with a cost-savings rationalization on top, and the bounced work plus the cleanup costs more than running the right tier the first time.

The shape this produces is a pyramid of work. The broad base — the mechanical mass, the renames and reshapes and boilerplate — runs on the cheap tiers, in parallel, fast and nearly free. Above it, the residual that the base tier couldn't clear escalates to a middle tier. And at the narrow apex, the genuinely hard fraction — the load-bearing architecture, the subtle reasoning, the work where being wrong is expensive — gets the frontier model, which is now affordable precisely because it's only handling the apex instead of the whole pyramid. Reserve the expensive thinking for the decisions that are expensive to get wrong. The fleet's whole economic argument is that the apex is small.

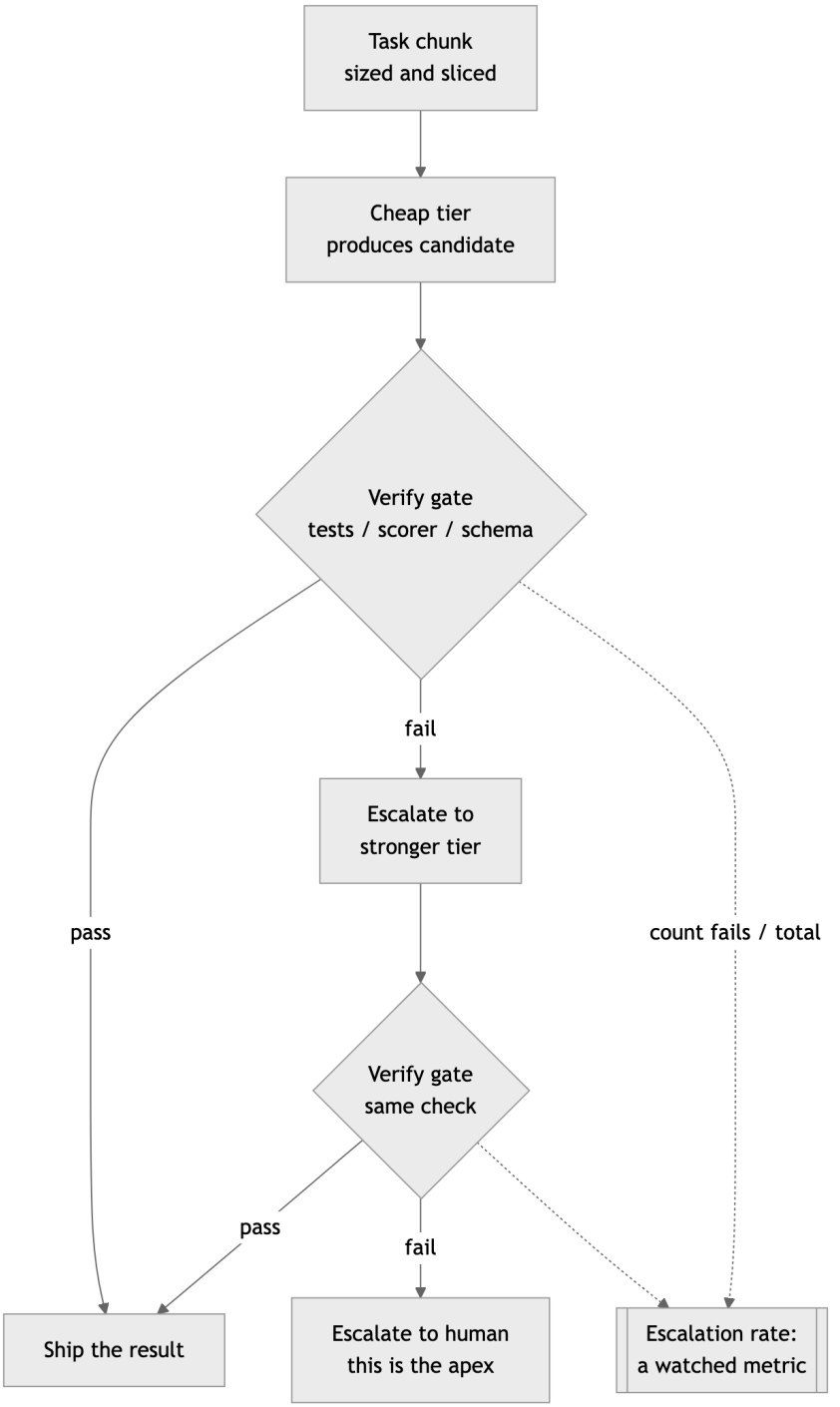
Pick the smallest tier whose every budget clears the work. Escalate the residual upward. Keep the frontier model for the apex, where it earns its rate.

Rule 99: Verify, then escalate

Gate cheap-tier output with a fast automated check — tests, a scorer, a schema. Ship what passes, bounce what fails up a tier. The escalation rate is a metric to watch, not a surprise.

This is the rule that makes Rule 97 mechanical instead of aspirational, and it's the heart of the pipeline. The cheap model produces

a candidate; before that candidate goes anywhere, an *automated* gate grades it; passes ship, failures escalate to a stronger tier. The word that carries the rule is *automated*. A human verifying every cheap-tier output is just expensive review with extra steps — you've moved the cost, not removed it. The gate has to be a program: a test suite, a schema validator, a rubric-scorer, a linter, a compile step. Something that runs in milliseconds and returns pass or fail without an opinion.


Task chunk
sized and sliced
Cheap tier
produces candidate
Verify gate
tests / scorer / schema
fail
pass
Escalate to
stronger tier
count fails / total
Verify gate
same check
pass
fail
Ship the result
Escalate to human
this is the apex
Escalation rate:
a watched metric

The pipeline. Solid edges are the work path: cheap tier, gate, ship-or-escalate, gate again, human at the apex. The dashed edges feed the escalation-rate metric — the gate's verdicts, counted.

The escalation rate is the instrument on the dashboard. It's the fraction of cheap-tier outputs the gate bounces upward, and you watch it the way you watch any health metric. Stable and low means the tiering is well-tuned — the cheap tier is clearing most of its assigned work and the gate is catching the rest. A *rising* escalation rate is an early warning: the cheap tier is being handed work above its budget, or the chunks have grown too big (Rule 93), or a model changed underneath you. Either way the system tells you before the cost blows out, because the metric moves before the bill does. A pipeline whose escalation rate you don't measure is a pipeline that can silently degrade into "frontier model on everything" — the exact cost structure you built the fleet to escape — and nobody notices until accounting does.

One caution the experiments made vivid: the gate is only as good as its check. A weak gate — one that passes output it shouldn't — ships the cheap tier's garbage with a green stamp on it, which is worse than no gate, because now the failures look verified. The gate inherits Chapter 4's whole discipline: real tests, real coverage, a real scorer. Verify, then escalate — but verify for real.

Rule 100: Externalize the reflex

The expertise a senior carries tacitly — the latency instinct, the re-run check, the untrusted-input flinch — must become an explicit rule, test, or gate, because the agent at the keyboard has none of it.

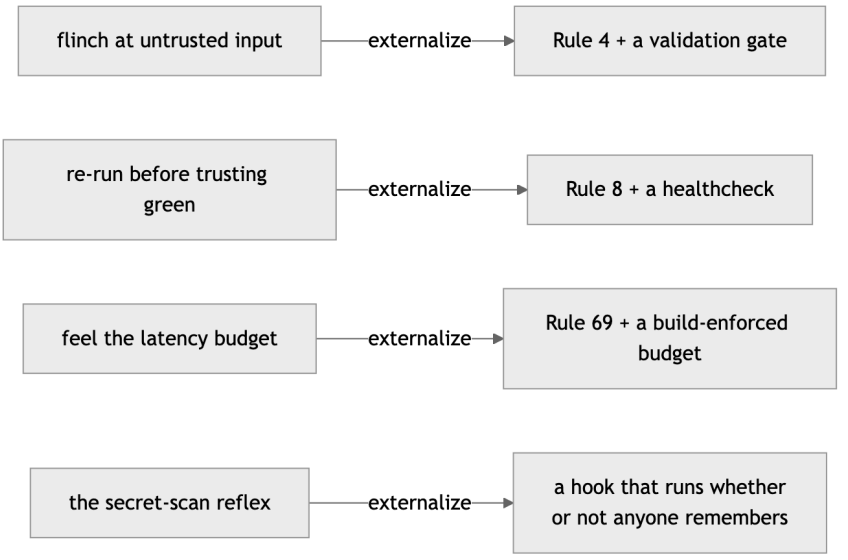
Here at the end, the rule that the other ninety-nine were quietly building toward.

A senior engineer is mostly reflexes. Forty-seven years in, the things that keep my software alive aren't the things I think about — they're the things I no longer have to. I flinch at a string concate-

nated into a query before I've consciously read it as SQL injection. I re-run the build instead of trusting the green, because some scar from a piped exit code taught my hands to. I feel a latency budget in my gut and wince when a change blows past it. None of that is knowledge I could hand you on a page. It's tacit — worn smooth by enough failures that it became instinct, the kind of thing a senior can do but can't fully explain.

The agent at the keyboard has none of it. Not the small model — the small model obviously has none. But not the frontier model either, not reliably. A model's instincts are an average of its training data, which means they're a B-minus engineer's instincts on a good day and absent on a bad one. You cannot wait for the flinch, because there is no flinch. The reflex that fires automatically in a senior's nervous system has to fire *somewhere else* in a fleet — and the only somewhere else is the system itself.

So that is what this whole book has been: a machine for externalizing the reflex. Every rule in it is a tacit senior instinct dragged into the open and welded to the pipeline where a model with no instincts can still be held to it. The flinch at untrusted input became Rule 4 and a validation gate. The re-run-before-trusting-green became Rule 8 and a healthcheck. The latency instinct became Rule 69 and a budget the build enforces. The secret-scan reflex became a hook that runs whether anyone remembers to or not.



Tacit on the left, executable on the right. Every rule in the book is a senior's instinct dragged into the open and welded to the pipeline — so a C-grade model with no instincts is structurally unable to ship below the bar. None of these depend on the agent being wise. They depend on the wisdom having already been extracted, written down, numbered, and turned into something that executes — a rule sliced to a seat, a test that must pass, a gate that bounces failures up a tier.

That's the closing argument, and it holds for the whole fleet at once. The machine's default output is C-grade — functional, average, instinct-free, exactly what an average of all code would predict. You do not get A-grade work by hoping for a better model; the better model has the same gap, just smaller. You get A-grade work by externalizing enough senior reflex into the system that a C-grade model, sliced and gated and escalated, is *structurally unable* to ship below the bar. The discipline doesn't live in the model. It lives in the rules, the tests, the gates, and the tiers — in everything that keeps firing after the human who knew better has left the room. Build

that, and the fleet produces work better than any single model in it. That is the entire point. That is why you write the rules down.

Chapter 6 card

1. **Chunk it so the model nails it** — bite-size, parallel-capable units, each ≥90% first-try; split anything >10% fail-risk or non-independent.
2. **The sizing law: one rule per billion** — ~1 rule per billion params (2B→~2, 8B→~8, 14B→~14); past the ceiling, compliance craters silently.
3. **Slice the rules to the seat** — each seat gets a *view* of the canon, not the whole book; the fleet's union is the canon.
4. **Mind the context ceiling** — little model ≤~8k, frontier safe to ~128k; bound the whole prompt, rules and input, to the tier.
5. **Good enough at the model, 90% at the system** — the cheap model only has to try first; the quality bar lives in the pipeline.
6. **Match the work to the model** — size, rules, and context scale together; smallest tier that clears it, frontier for the apex.
7. **Verify, then escalate** — automated gate grades cheap output; pass ships, fail goes up a tier; watch the escalation rate.
8. **Externalize the reflex** — the senior's tacit instinct becomes an explicit rule, test, or gate, because the agent has none of its own.

That's the fleet, and that's the hundredth rule. The thread that runs from the first scan-before-ship to this last externalized reflex is a single claim: software is built by collaborators who forget, misremember, and have no instincts of their own — and written, enforced, externalized discipline is the only thing that makes their C-grade defaults produce A-grade work. The models will change. The ceilings will rise, the tiers will shuffle, the parameter counts in Rule 94 will look quaint in a year. The discipline of extracting your expertise into rules a memoryless, instinct-free agent can be held to — that survives every model you'll ever bind to a seat. Write

it down, slice it, gate it, and let the machine fire the reflex you no longer have to.

Chapter 7 — From a Hundred Rules to an Axiom Core

> **Standing on Chapters 1–6:** - The hard rules never bend, and the Powell rule holds: gather 90% of the information, then decide (Chapter 1). - Nothing is hardcoded; everything that can change flows through config, behind an interface (Chapter 2). - Builds are portable, fail loudly, and carry the fewest dependencies that do the job (Chapter 3). - Every artifact is scanned clean and proven against the rubric with 100% branch coverage (Chapter 4). - Every artifact is identified, every decision and plan written down for the amnesiac who reads next (Chapter 5). - A fleet earns its keep by trusting the pipeline, not the model: a rule holds about one rule per billion parameters, so each seat carries a slice of the canon, not the whole book (Chapter 6).

Chapter 6 left a debt. It said a small model holds about one rule per billion parameters, so you slice the canon and give each seat only the rules its job needs. Fine. But slice *how*? A hundred rules in a flat list is a hundred things of supposedly equal weight. Hand that to the knife and the first question back is: which of these can the migration seat do without, and which can it never, ever miss? The list doesn't say. It's a hundred peers. A flat list has no grain to cut along.

This chapter is the grain. It adds no rules — the canon is still exactly one hundred, and the count is sacred. What it adds is a *shape*: the hundred is not a flat list and never was. It is a small immutable core that every seat must carry, wrapped in layers of preference that get paged in only when a task touches them. The number is the union. The structure is what makes the union fit on machines that can't hold it.

The shape came from an outside knife. I had been defending the flat hundred for months — keep the count, flag the weak ones, never cut. A reader named Guy Turgeon asked the one question I'd been routing around: *how small can the crew get and still be safe?* You cannot answer that about a flat list. You can only answer it about a structured one. So I structured it. The rest of this chapter is his scalpel.

The hundred was never flat

I graded all hundred rules on one scale — a six-dimension rubric, the same one Chapter 4 demands of any project. The role rules and the project rules sank to the bottom. "Meet the crew" graded 33.5. "Go local" graded 25.5. The Podman-and-UBI stack graded 45. Down at the floor, looking like the weakest rules in the book.

They aren't weak. They're *mis-scoped*. The rubric scores a rule partly on generality — does it hold for everyone, everywhere — and a rule about my five personas graded as a universal law always loses on generality, because it was never a universal law. It's my preference. Grading it against the secret-scan rule is grading a house style against a law of physics. The low score isn't a verdict on the rule. It's the rubric noticing the rule is a different *kind* of rule and the flat list pretending it isn't.

That's the crack the whole chapter opens. Some of these rules are axioms — they hold for any serious engineer, any project, any shop. Some are opinions — good ones, mine, defensible, and disputed by reasonable people. The flat hundred filed them side by side and the grader couldn't tell them apart. Stop pretending they're the same kind, and the structure falls out on its own.

The axiom core

At the center is the axiom core: the rules that hold no matter who you are or what you're building. Scan before every boundary. Never hardcode a secret. Destruction needs a human. Distrust every external input. The Powell rule. Autonomy bounded by version

control. Green before commit. Fail fast. Around two dozen of them — the full list is in Appendix F.

The test for the core is not "is this rule good." Most of the hundred are good. The test is "would skipping this ever be defensible, anywhere?" An axiom is a rule whose violation is indefensible in every shop on the planet. Almost nobody argues that you should commit secrets, or skip the scan, or drop a production table without asking. Those are the axioms. They are few on purpose, because the core has to be the smallest set a seat must *never* be without — and a small model has room for a small core and nothing more.

Keep the core few and it travels. Every seat carries it, the 2-billion-parameter mechanical drone as much as the frontier architect, because there is no task cheap enough to be allowed to leak a key. The axioms are the rules with no off switch.

The preference layers

Everything outside the core is a preference, and preferences come in scopes. Mine stack like this:

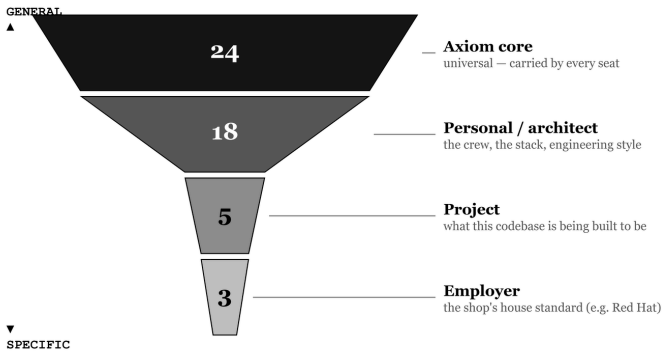
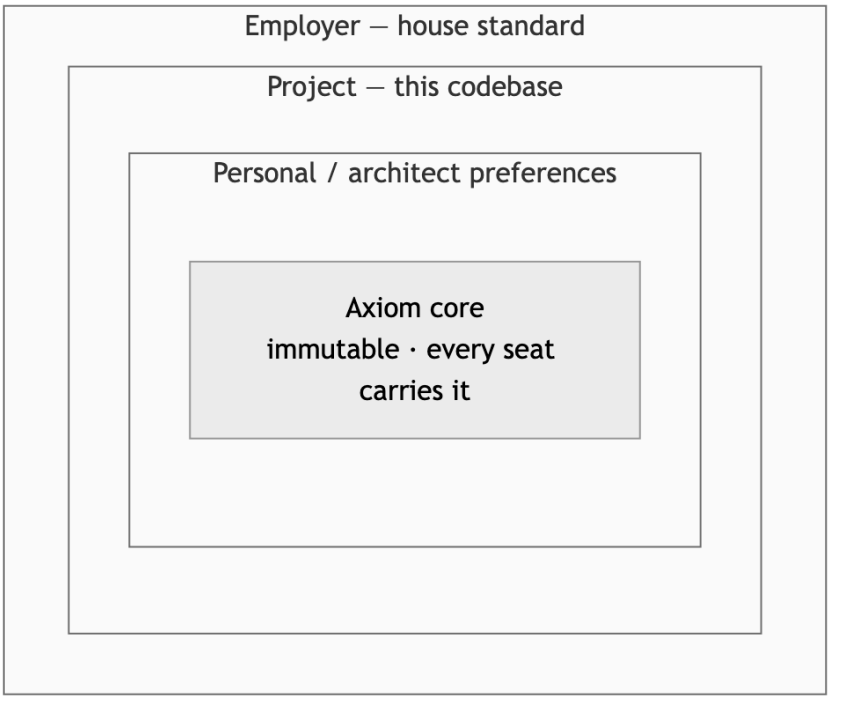


Figure 2: The hundred as layers: a wide, universal axiom core tapering to the narrow, most-specific employer layer. Slice width tracks rule count; the brand-name stack lives at the bottom, not the core.

- **Personal and architect preferences** — my engineering opinions and my crew. Objects with one job. Files under five hundred lines. One non-trivial class per file. The config-precedence stack. The five personas and which model sits in each seat. All defensible, all disputed, all *mine* — opinions, so preferences, not axioms.
- **Project preferences** — what this particular codebase needs. The migration rules a database project lives by and a static site never sees. The schema a payload-reshaping task is held to.
- **Employer preferences** — the shop's house standards. Mine: Podman over Docker, UBI base images, OpenShift. Correct for me. Hardcoded to nobody else.

This is the book's own governance hierarchy, finally made literal — the town, the library, the bookshelf, the book, each free to tighten the rules it inherits and forbidden to loosen them. The axioms are the town laws: they bind everyone below and nobody may relax them. Each layer down adds its own constraints on top. A preference can make a rule stricter for its scope; it can never cancel an axiom. Constraints tighten downward. They never loosen.



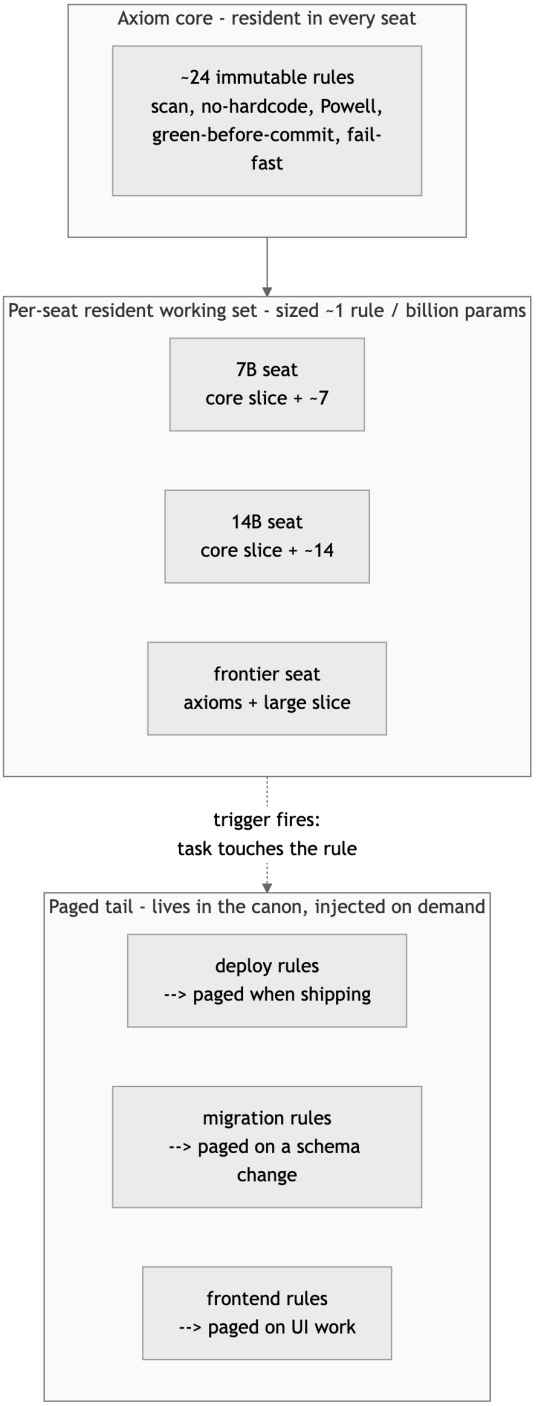
The same hierarchy as containment. The axiom core sits at the center, carried by every seat; each enclosing layer may make an inherited rule stricter for its scope, never laxer — and pages in only when a task reaches that scope.

The low grades make sense now. "Go local" graded 25.5 not because it's a bad rule but because it's a personal-layer preference being scored as a town law. In its own layer, scored against its own scope, it's exactly right. It was only ever in the wrong column.

Rules as a cache hierarchy

The layers aren't filing. They're a memory budget, and the budget is the one from Chapter 6: about one rule per billion parameters. Treat the ruleset the way a processor treats memory — a cache hierarchy, fastest and smallest at the center, largest and slowest at the edge.

- **Resident, per seat.** Each model carries a working set sized to its budget — held in weights by a fine-tune, or pinned in the system prompt on every call. The 7-billion seat resides about seven rules. The 14-billion seat resides fourteen. The frontier seat resides the axioms and a large slice besides.
- **Team-resident union.** No single seat holds the whole canon, but the seats *together* hold every rule that needs to be standing. That union is the canon made operational — Chapter 6′s "the fleet carries the book" stated as a memory layout.
- **Paged tail.** Every rule no seat keeps resident still lives in the canon, and it gets paged into the context window the moment a task touches it. About to deploy — inject the deploy rules. Untrusted input on the bench — inject the input-validation rule. Just-in-time, bounded by the tier's context ceiling, gone again when the task is done.


Axiom core - resident in every seat
~24 immutable rules
scan, no-hardcode, Powell,
green-before-commit, fail-
fast
Per-seat resident working set - sized ~1 rule / billion params
7B seat
core slice + ~7
14B seat
core slice + ~14
frontier seat
axioms + large slice
trigger fires:
task touches the rule
Paged tail - lives in the canon, injected on demand
deploy rules
--> paged when shipping
migration rules
--> paged on a schema
change
frontend rules
--> paged on UI work

The ruleset as a cache hierarchy. The axiom core is resident everywhere. Each seat pins a working set sized to its budget. The rest is paged in by trigger and released — the canon held whole across the fleet, never whole in one seat.

This is why the core has to be small. Resident space is the scarcest thing in the system. The always-on center can only be the rules that must never be a page-fault away — because the one time you need the secret-scan rule is the one time there's no slack to go fetch it.

Guy's test — how small can the crew get?

Now Guy's question has an answer, because there's something to count. Team-resident capacity is the sum of the per-seat budgets. A crew is *safe* when that summed budget covers the required-resident set — the axioms, plus whatever project rules this job can't afford to page. Guy's test is the floor: the smallest crew, fewest seats and fewest parameters, whose budgets still cover the required-resident set. Below it, an axiom has nowhere to live and the system is unsafe. At it or above it, every must-hold rule has a home.

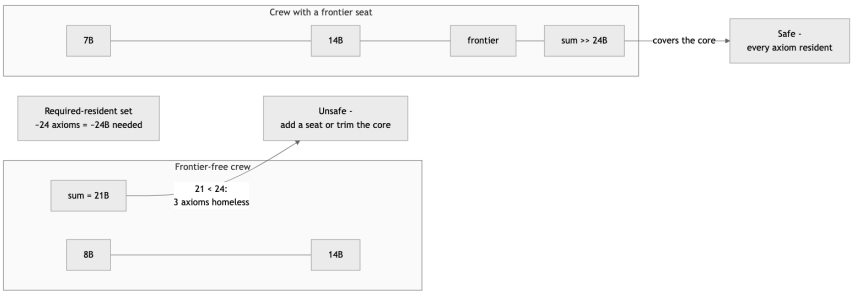
Run the test on my own crew and it bites immediately. The axiom core is twenty-four rules, so holding every axiom resident needs about twenty-four billion parameters of summed budget. A frontier-free crew of small models — say an 8-billion seat and a 14-billion seat — sums to twenty-one. Twenty-one is short of twenty-four. Three axioms have nowhere to sit. That crew is unsafe by three rules, and the test says so in one line of arithmetic.

Two levers fix it. Add a frontier seat, whose budget swallows the whole core and a long slice besides — which is what my working crew does:

Seat	Model	Budget	Resident	Paged
Jason	7B coder	7	7	35
Claude	14B	14	14	33

Seat	Model	Budget	Resident	Paged
Claudius	frontier	200	50	0

Or — if you want a crew with no frontier seat at all, cheaper and fully local — trim the axiom core until it fits the small models' summed budget. Get the core to twenty-one and the 8-plus-14 crew clears it with nothing to spare. That is the real pressure to keep the core lean: every axiom you add is a billion parameters of crew you've just made mandatory.



Guy's test. Sum the seats' budgets; compare to the required-resident set. The frontier-free pair falls three rules short — fix it by adding a seat or trimming the core. The crew with a frontier seat clears it outright. Crew size is the capacity dial: harder project, add seats; easier one, drop a seat and let more rules page.

Crew size, then, is not a staffing accident. It's the capacity dial. A harder project needs more rules held standing at once, so you add seats and the resident capacity grows. An easier one drops a seat and lets more of the canon page in on demand. The dial reads in billions of parameters, and Guy's test tells you where the bottom of the dial is.

Chapter 7 card

This chapter adds no rules. It gives the hundred a shape — carry this shape, not a flat list:

1. **The hundred was never flat** — some rules are axioms (indefensible to skip, anywhere), some are preferences (yours, good, disputed). A flat list hides the difference; the grader can't.
2. **The axiom core** — about two dozen immutable, universal rules, resident in every seat. Few on purpose: the always-on center holds only what must never be a page-fault away.
3. **The preference layers** — personal/architect, project, employer, by scope. The governance hierarchy made literal: each layer tightens what it inherits, never loosens. Constraints tighten downward.
4. **Rules as a cache hierarchy** — resident working set per seat (~1 rule/billion), team-resident union = the canon, paged tail injected by trigger and released.
5. **Guy's test** — the smallest crew whose summed budget covers the required-resident set. Below it, an axiom is homeless and the system is unsafe. Fix it by adding a seat or trimming the core.
6. **Crew size is the capacity dial** — harder project, add seats and hold more resident; easier one, drop a seat and page more. The dial reads in billions of parameters.

The hundred didn't shrink to twenty-four. It got a center. The count is still the union of everything the fleet holds; the structure is the cache that lets a fleet of forgetful machines hold it — a small core that never leaves, and a long tail that arrives exactly when a task reaches for it. Chapter 6 said distribute the canon, don't replicate it. This is the map of the distribution: what stays, what pages, and how few seats it takes to keep every law of the place standing. Guy asked how small the crew could get. The honest answer is a number now, and you can read it off the dial.

Back Matter

Companion media

A two-minute demonstration accompanies these rules: one local Llama 3.1 8B model, the same weights run twice — vanilla on one side, and with these hundred rules standing in front of it on the other. The ruled model quotes a rule cold, catches a hardcoded key and *names the rule it breaks*, and refuses to invent a rule that does not exist. The "best rule × worst rule" podcast series then walks the hundred one pair at a time.

- **Watch and listen — the 100 Rules podcast:** youtu.be/rmMGM460FMw
- **Companion repository (`RULES.md`, CC-BY-4.0):** github.com/edhaynes/eds-rules

Built with Bard. The local models behind the demonstration — and the on-device inference this book keeps holding up as the cheap, private alternative to a frontier API — run on **Bard**, *LLM on the Go*: local models on iPhone, iPad, and Mac.

- **Bard on the App Store:** apps.apple.com/app/bard-llm/id6772813533

Appendix A — Drop-in pre-commit config

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.0
    hooks: [{ id: gitleaks }]
  - repo: https://github.com/Yelp/detect-secrets
    rev: v1.5.0
    hooks: [{ id: detect-secrets, args: ["--baseline",
".secrets.baseline"] }]
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: check-added-large-files
```

```
      - id: check-merge-conflict
      - id: detect-private-key
      - id: end-of-file-fixer
      - id: trailing-whitespace
  # add language-specific linters/formatters below
```

Appendix B — Minimum `.gitignore` for secrets

```
.env
.env.*
!.env.example
*.pem
*.key
*.pfx
*.p12
*.crt
id_rsa*
credentials.json
service-account*.json
.aws/
.azure/
.gcp/
*.sqlite
*.db
```

Appendix C — Adopting the rules per harness

The canonical short form of all one hundred rules lives in the companion repository (`RULES.md` at github.com/edhaynes/eds-rules, CC-BY-4.0).

1. Copy (or submodule) `RULES.md` into your repo.
2. Point your harness at it:
   - **Claude Code:** reference it from `CLAUDE.md` with an `@RULES.md` import.
   - **OpenCode / Codex / others:** reference it from `AGENTS.md`.
   - A symlink from both `CLAUDE.md` and `AGENTS.md` to one canonical file keeps them from drifting.
3. Project-specific files may tighten the rules; they must never loosen them.

4. Edit what you disagree with — the rules are numbered so reviews can cite them ("violates rule 55").

Appendix D — Book number ↔ repository `RULES.md` mapping

The repository groups the rules by topic; the book orders them pedagogically. Same one hundred rules, two orderings. (The repository was consolidated 2026-06-14 — overlapping rules merged, freed slots refilled, and a new *Operating an AI fleet* chapter added; the count held at 100. This table tracks that numbering.)

Book rules	RULES.md rules	Topic
1–14	1–14	Hard rules, the Powell rule, the five-roles charter
15	90	Plan first
16–20	15–19	The crew — Claudius, Jason, Claude, Claudina, Linda
21	91	No flattery
22	20	Go local
23	21	If it can change, it's config (absorbs magic numbers)
24–27	28–31	Architecture thesis, vendor interfaces, search-first, DI
28–30	22–24	No silent fallbacks, startup validation, one config layer

Book rules	RULES.md rules	Topic
31	32	Objects with one job (absorbs SOLID)
32–33	43–44	Storage adapter, on-prem/cloud as config
34–35	25–26	"Use X locally", zero-setup defaults
36	34	The size gauge — files, functions, nesting
37	35	No god classes
38	33	One non-trivial class per file
39	27	Ship the `.env.example`
40	36	Mechanical refactor commits
41	45	Deploy gates never disabled
42	71	Catch specific, never swallow (absorbs cleanup)
43	77	Pin and lock (absorbs vulnerability audits)
44–45	47–48	Health endpoints; container-friendly + least privilege

Book rules	RULES.md rules	Topic
		(stack is a preference)
46	46	Idempotency
47	72	Loud in dev, graceful in prod
48	37	Three platforms, two in CI
49	73	Logger not print (absorbs structured logs)
50	74	AI errors surface; agents don't invent tools
51	38	Path libraries (absorbs LF line endings)
52	78	Stdlib plus one dependency
53–55	39–41	Temp/home/drive, both architectures, no shell-isms
56	42	Time is UTC until it's displayed
57	79	Project-local virtualenvs
58	49	Progress on slow paths

Book rules	RULES.md rules	Topic
59	75	Remote-call timeout, retry, circuit-breaker
60	76	Correlation/trace ID
61	50	Reversible, idempotent migrations
62–65	51–54	Hooks first, rotate first, never copy a secret, scan every boundary
66	62	The quality rubric (you get what you inspect)
67–69	63–65	Tests ship with logic, contract first, 100% coverage
70–72	66–68	Correctness over speed, coverage ratchet, full regression
73	69	Latency/throughput budget
74	70	No network in unit tests
75	55	Fix the hook that missed it
76	56	Immutable tags (absorbs fetch-before-tag, push by name)

Book rules	RULES.md rules	Topic
77	57	Versions only move forward
78	86	Persist decisions same-commit (absorbs immutable ADRs)
79	58	One canonical version home
80	87	Bug/feature ledgers
81	59	Build numbers
82	92	Verbatim errors, diffs not prose
83	60	Changelog in the bump commit
84	88	Plan status tracking
85	61	Version displayed everywhere
86–88	80–82	Lint every commit, dead-code sweep, post-release cleanup
89	89	README regeneration
90–91	83–84	No commented-out code, no orphan TODOs
92	85	A living `STATE.md`
93–100	93–100	Operating an AI fleet — sizing

Book rules	RULES.md rules	Topic
		law, slicing, context ceilings, verify-then-escalate

Appendix E — Bending the model: the three ways to retrain

The foreword names three ways to move a model off its C-grade defaults. In order of cost:

Path	Cost	Robustness	Who it's for
Train from scratch	Millions	Highest	Labs, not you
Tune open weights	Days to weeks — the dataset is the work	Medium-high — shapes style and defaults; process discipline still needs gates	Anyone with a capable machine
Standing rules in the prompt	Free, immediate	Lowest — instructions dilute	Everyone; start here

Training from scratch is listed for completeness. If you have to ask what it costs, it is the wrong path.

Tuning open weights is more approachable than its reputation, with one honest caveat up front: tuning shapes the model's defaults and style; it does not install procedural discipline. A tuned model writes in your idiom — it still won't refuse to commit on a red suite unless a gate stops it. Take a strong open-weight coding model and fine-tune it with LoRA or QLoRA — techniques that adjust a small fraction of the model's weights, so the hardware stays reasonable: a single consumer GPU handles the small-and-mid sizes, and one

large unified-memory machine handles the 70-billion-parameter class. The established open-source tooling (Hugging Face PEFT, Axolotl, Unsloth) makes the mechanics a configuration file, not a research project. The hard part — and the entire point — is the dataset: a few thousand *curated* examples of your standard (diffs that passed your review, before-and-after refactors, rules-compliant modules) beat millions of scraped files. Quantity got the model into this condition; only quality gets it out. Grade the tuned model against your rubric, not against public benchmarks.

Your starting point matters too. A few less famous model families are trained on curated, governed corpora rather than the raw scrape — IBM's Granite Code models are the notable example: Apache-2.0 licensed, trained on license-filtered data with documented provenance, in over a hundred programming languages — a pipeline that deduplicates aggressively, scans for malware, and even redacts keys and passwords from the training data itself. Secret hygiene all the way down. And InstructLab, the open-source fine-tuning project, is exactly this appendix's second path packaged for people who are not researchers: it turns "tune open weights" into a command-line workflow fed by your own curated examples. A model that begins license-clean and provenance-tracked has less of the internet's average to unlearn.

Standing rules in the prompt is this book: the hundred rules, dropped into the file your harness reads on every request (Appendix C shows how, per harness). It costs nothing and works today, on any model, including the closed ones you cannot tune. Its weakness is honesty itself: long sessions dilute instructions, and a model can drift from a rule it read an hour ago. That is why the rules lean so hard on gates that do not depend on the model's memory — pre-commit hooks (Appendix A), scans, and tests run whether or not anyone remembered the rule. Not foolproof is acceptable when the hooks are.

Appendix F — The axiom core and the layers

Chapter 7 gives the hundred a shape: a small immutable core every seat carries, wrapped in layers of preference paged in on demand. This appendix is the reference behind that chapter — the core enumerated, the layers named, and one crew's capacity worked out.

The axiom core

The rules whose violation is indefensible in any shop. Few by design: the core is the always-resident center, so it holds only what must never be a page-fault away. Short form below; the full statements are the rules themselves, by topic, in the companion `RULES.md`.

Axiom	In one line
Secret scan before ship	Scan before every commit, push, and deploy. No scan, no ship — any agent, any target.
Never hardcode secrets	Found one → stop and flag; never propagate, even temporarily.
Destruction needs a human	No delete, drop, destructive command, force-push, or history rewrite without explicit human confirmation.
Distrust every external input	Validate and constrain at the boundary; never interpolate untrusted data into SQL, a shell, HTML, or a deserializer.
The 90% rule (Powell)	Get 90% of the information, then decide; below 90%, ask. Route by the same bar.
Autonomy bounded by version control	An agent only writes inside a git repo with a synced remote.

Axiom	In one line
	No recoverable history, no autonomy.
Least privilege by default	Narrowest scope that works; no wildcard perms, no shared admin keys; expire and rotate.
Green before commit, healthy before handover	Never commit on red; never present a service as done without watching it answer.
Secret-scan hooks from day one	Hooks installed before the first commit; `.gitignore` covers keys/certs/`.env*`.
Zero hardcoded values	Nothing that can change is hardcoded; no magic numbers.
A touched secret is burned	Rotate first, clean history second; surface a leaked secret only to its owner.
Plan first for non-trivial work	State approach and files before editing; never silently change scope.
Fail fast	Crash loudly at startup on bad config or missing deps; never limp along degraded.
Inspect, don't expect — grade to a rubric	Test-driven and graded; 90% to work, 95% to publish.
Tests with logic; regression first	New logic ships with tests; bug fixes ship a failing-first regression test.
One purpose per commit/deploy	No "while I'm in there" fixes; mechanical refactors stand alone.

Axiom	In one line
Correctness over speed	The delay to reach verified behavior and full coverage is acceptable and expected.
Push early, push always	Working code lands on main often; uncommitted work is a liability.
Contract first	Freeze the interface, write tests against it, then implement.
Disclose every dependency	Never add one without name, purpose, license, maintenance, and platform support.
No OS assumptions; script everything; headless	Cross-platform primitives; assume no display and nobody at a prompt.
No flattery, no yes-manning	Agree only when it carries information; defend your reasoning before capitulating.
Verbatim errors; diffs; surfaced assumptions	Quote errors exactly; show diffs not prose; state assumptions.
100% line + branch coverage	Every branch exercised and asserted; the runner fails under it.

Two dozen rules. Hold every one of them resident and you need roughly two dozen billion parameters of summed crew budget — the arithmetic behind Guy's test.

The layers, by scope

Everything outside the core is a preference, scoped. Constraints tighten downward; a layer may make an inherited rule stricter, never looser.

- **Personal / architect preferences** — engineering opinions and the crew: objects with one job, file and function size ceilings, one class per file, the config-precedence stack, the five personas, and which model binds to each seat.
- **Project preferences** — what one codebase needs: its migration rules, its schemas, its domain constraints — paged in for that project, absent everywhere else.
- **Employer preferences** — the shop's house standard. Mine: Podman over Docker, UBI base images, OpenShift. Correct for me, hardcoded to nobody else.

A worked crew — capacity and Guy's test

One crew sized for a cloud-native, multi-platform project. Budget is the sizing-law allowance (≈1 rule per billion parameters); *resident* is what the seat pins; *paged* is the tail it fetches on demand.

Seat	Model	Budget	Resident	Paged
Jason	7B coder	7	7	35
Claude	14B	14	14	33
Claudius	frontier	200	50	0

Team-resident union: every active rule (50 here) is held by some seat — the canon, distributed. **Guy's test:** the 24-axiom core needs ≈24B of summed budget to hold every axiom resident. A frontier-free pair (8B + 14B = 21B) falls three short and is unsafe; add a frontier seat, or trim the core to ≤21, and every axiom has a home. Crew size is the capacity dial — add seats for harder projects, drop one and let more rules page for easier ones.

Glossary

- **The crew** — five fixed AI roles (Jason, Linda, Claude, Claudina, Claudius) plus one human; model bindings are configuration, never canon.
- **The Powell rule** — get 90% of the information you need to make a decision, then make the decision; below 90% certainty, ask the human more questions. Never guess ahead; never stall gathering past 90%.
- **The quality bar** — every project keeps a rubric grading the software; 90% (solid A–) is the working bar, polish takes it to 95%, nothing publishes below 95%.
- **Trust boundary** — anything beyond the local working tree: the remote repo, a registry, a cluster, a host you don't own. Everything that crosses one gets scanned.
- **Go local** — the standing instruction that rebinds every persona to a local model backend; same roles, same rules, different engine.
- **Chapter card** — the rule checklist closing each chapter.
- **Axiom core** — the ~two dozen immutable, universal rules whose violation is indefensible anywhere; carried resident by every seat. The smallest set that must never be missed (Appendix F lists them).
- **Preference layer** — a scoped set of rules outside the core (personal/architect, project, employer); a layer may tighten an inherited rule, never loosen it. Constraints tighten downward.
- **Cache hierarchy** — the ruleset treated like memory: a per-seat *resident* working set (≈1 rule per billion parameters), the *team-resident union* that equals the canon, and a *paged tail* injected on demand when a task touches it.
- **Capacity dial** — crew size as the lever on resident capacity: summed per-seat budgets. Add seats to hold more rules standing; drop one to page more.
- **Guy's test** — the smallest crew whose summed budget covers the required-resident set (axioms plus must-hold project rules). Below it, an axiom is homeless and the system is unsafe.